



SYSTÈMES D'EXPLOITATION 2

Cycle Préparatoire Intégré 2^{ème} Année (CPI 2)

Ali OTHMAN

A.U 2024/2025

IDENTIFICATION DU MODULE

- **Prérequis :**

- Système d'exploitation 1.

- **Objectifs :**

- Présenter les techniques de gestion des ressources.
- Présenter les notions de processus et threads et les techniques de gestion des processus.
- Présenter les techniques de gestion de la mémoire principale.

- **Volume horaire :** 30h CI / 15h TP.

- **Evaluation :** 25% DS + 25% Examen TP + 50% Examen.

PLAN

- Mécanismes de base des systèmes d'exploitation
- Gestion des ressources
- Gestion des processus
- Gestion de la mémoire principale
- Installation et paramétrage de systèmes

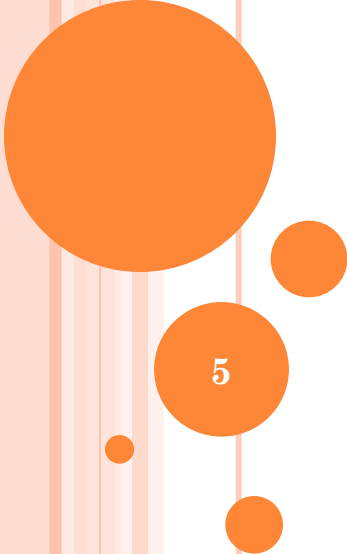
Windows et Unix en mode virtuel et non virtuel

PLAN Détaillé :

- Mécanismes de base des systèmes d'exploitation
- Gestion des ressources :
 - Mémoire secondaire
 - Système de gestion des fichiers
- Gestion des processus :
 - Processus et Threads
 - Ordonnancement
- Gestion de la mémoire principale :
 - Mémoire virtuelle
 - Pagination et segmentation

PLANNING

Séances	Chapitres
Séances 1-3	Gestion des ressources
Séances 4-5	TD N°1
Séances 6-10	Gestion des processus
Séances 11-12	TD N°2
Séances 13-17	Gestion de la mémoire principale
Séances 18-19	TD N°3
Séance 20	Installation et paramétrage de systèmes Windows et Unix en mode virtuel et non virtuel



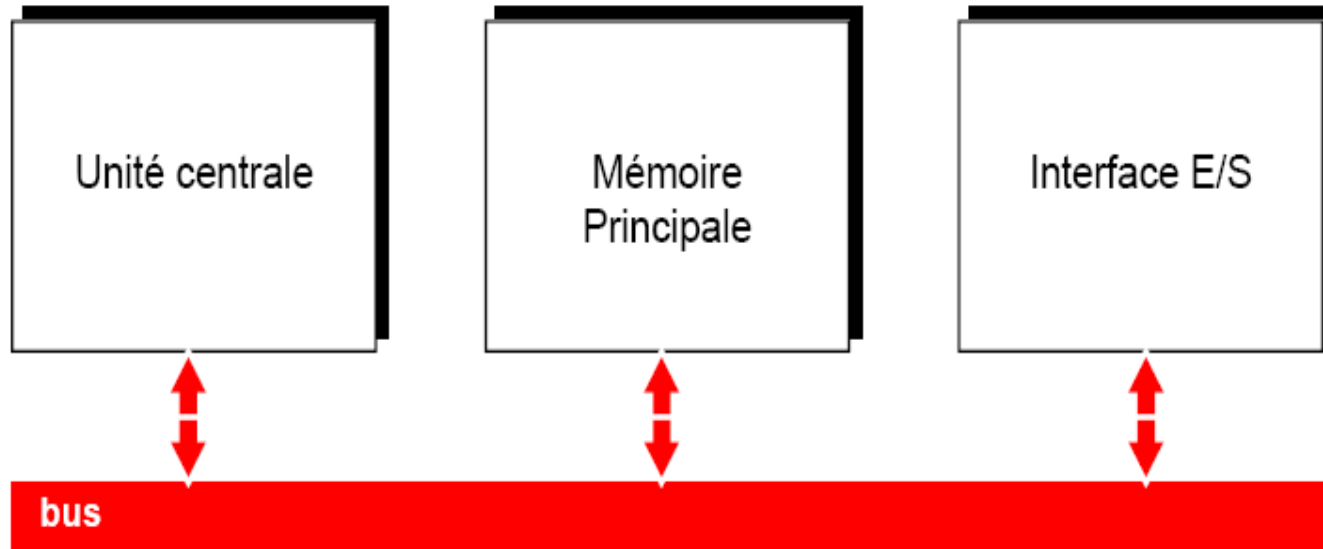
MÉCANISMES DE BASE DES SYSTÈMES D'EXPLOITATION

5

RAPPEL :

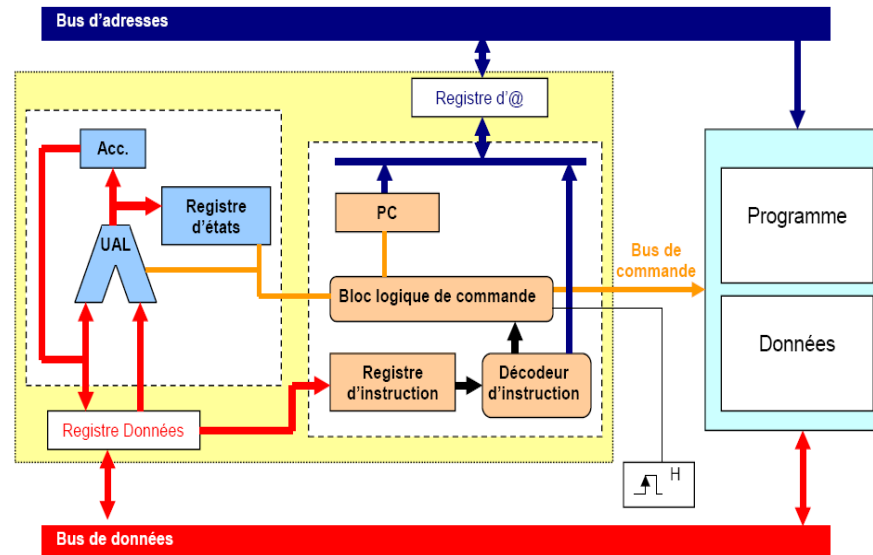
MODÈLE DE LA MACHINE UNIVERSELLE

- John Von Neumann (1946) est à l'origine d'un modèle de machine universelle de traitement programmé de l'information.



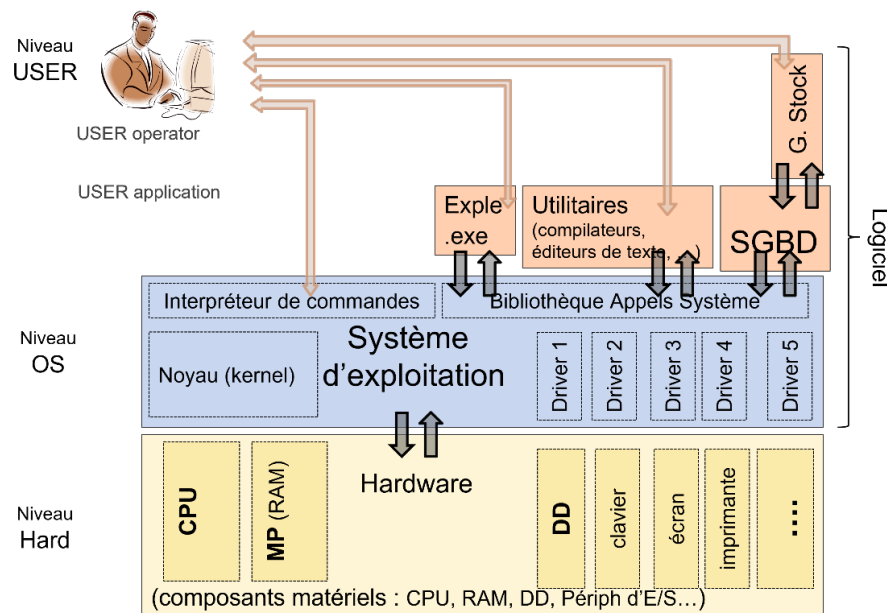
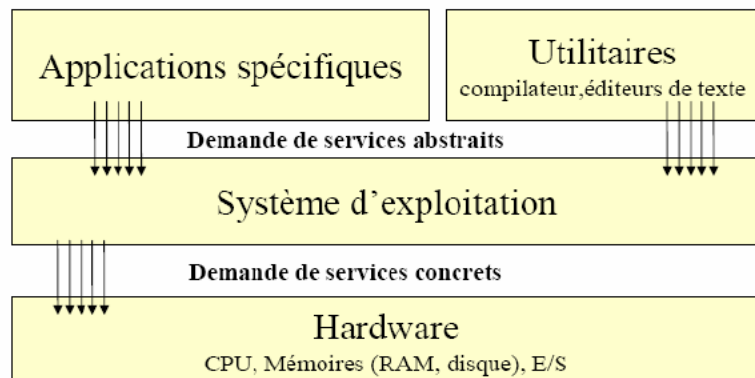
RAPPEL :

COMPOSANTS DE BASE MATÉRIELS



- **L'unité Centrale (UC / UCT ou CPU)** : exécute le code (instructions)
- **La mémoire principale (centrale) (MP = RAM)** : contient les données et les instructions des programmes en cours d'exécution.
- **Les interfaces d'entrées/sorties (E/S)** : assurent la communication entre le microprocesseur et les périphériques.
- **Les bus** : fait la connexion entre le CPU, le RAM et les périphériques E/S.

RAPPEL : SYSTÈME D'EXPLOITATION (SE)



- Le SE est une couche logicielle qui permet de masquer la complexité du matériel et de proposer des instructions plus simples à l'utilisateur (**Machine virtuelle**).
- Le SE est un ensemble de programmes qui contrôlent les ressources de l'ordinateur (**gestion et partage de ressources**).
- Deux dimensions de partage : temps et espace.
- Les ressources d'une machine :
 - Physique** : CPU, MP, E/S, mémoires de masse ou secondaires (disque dur, ...)
 - Logique (virtuelle)** : fichiers, bases de données partagés, canaux de communications logiques, ...

RAPPEL :

SYSTÈME D'EXPLOITATION (SE)

- Les ressources logiques sont bâties par le logiciel sur les ressources physiques.
- Les programmes sont classées en :
 - Programmes systèmes (routines systèmes) : gèrent le fonctionnement de l'ordinateur.
 - Programmes d'applications : exécutent le travail demandé par les utilisateurs.
- Un **appel système** est une fonction fournie par la bibliothèque des primitives système d'un SE et utilisée par les programmes s'exécutant dans l'espace utilisateur (*fork, wait, pthread_create, ...*).
- Mode d'exécution du processeur :
 - Mode superviseur (noyau) :
 - Actions privilégiées
 - C'est le mode utilisé par le SE pour s'exécuter
 - Mode utilisateur : programmes utilisateur
 - Restrictions d'accès aux ressources,
 - C'est le mode utilisé pour les applications

RAPPEL :

SYSTÈME D'EXPLOITATION (SE) : COMPOSANTS

- **Le noyau (kernel)** : permet de gérer les ressources matérielles et de fournir aux programmes une interface uniforme pour l'accès à ces ressources (processeur, mémoire, fichiers, E/S). Il représente les fonctions fondamentales du système d'exploitation.
- **L'interpréteur de commandes (Shell)** : permet la communication avec le système d'exploitation par l'intermédiaire des commandes.
- **Le système de fichiers** : permet d'enregistrer les fichiers dans le disque sous forme d'une arborescence.
- **Les pilotes** : Les pilotes sont fournis par l'auteur du système d'exploitation ou le fabricant du périphérique.

RAPPEL : FONCTIONS DE BASE D'UN SYSTÈME D'EXPLOITATION (SE) : GESTION DES RESSOURCES

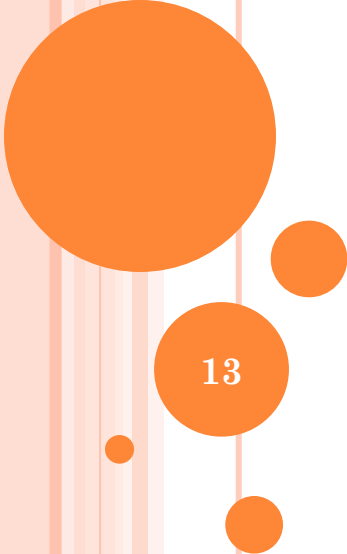
- **Gestion des processeurs:** gérer l'allocation des processeurs entre les différents programmes grâce à un algorithme d'ordonnancement (*gestion de processus*).
- **Gestion des mémoires:** gérer l'espace mémoire alloué à chaque processus (mémoire centrale (MC)).
- **Gestion des fichiers:** gérer l'organisation des fichiers (système de gestion de fichiers (SGF)) et du disque dur.
- **Gestion des entrées-sorties :** gérer l'accès aux périphériques via les pilotes.

RAPPEL :

PRINCIPAUX SYSTÈMES D'EXPLOITATION (SE)

- UNIX (1971)
- Windows (1985)
- Linux (1991)
- MacOS (2001)
- ...
- iOS (2007)
- Android (2008)
- ...





GESTION DES RESSOURCES

Ressources physiques et logiques

13

GESTION DES RESSOURCES (LOGIQUE)

SYSTÈME DE GESTION DES FICHIERS (SGF)

- Le SGF est la partie la plus visible du SE.
- Le SGF définit la structure de données utilisé par le SE.
- Le SGF permet d'allouer des mémoires de masse ainsi que l'accès aux données stockées.
- Le SGF permet de gérer les fichiers et offrir des primitives pour les manipuler.
- Il offre à l'utilisateur une unité de stockage indépendante des propriétés physiques des supports de conservation: **le fichier**
 - Fichier logique (vue de l'utilisateur)
 - Fichier physique
- Il assure la correspondance entre le fichier logique et le fichier physique.

GESTION DES RESSOURCES (LOGIQUE)

TACHES D'UN SGF

- Fournir un mécanisme d'accès et de stockage des données et programmes
- Gérer les accès concurrents aux fichiers (partage)
- Gérer les droits d'accès aux fichiers
- Gérer l'espace sur le disque
- Donner des utilitaires pour le diagnostique, la récupération en cas d'erreur, l'organisation des fichiers, ...
- Tout SGF intègre la notion de fichier et de répertoire

GESTION DES RESSOURCES (LOGIQUE)

EXEMPLES DES SYSTÈMES DES FICHIERS

- EXT2 / EXT3 / EXT4 → **UNIX**

- ReiserFS

- AppleFS

- NTFS →
- FAT32 →

} **Windows**

- ISO9660

- UDF

GESTION DES RESSOURCES (LOGIQUE)

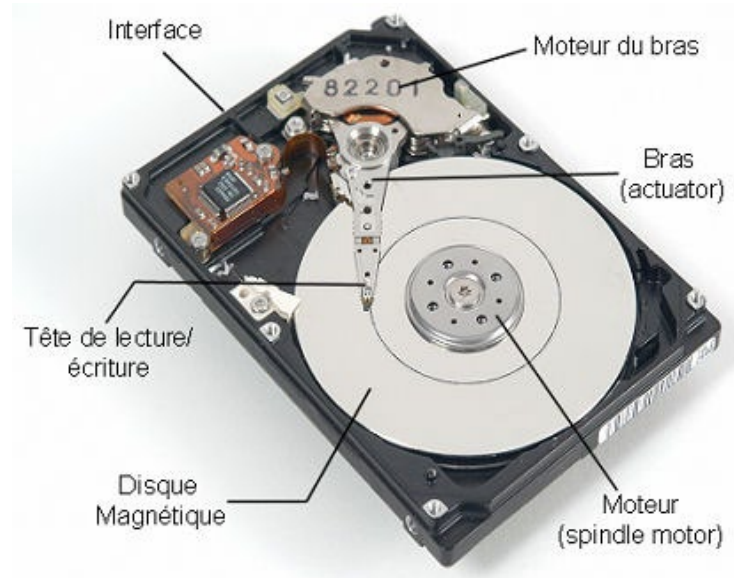
SYSTÈME DE GESTION DES FICHIERS (SGF)

- **Un fichier logique** : un type de données standard défini dans les langages de programmation sur lequel un certain nombre d'opérations peuvent être réalisées
 - Création, ouverture, fermeture, destruction
 - Les opérations de création ou d'ouverture effectuent la liaison du fichier logique avec le fichier physique
- **Un fichier physique** : correspond à l'entité allouée sur le support permanent et contient physiquement les enregistrements définis dans le fichier logique. Il est constitué d'ensemble de blocs physiques (secteurs).

GESTION DES RESSOURCES (PHYSIQUE)

MÉMOIRE DE MASSE : LE DISQUE DUR

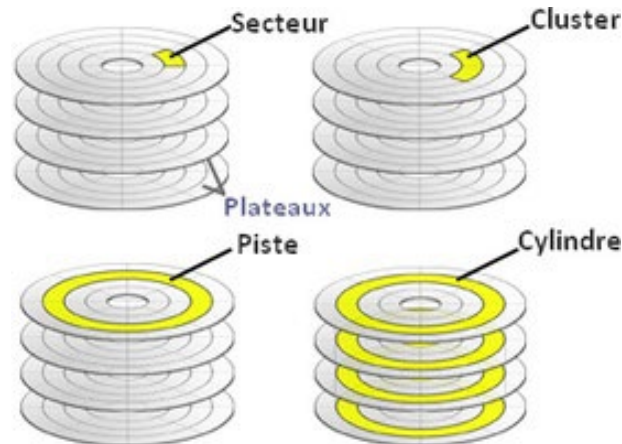
- Le disque dur est un support de stockage servant à conserver les données de manière permanente, contrairement à la mémoire vive, qui s'efface totalement dès la mise hors tension de l'ordinateur.
- Un disque dur est constitué de plusieurs plateaux (2 faces par plateau) en métal, en verre ou en céramique empilés les uns sur les autres.



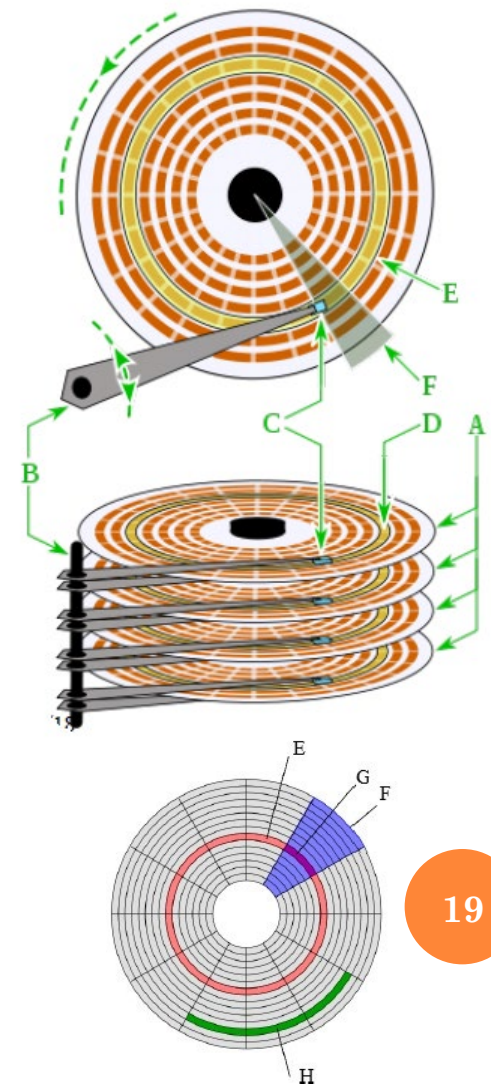
GESTION DES RESSOURCES (PHYSIQUE)

MÉMOIRE DE MASSE : LE DISQUE DUR

- **Pistes (E)** : des cercles concentriques sur les différents plateaux et contenant les données écrites.
- **Secteurs (G)** : des quartiers (entre deux rayons) de taille fixe formant les pistes pour le stockage des données. Généralement de taille 512 octets.
- **Cylindres (D)** : l'ensemble des pistes portant le même numéro et appartenant aux différents plateaux.
- **Bloc (Cluster) ou Unité d'allocation (H)** : la zone minimale que peut occuper un fichier sur le disque. Un fichier occupera plusieurs secteurs (un cluster) même si sa taille est très petite.



Plateaux (A)
Bras (B)
Têtes de lecture (C)
Secteurs géométriques (F)



GESTION DES RESSOURCES (PHYSIQUE)

MÉMOIRE DE MASSE : LE DISQUE DUR

Paramètres	Disquette IBM 3.5"	Disque dur WD
Nb de plateaux	1	10601
Nb de pistes par face	80	12
Nb de secteurs par piste	18	281
Nb de secteurs par disque	2880	71 493 144
Octets par secteur	512	512
Capacité du disque	1440 ko	34.1 Go

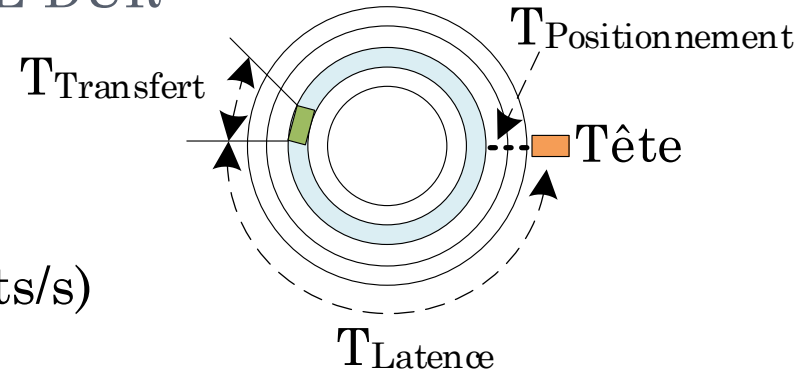
- Plus les clusters sont gros, plus il y aura d'espace disque gaspillé pour entreposer des fichiers.
 - Si les clusters ont une taille de 512 octets, combien d'octets seront gaspillés pour un fichier de 1000 octets ?
 - **24 octets seront gaspillés.**
 - Si les clusters ont une taille de 2048 octets, combien d'octets seront gaspillés pour ce même fichier de 1000 octets ?
 - **1048 octets seront gaspillés.**

GESTION DES RESSOURCES (PHYSIQUE)

MÉMOIRE DE MASSE : LE DISQUE DUR

○ Facteurs de performance :

- Capacité (octets)
- Débit en écriture / lecture (octets/s)
- Temps d'accès (TA) = S + R + T
 - Temps de positionnement (déplacement) S : déplacement radial de la tête de lecture vers la piste contenant le bloc de données voulu.
 - Temps de rotation (latence) R : passage du bloc voulu sous la tête par rotation des plateaux.
 - R = nombre de tour / vitesse de rotation.
 - R moyen = nombre de tour / (2*vitesse de rotation).
 - Temps de transfert T : temps effectif de lecture des données vers (ou depuis) le bloc voulu par rotation des plateaux.
 - T = 1/(nombre de secteurs*vitesse de rotation).



GESTION DES RESSOURCES (PHYSIQUE)

MÉMOIRE DE MASSE : LE DISQUE DUR

- Objectif : réduire le temps d'accès moyen → **Ordonnancement**
- Comportement du pilote de périphérique :
 - Recevoir les requêtes émanant du logiciel
 - Choix de la prochaine requête à traiter
 - Vérification de la validité des paramètres de la requête
 - Détermination des opérations que le contrôleur doit exécuter
 - Traduction des requêtes d'E/S en termes concrets

GESTION DES RESSOURCES (PHYSIQUE)

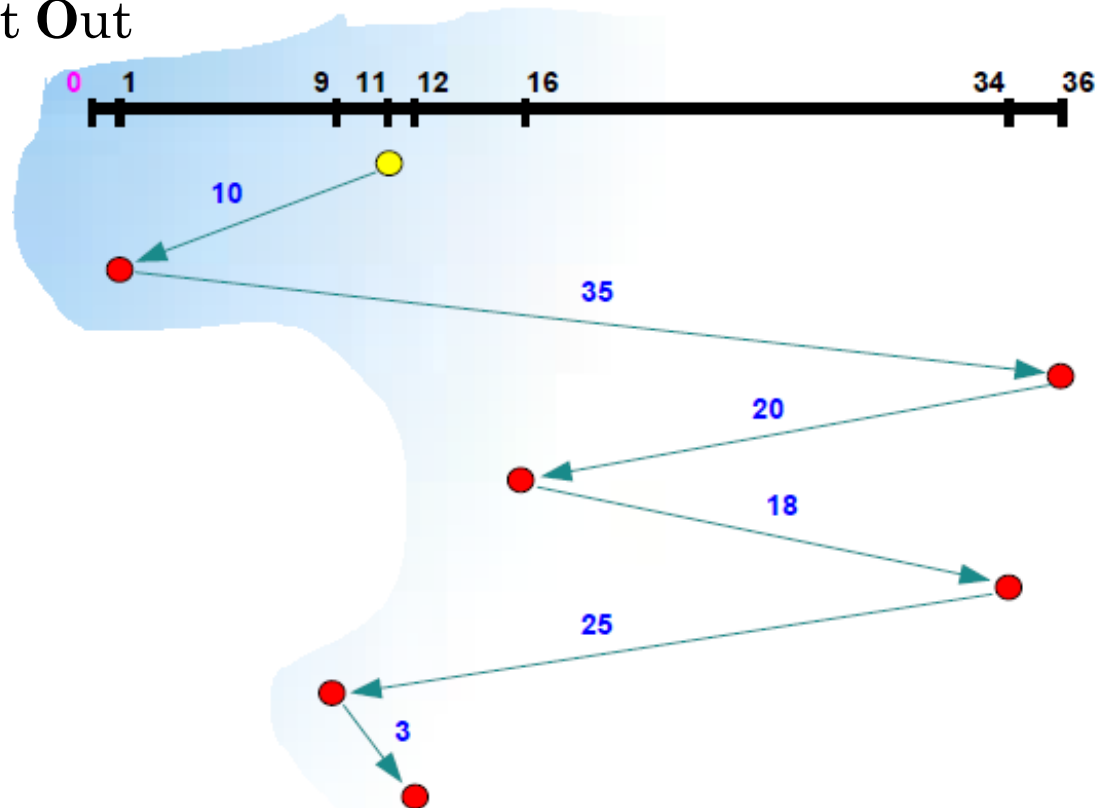
MÉMOIRE DE MASSE : ORDONNANCEMENT

- Algorithmes d'ordonnancement :
 - FIFO
 - SSTF
 - SCAN et C-SCAN
 - LOOK et C-LOOK
 - F-SCAN
 - N-step SCAN
- La performance dépend du nombre et du type des requêtes.
- Les disques durs modernes assurent souvent eux-mêmes l'ordonnancement

GESTION DES RESSOURCES (PHYSIQUE)

MÉMOIRE DE MASSE : ORDONNANCEMENT

- **FIFO : First In First Out**

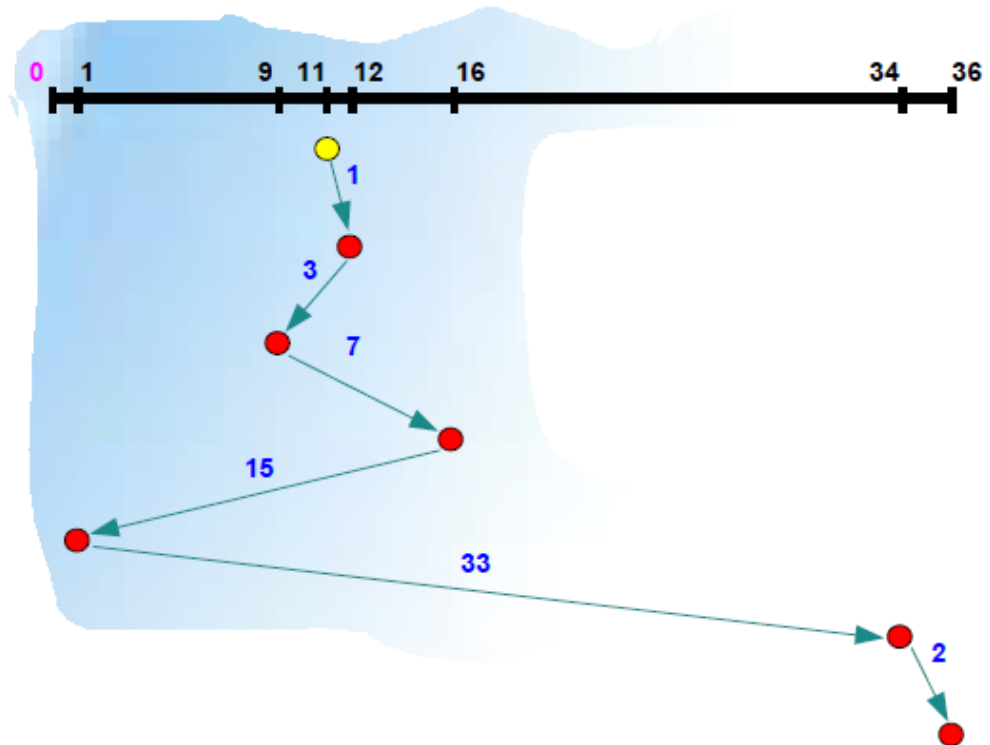


- Requêtes : 1, 36, 16, 34, 9, 12
- Ordre de service : 1, 36, 16, 34, 9, 12
- Nb de cylindres parcourus : 111

GESTION DES RESSOURCES (PHYSIQUE)

MÉMOIRE DE MASSE : ORDONNANCEMENT

- **SSTF : Shortest Seek Time First**

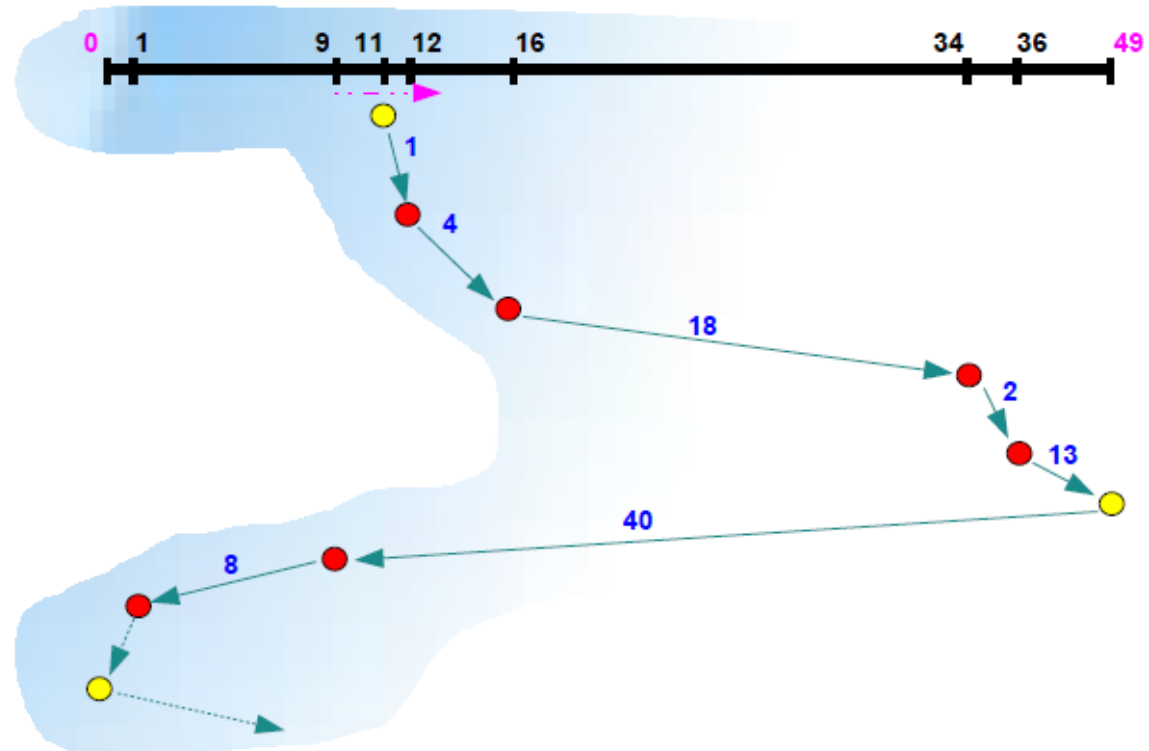


- Requêtes : 1, 36, 16, 34, 9, 12
- Ordre de service : 12, 9, 16, 1, 34, 36
- Nb de cylindres parcourus : 61

GESTION DES RESSOURCES (PHYSIQUE)

MÉMOIRE DE MASSE : ORDONNANCEMENT

- **SCAN** : algorithme de l'ascenseur

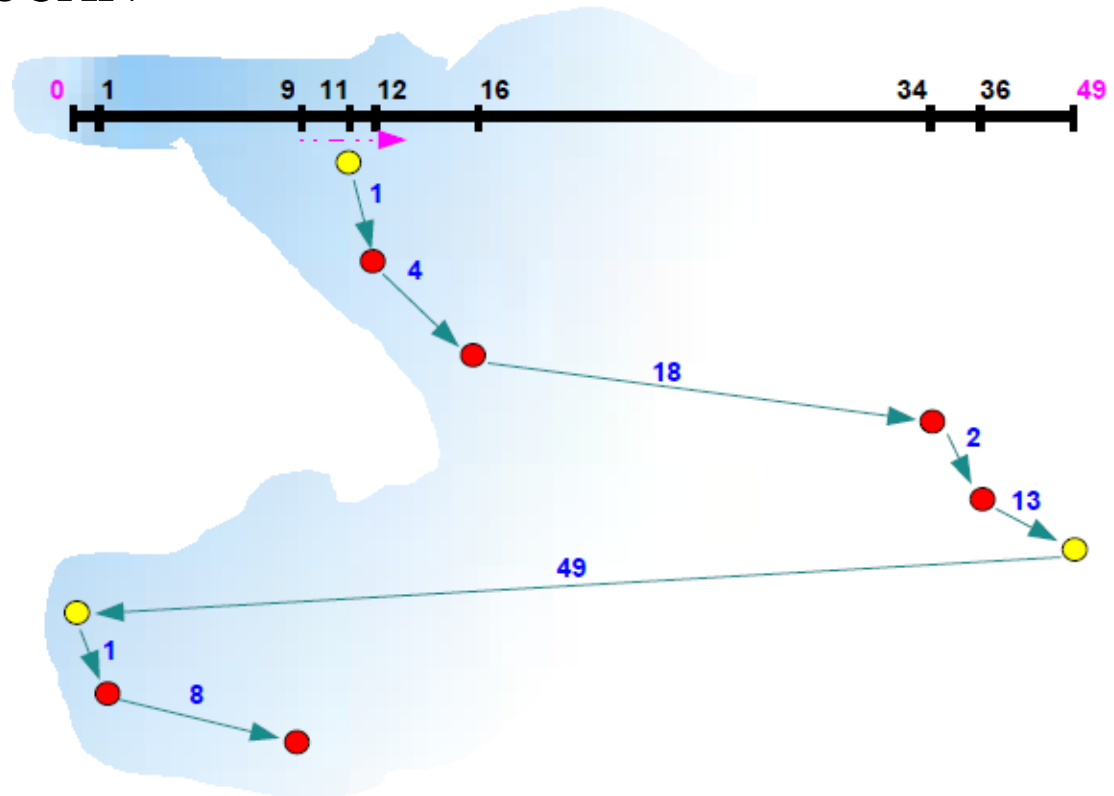


- Requêtes : 1, 36, 16, 34, 9, 12
- Ordre de service : 12, 16, 34, 36, 9, 1
- Nb de cylindres parcourus : 86

GESTION DES RESSOURCES (PHYSIQUE)

MÉMOIRE DE MASSE : ORDONNANCEMENT

- **C-SCAN : Circular SCAN**

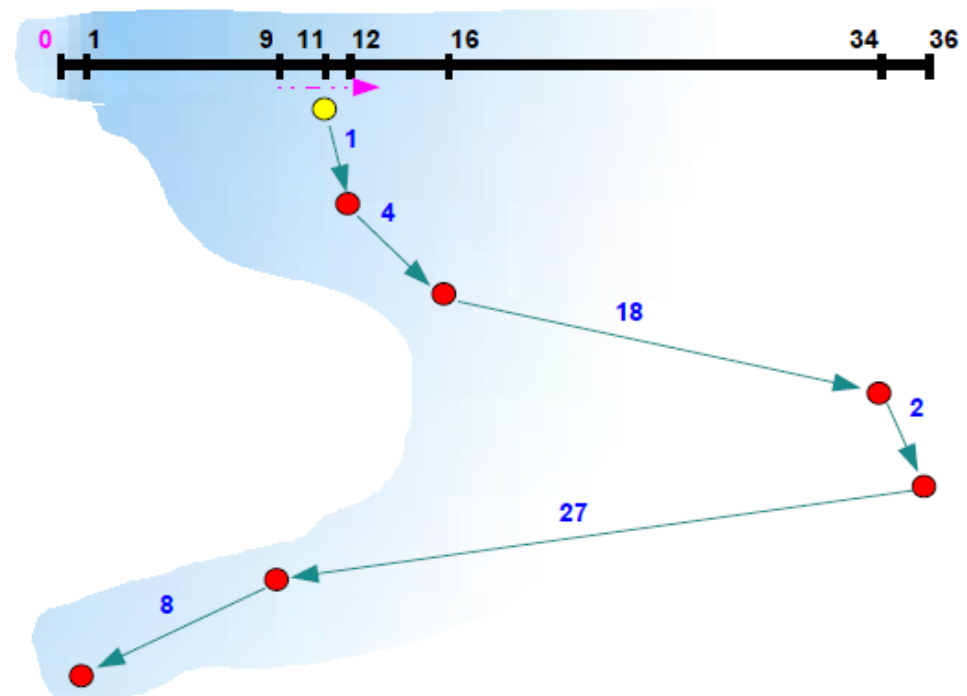


- Requêtes : 1, 36, 16, 34, 9, 12
- Ordre de service : 12, 16, 34, 36, 1, 9
- Nb de cylindres parcourus : 96

GESTION DES RESSOURCES (PHYSIQUE)

MÉMOIRE DE MASSE : ORDONNANCEMENT

- **LOOK** : SCAN avec changement de direction lorsqu'il n'y a plus de requêtes en avant de la tête

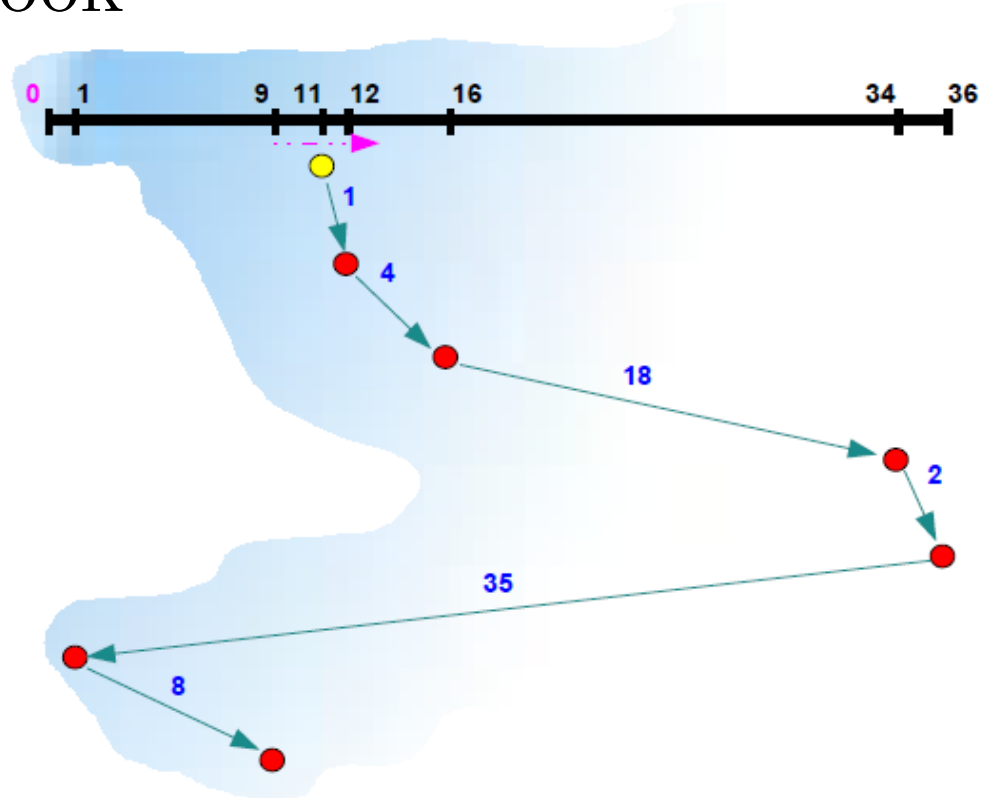


- Requêtes : 1, 36, 16, 34, 9, 12
- Ordre de service : 12, 16, 34, 36, 9, 1
- Nb de cylindres parcourus : 60

GESTION DES RESSOURCES (PHYSIQUE)

MÉMOIRE DE MASSE : ORDONNANCEMENT

- **C-LOOK : Circular LOOK**



- Requêtes : 1, 36, 16, 34, 9, 12
- Ordre de service : 12, 16, 34, 36, 1, 9
- Nb de cylindres parcourus : 68

GESTION DES RESSOURCES (PHYSIQUE)

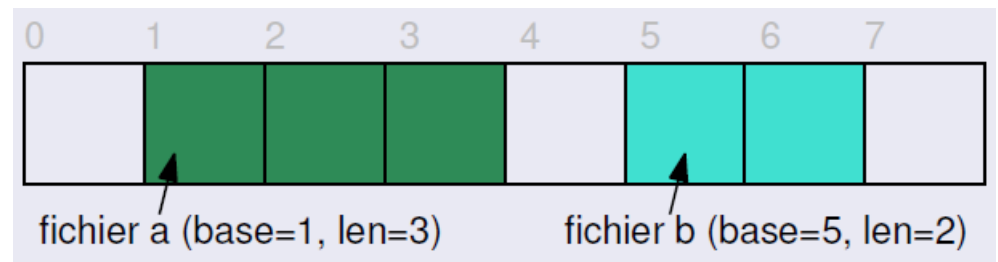
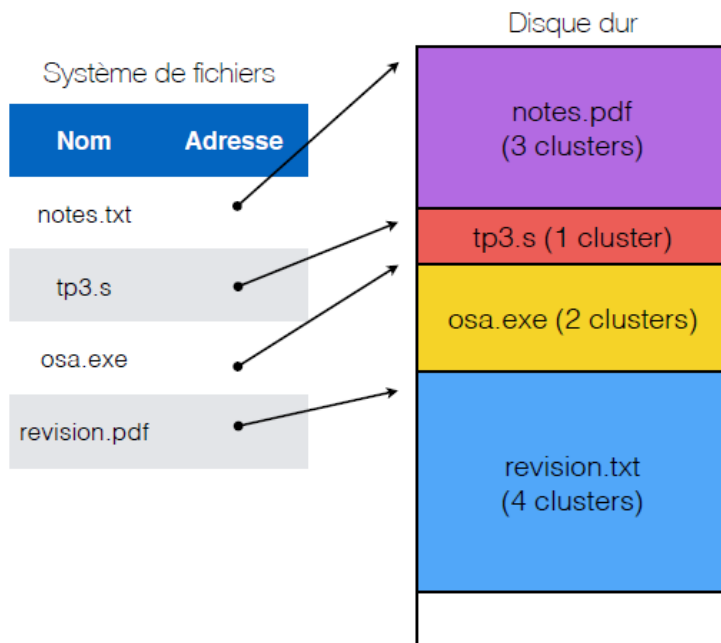
MÉMOIRE DE MASSE : MÉTHODE D'ALLOCATION

- Méthodes d'allocation de la mémoire secondaire :
 - **Allocation contiguë** : le fichier est enregistré sur des blocs consécutifs.
 - **Allocation chaînée** : le fichier est divisé en plusieurs blocs enregistrés à des endroits espacés et reliés avec des liens.
 - **Allocation chaînée + table d'allocation (parfois nommée allocation indexée 1^{ère} variante)** : les blocs sont indépendants et ayant des numéros (bloc index) conservés dans une structure statique ou dynamique.
 - **Allocation indexée (2^{ème} variante) (i-nodes)** : à chaque fichier est associé une structure de données contenant les attributs et les adresses des blocs.

GESTION DES RESSOURCES (PHYSIQUE)

MÉMOIRE DE MASSE : MÉTHODE D'ALLOCATION

- **Allocation contiguë :**
- Principe : *un fichier = une suite de blocs consécutifs*
 - Pour chaque fichier à enregistrer, le système recherche une zone suffisamment grande pour accueillir le fichier.
 - Info nécessaire : N° du 1^{er} bloc, taille du fichier



GESTION DES RESSOURCES (PHYSIQUE)

MÉMOIRE DE MASSE : MÉTHODE D'ALLOCATION

- **Allocation contiguë :**
- Avantages :
 - Implémentation simple, accès séquentiel et arbitraire efficaces.
- Inconvénients :
 - Difficulté de prévoir la taille qu'il faut réserver pour un fichier. En plus, un fichier est amené à augmenter. → obligation de connaître la taille du fichier à l'avance
 - Perte d'espace (prévoir plus d'espace → risque de ne pas l'occuper, prévoir moins d'espace → risque d'être insuffisant).
 - Fragmentation (externe) : apparition des trous (blocs vides) après suppression d'un fichier dont la taille ne suffit pas pour allouer un autre fichier.

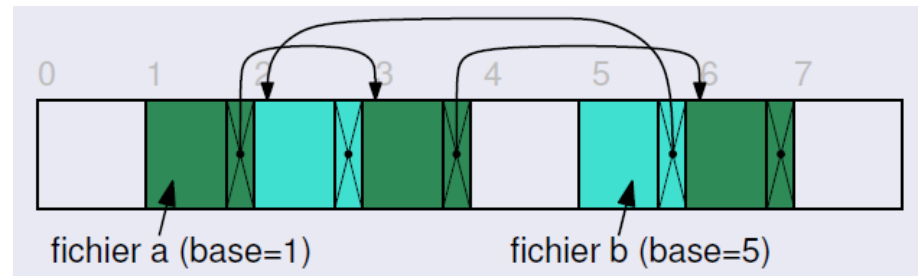
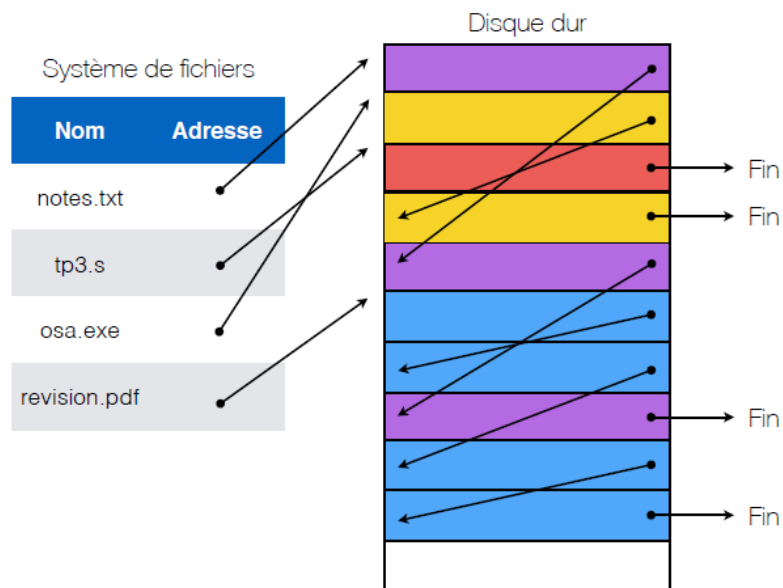
GESTION DES RESSOURCES (PHYSIQUE)

MÉMOIRE DE MASSE : MÉTHODE D'ALLOCATION

○ Allocation chaînée :

○ Principe : un fichier = une liste chaînée de blocs

- Le premier mot de chaque bloc indique le numéro de bloc suivant, le reste du bloc contient les données.
- Un fichier peut être éparpillé sur le disque puisque chaque bloc permet de retrouver le bloc suivant.
- Info nécessaire : N° du 1^{er} bloc



GESTION DES RESSOURCES (PHYSIQUE)

MÉMOIRE DE MASSE : MÉTHODE D'ALLOCATION

- **Allocation chaînée :**
- **Avantages :**
 - Taille dynamique, pas de fragmentation, accès séquentiel efficace.
- **Inconvénients :**
 - Accès arbitraire inefficace : pour accéder au bloc n , le système doit démarrer au début et lire les $n-1$ blocs précédents, un par un.
 - En cas de bug ou crash, la perte du chaînage entraîne la perte de tout le reste du fichier.

GESTION DES RESSOURCES (PHYSIQUE)

MÉMOIRE DE MASSE : MÉTHODE D'ALLOCATION

- Allocation chaînée + table de mémoire (allocation indexée 1) :
- Principe : séparer le chaînage et le contenu du fichier
 - FAT** (File Allocation Table) : tableau de pointeurs (1 case par bloc).
 - Chaque case = N° de bloc suivant.
 - Cette table s'appelle la **FAT** (File Allocation Table) et elle doit être en mémoire RAM pour réduire le temps d'accès.

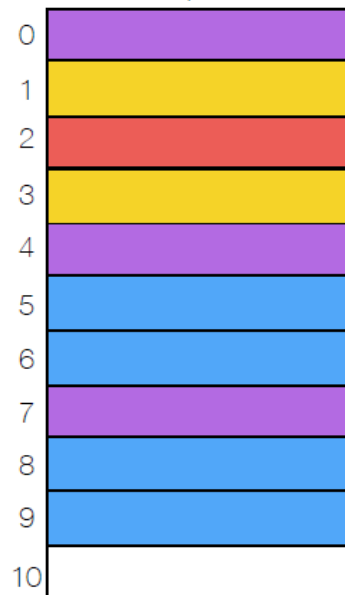
Table d'allocation (FAT)

Cluster	Cluster suivant
0	4
1	3
2	-1
3	-1
4	7
5	6
6	8
7	-1
8	9
9	-1
10	-1

Table de répertoire

Nom	1er cluster
notes.txt	0
tp3.s	2
osa.exe	1
revision.pdf	5

Disque dur



-1 → fin du fichier

0 → le cluster n'a jamais été utilisé

GESTION DES RESSOURCES (PHYSIQUE)

MÉMOIRE DE MASSE : MÉTHODE D'ALLOCATION

- **Allocation chaînée + table de mémoire (allocation indexée 1) :**
- Avantages :
 - FAT mise en cache en RAM → accès arbitraire efficace.
 - Meilleure robustesse
 - Redondance → plusieurs exemplaires sur le disque.
- Inconvénients :
 - Consommation mémoire : FAT32 (32 bits d'adressage)
 - Disque de 20 Go, blocs de 1 ko → la taille de FAT est de 80 Mo
 - Disque de 1 Go, blocs de 1 ko → la taille de FAT est de 4 Mo

GESTION DES RESSOURCES (PHYSIQUE)

MÉMOIRE DE MASSE : MÉTHODE D'ALLOCATION (EX 1)

Table de répertoire

Nom	1er cluster
osa.txt	10

Table d'allocation (FAT)

cluster	Cluster suivant
0	-1
1	4
2	1
3	0
4	-1
5	7
6	0
7	2
8	5
9	0
10	8

Disque dur

cluster	Données
0	
1	'OS
2	S D
3	
4	A!\0
5	E C
6	
7	OUR
8	E L
9	
10	VIV

GESTION DES RESSOURCES (PHYSIQUE)

MÉMOIRE DE MASSE : MÉTHODE D'ALLOCATION (EX 2)

Table de répertoire

Nom	1er cluster
venus.txt	1
mars.txt	5
terre.txt	8

Table d'allocation (FAT)

cluster	Cluster suivant
0	-2
1	2
2	12
3	-1
4	-2
5	6
6	7
7	-1
8	9
9	10
10	3
11	-2
12	-1
13	-2

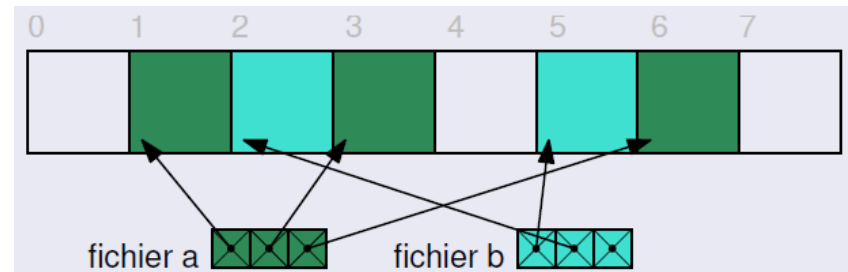
Disque dur

cluster	Données
0	
1	fe
2	mm
3	s
4	
5	ho
6	mm
7	es
8	en
9	fa
10	nt
11	
12	es
13	

GESTION DES RESSOURCES (PHYSIQUE)

MÉMOIRE DE MASSE : MÉTHODE D'ALLOCATION

- **Allocation indexée (2^{ème} variante) : i-nodes**
- Principe : associer à chaque fichier un tableau de numéros de blocs (*i-node*)
 - I-node = "index-node" (*nœuds d'information*).
 - Inclut les attributs (métadonnées) et les @ de blocs de données du fichier sur le disque (pointeurs directs).
 - Métadonnées : utilisateur et groupe propriétaire, type de fichier, droits d'accès, taille de fichier, ...
 - La $i^{\text{ème}}$ entrée de l'i-node pointe sur le $i^{\text{ème}}$ bloc de données du fichier.
 - Une entrée de répertoire pointe sur l'i-node.



GESTION DES RESSOURCES (PHYSIQUE)

MÉMOIRE DE MASSE : MÉTHODE D'ALLOCATION

- **Allocation indexée (2^{ème} variante) : i-nodes**
- Avantages :
 - Accès séquentiel et arbitraire efficaces.
 - Pas de fragmentation externe.
 - Peut accéder à n'importe quelle section d'un fichier facilement.
 - Meilleure utilisation de la mémoire: seules les tables d'index des fichiers ouverts sont chargés en mémoire (en FAT, la table doit entière doit être chargée en mémoire).
- Inconvénients :
 - Fragmentation interne plus grande qu'avec l'allocation chaînée.
 - Problème de la taille des i-nodes.



GESTION DES RESSOURCES (PHYSIQUE)

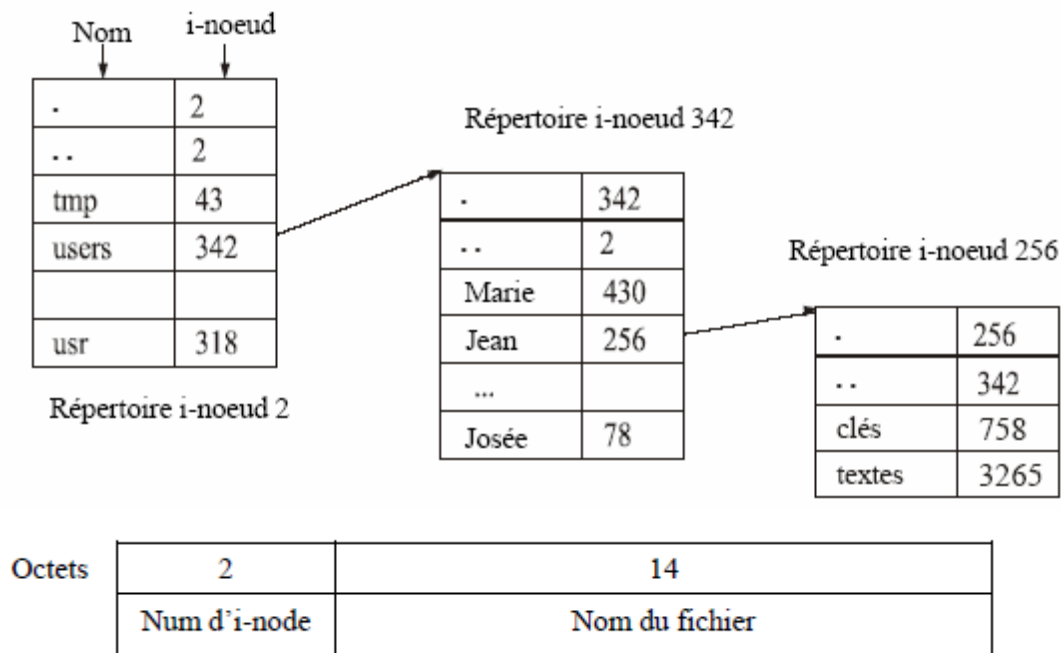
MÉMOIRE DE MASSE : MÉTHODE D'ALLOCATION

- **Allocation indexée (2^{ème} variante) : i-nodes (indexation multi-niveaux)**
- Principe : organiser l'i-node lui-même comme un arbre
 - Une seule taille d'inode : metadata + N pointeurs (généralement 64 octets)
 - 12 premiers pointeurs → 12 premiers blocs du fichier
 - 13^{ème} pointeur → un tableau de pointeurs sur des @ de blocs de données (bloc indirect simple) (Niveau 1)
 - 14^{ème} pointeur → tableau de pointeurs sur des blocs indirect simples (bloc indirect double) (Niveau 2)
 - 15^{ème} pointeur → tableau de pointeurs sur des blocs indirect doubles (bloc indirect triple) (Niveau 3)
- **Avantage : flexible**
 - Très rapide pour de petits fichiers (pointeurs directs).
 - Peut supporter de très gros fichiers (100's de Go !).

GESTION DES RESSOURCES (PHYSIQUE)

MÉMOIRE DE MASSE : MÉTHODE D'ALLOCATION

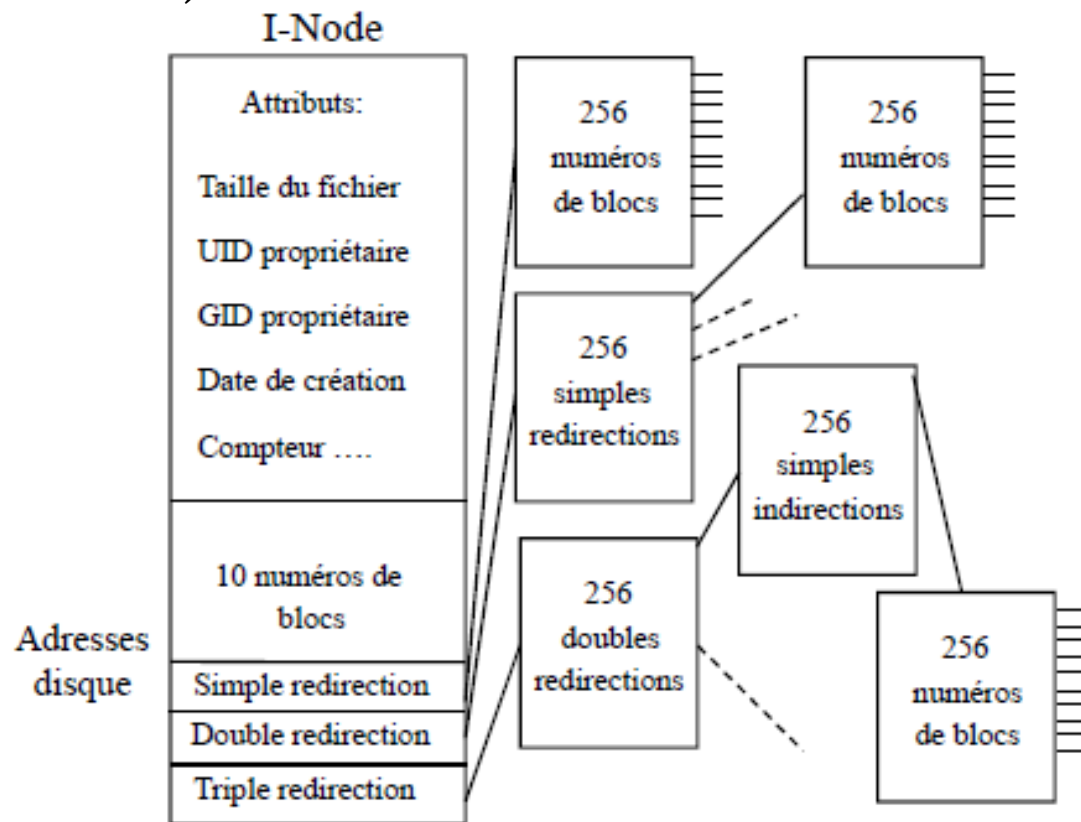
- Allocation indexée (2^{ème} variante) : i-nodes (indexation multi-niveaux)
- Unix possède une structure appelée "table des inodes".
- Un inode ("index node") de la table est utilisé pour chaque fichier. En d'autres mots, chaque entrée dans le répertoire de fichier contient un numéro d'Inode associé au fichier de l'entrée.



GESTION DES RESSOURCES (PHYSIQUE)

MÉMOIRE DE MASSE : MÉTHODE D'ALLOCATION

- Allocation indexée (2^{ème} variante) : i-nodes (indexation multi-niveaux)

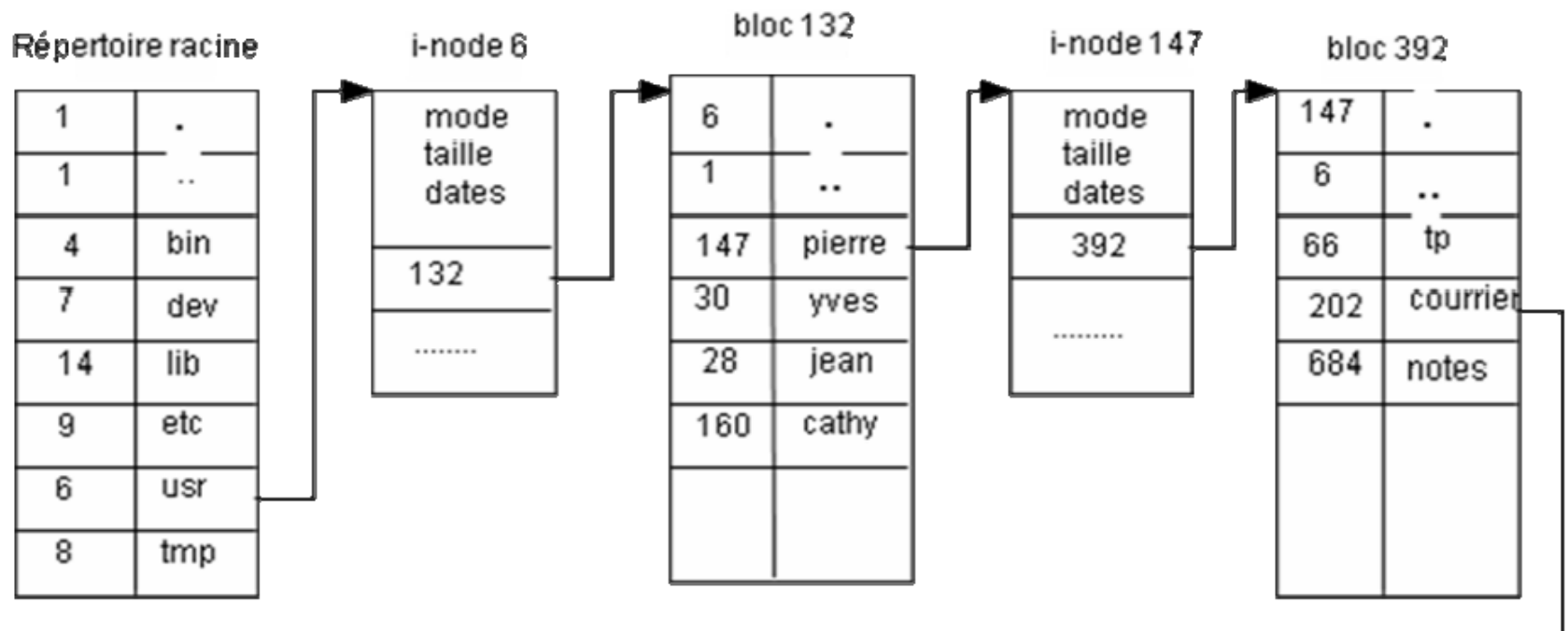


- Systèmes de fichier modernes : hybrides
- Allocation contiguë pour les petits fichiers
 - Indexation multi-niveaux pour les gros fichiers

GESTION DES RESSOURCES (PHYSIQUE)

MÉMOIRE DE MASSE : MÉTHODE D'ALLOCATION

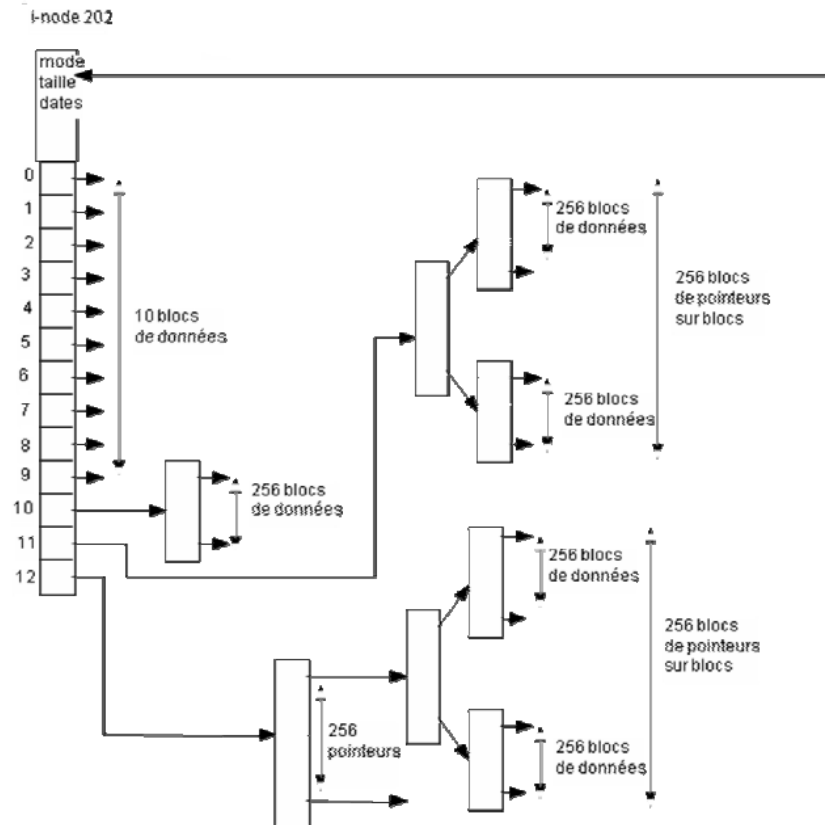
- Allocation indexée (2^{ème} variante) : i-nodes (indexation multi-niveaux)



GESTION DES RESSOURCES (PHYSIQUE)

MÉMOIRE DE MASSE : MÉTHODE D'ALLOCATION

- Allocation indexée (2^{ème} variante) : i-nodes (indexation multi-niveaux)



GESTION DES RESSOURCES (PHYSIQUE)

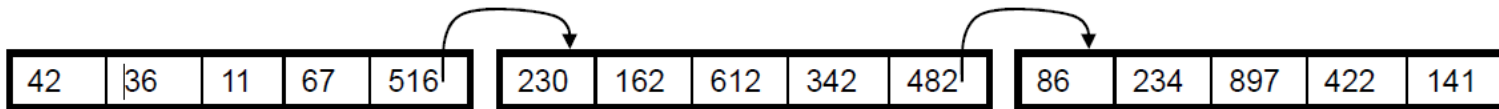
MÉMOIRE DE MASSE : MÉTHODE D'ALLOCATION (EX 3)

- **Allocation indexée (2^{ème} variante) : i-nodes (indexation multi-niveaux)**
- Structure d'un i-node :
 - 12 pointeurs directs qui pointent vers des blocs de données du fichier.
 - Un pointeur indirect simple.
 - Un pointeur indirect double.
 - Un pointeur indirect triple.
- En supposant que la taille d'un bloc est de 512 octets et la taille d'un index est de 4 octets, indiquer (en nombre de blocs et en octets) :
 1. La taille minimale d'un fichier ?
 - 512 octets
 2. la taille maximale d'un fichier ?
 - Nbre de pointeurs occupe un bloc (512 octets)
 - Nbre de pointeurs = taille bloc / taille d'un pointeur = $512 / 4 = 128$
 - Nbre de blocs = $12 + 128 + 128 * 128 + 128 * 128 * 128 = 2\ 113\ 676$
 - Taille = Nbre de blocs * taille d'un bloc = $1\ 082\ 202\ 112$ octets = 1 Go

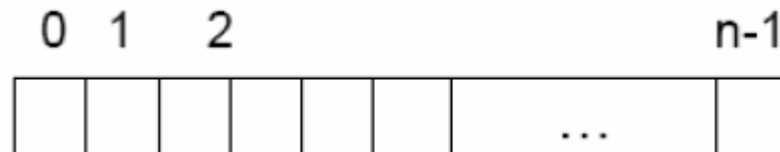
GESTION DES RESSOURCES (PHYSIQUE)

MÉMOIRE DE MASSE : GESTION DE L'ESPACE LIBRE

- **Repérage des blocs libres :**
- Dès qu'on a choisi la taille des blocs, on doit trouver un moyen de mémoriser les blocs libres.
- Les deux méthodes les plus répandues sont :
 - Liste chaînée : consiste à utiliser une liste chaînée des blocs du disque, chaque bloc contenant des numéros de blocs libres



- Table de bits : chaque bit représentant un bloc et valant 1 si le bloc est occupé (ou libre suivant le système d'exploitation).
 - Un disque de n blocs requiert une table de n bits.



GESTION DES RESSOURCES (PHYSIQUE)

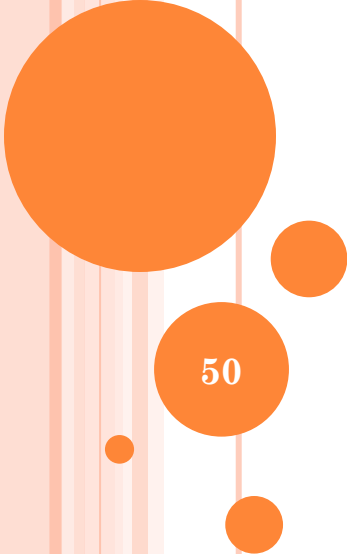
MÉMOIRE DE MASSE : GESTION DE L'ESPACE LIBRE

- **Repérage des blocs libres : liste chaînée**
- Considérons un disque dur de 500 Go. Une taille d'un bloc de 1 ko.
 - Le disque contient 524 millions de bloc ($500 \times 1024 \times 1024 = 524\,288\,000$).
- Le numéro de bloc codée sur 32 bits. Taille d'un élément (bloc) de la liste chaînée = 255 (256-1) numéros de bloc libres.
 - Taille de la liste chaînée (nbre de numéro de bloc libre que peut contenir un bloc de 1 ko) = Taille d'un bloc en bit / taille du numéro du bloc = $1024 \text{ octet} \times 8 \text{ bit} / 32 \text{ bits} = 256 - 1 = 255$ numéros.
- Pour adresser tous les blocs du disque dur, on a besoin de $524\,288\,000 / 255 = 2\,056\,031,37 = 2\,056\,032$ blocs.
 - 2 millions de blocs !!!!

GESTION DES RESSOURCES (PHYSIQUE)

MÉMOIRE DE MASSE : GESTION DE L'ESPACE LIBRE

- **Repérage des blocs libres : table de bits**
- Considérons un disque dur de 500 Go. Une taille d'un bloc de 1 ko.
 - Le disque contient 524 millions de bloc ($500 * 1024 * 1024 = 524\,288\,000$).
- La taille de la table de bit est de 524 288 000 bits.
- Pour adresser tous les blocs du disque dur on a besoin de $524\,288\,000 / (1024 * 8) = 64\,000$ blocs.



50

GESTION DES PROCESSUS

Processus & Threads

Ordonnancement

GESTION DE PROCESSUS

FONCTIONS / OPÉRATIONS

- Création, suppression et interruption
- Ordonnancement de processus
- Synchronisation de processus
- Gestion des conflits d'accès à des ressources partagées

GESTION DE PROCESSUS

NOTION DE BASE

- Programme / Processus / Thread
- Table de processus / Espace d'adressage/PCB
- Préemptif / non préemptif
- États d'un processus:
 - Standards : Prêt, En Exécution, Bloqué, Terminé
 - Particuliers : Orphelin, Zombie, Swappé

GESTION DE PROCESSUS

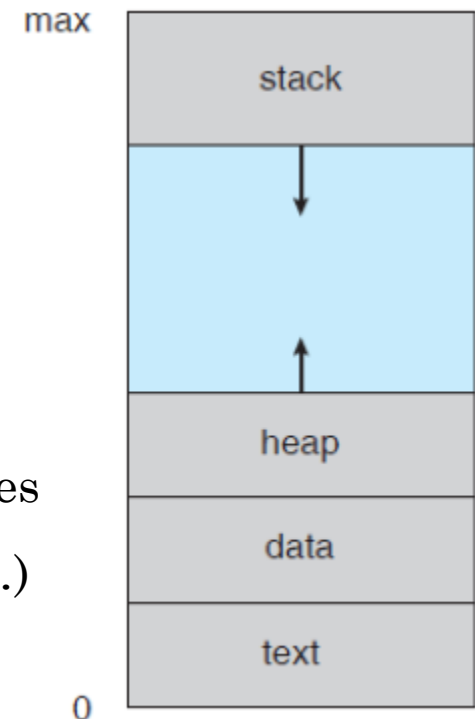
PROGRAMME / PROCESSUS / THREAD

- **Programme** : c'est une suite d'instructions (lignes de codes).
- **Processus** : c'est un programme en exécution et possédant son propre environnement (CPU : son compteur ordinal, registres / Mémoire : ses données, sa pile).
- **Thread (processus allégé)** : c'est un flot (fil) d'exécution dans le code du processus doté :
 - D'un identifiant
 - D'un compteur ordinal (Program Counter (PC)).
 - D'un ensemble de registres
 - D'une pile
- Plusieurs processus permettent à un ordinateur d'effectuer plusieurs tâches à la fois. → Partage de ressources physiques.

GESTION DE PROCESSUS

ESPACE D'ADRESSAGE DES PROCESSUS

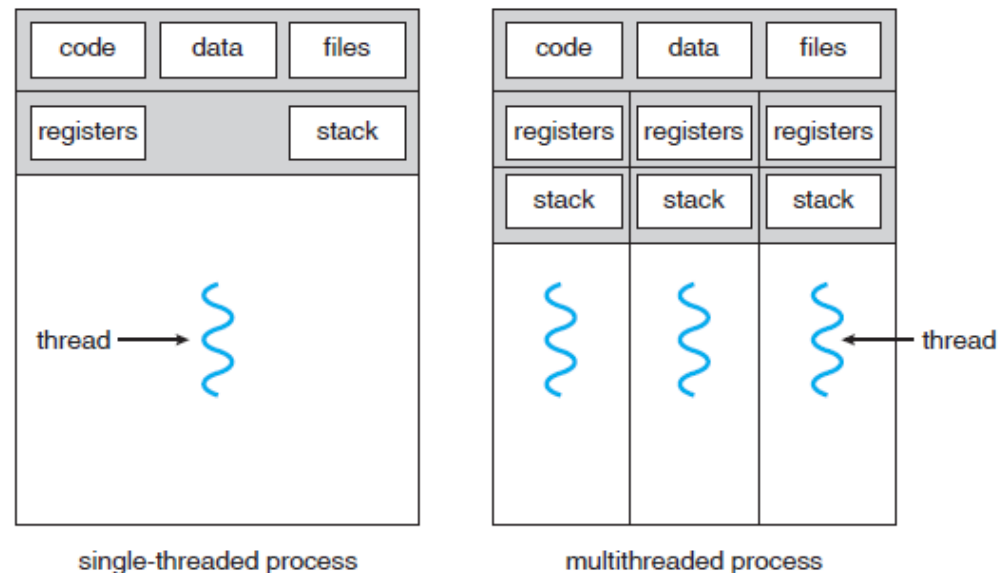
- Chaque processus possède un espace d'adressage, c'est à dire un ensemble d'adresses mémoire dans lesquelles il peut lire et écrire. Cet espace est divisé en quatre parties :
 - Le segment de texte → le code du programme
 - Le segment de données → les variables globales
 - La pile (d'exécution) → les données provisoires (les fonctions, les paramètres, les variables locales, ...)
 - Le tas (heap) qui est un mémoire allouée dynamiquement lors de l'exécution processus.



GESTION DE PROCESSUS

THREADS

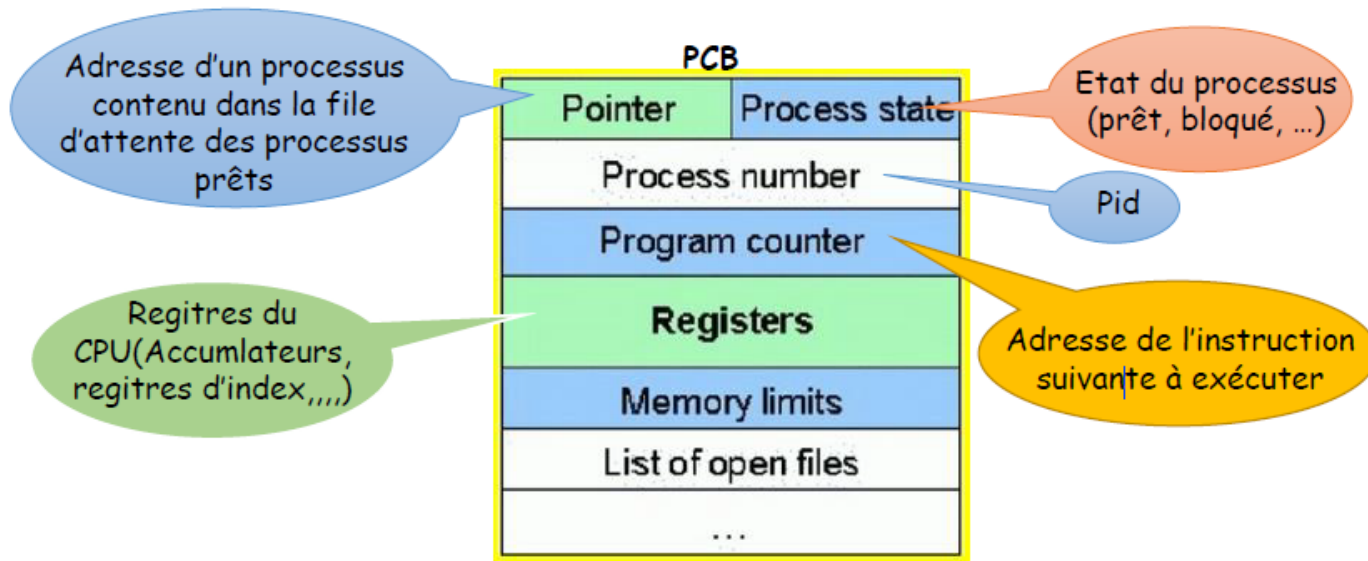
- Plusieurs threads permettent à un processus de décomposer le travail à exécuter en parallèle (**Multithreading**). → Partage de ressources physiques et virtuelles.
- Un thread partage avec les autres threads appartenant au même processus :
- La section de code
- La section de données
- Et d'autre ressources du système d'exploitation, telles que les fichiers ouverts et les signaux.



GESTION DE PROCESSUS

PROCESS CONTROL BLOCK (PCB)

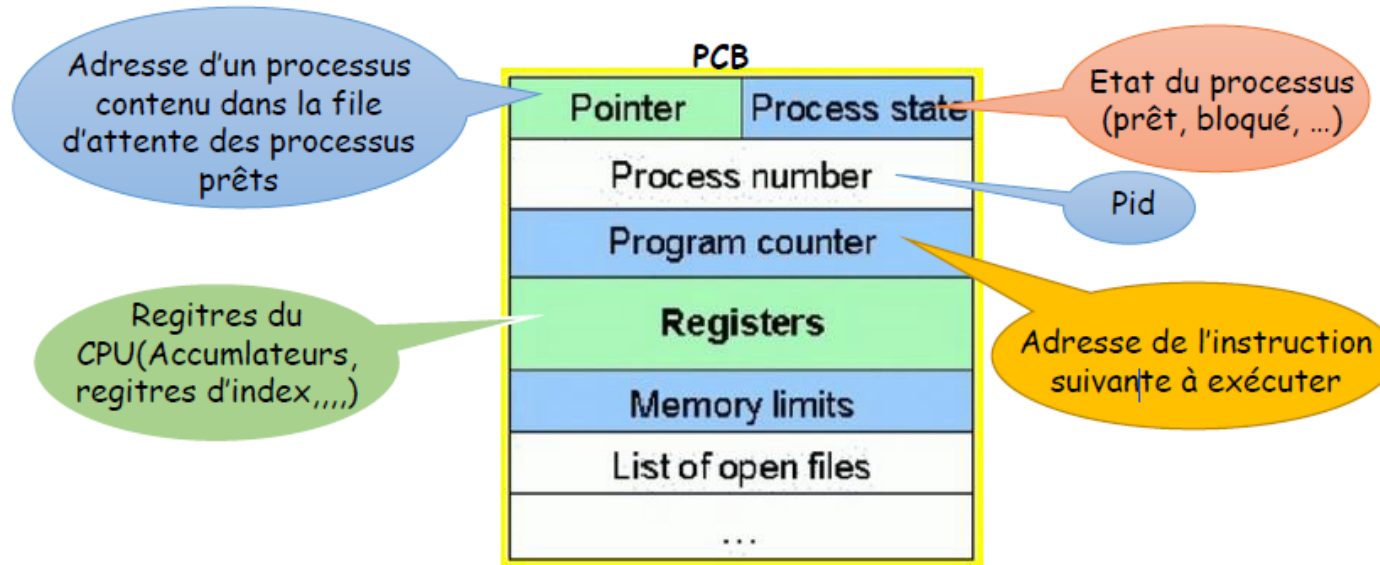
- **Pointer** : contient l'adresse du PCB suivant en état prêt.
- **Process state** : contient l'état du processus.
- **Process number (PID)** : c'est l'identifiant unique du processus.
- **Program counter (PC)** : contient l'adresse de l'instruction suivante à exécuter.



GESTION DE PROCESSUS

PROCESS CONTROL BLOCK (PCB)

- **Registers** : ces sont les registres du processeur (accumulateur, registres de base, pointeur de pile, ...).
- **Memory limits** : contient des informations sur le système de management de la mémoire (valeurs de registre de base et limiteur). Il contient aussi l'espace d'adressage.
- **Open files list** : contient la liste de fichiers utilisés par le processus.



GESTION DE PROCESSUS

PROCESS CONTROL BLOCK (PCB)

○ *Autres informations :*

- Une copie du mot d'état du processeur (*Processor Status Word (PSW)*) au moment de la dernière interruption du processus.
- Des informations sur les ressources utilisées :
 - Mémoire principale
 - Temps d'exécution
 - Périphériques d'E/S en attente
 - Files d'attente dans lesquelles le processus est inclus, etc...
- Et toutes les informations nécessaires pour assurer la reprise du processus en cas d'interruption.

GESTION DE PROCESSUS

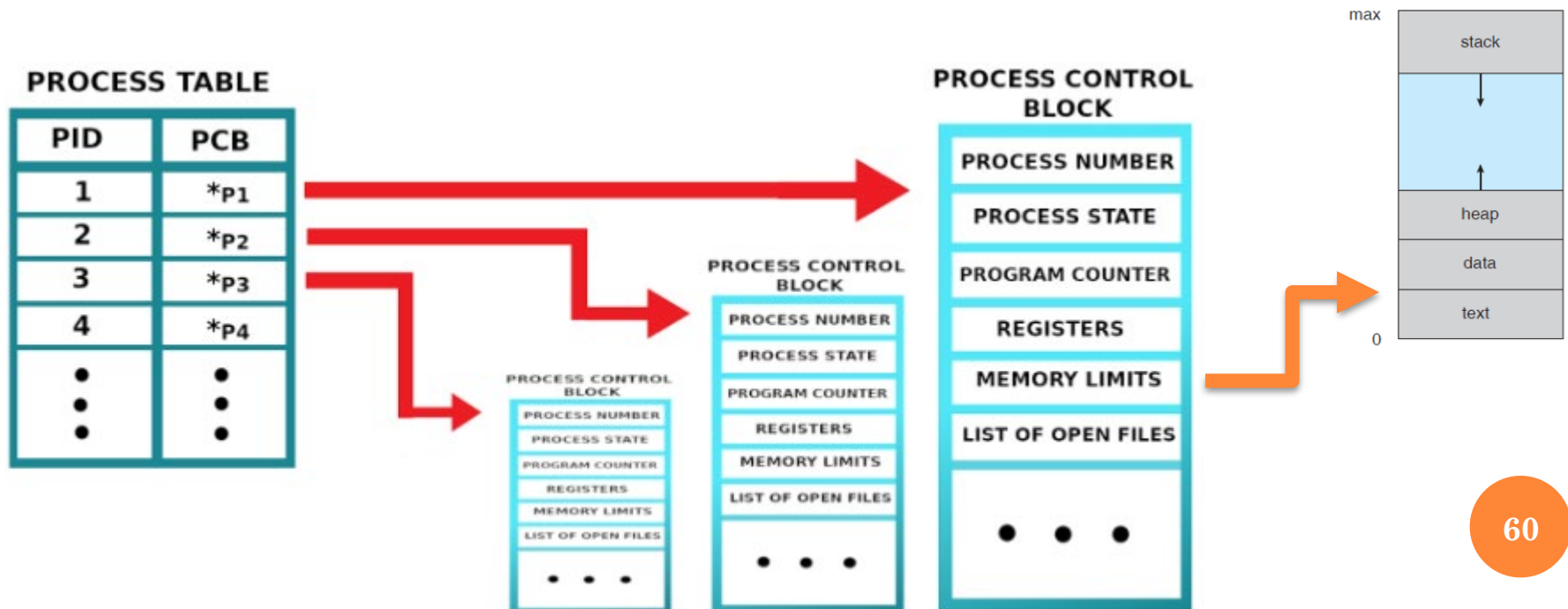
PROCESSOR STATUS WORD (PSW)

- Mot d'état du processeur (***Processor Status Word (PSW)***) :
une entité logique dans laquelle le SE regroupe des informations clés sur le fonctionnement du processeur
- Il comporte généralement :
 - la valeur du compteur ordinal
 - des informations sur les interruptions
 - le privilège du processeur (mode maître ou esclave)
 - etc.

GESTION DE PROCESSUS

TABLE DE PROCESSUS

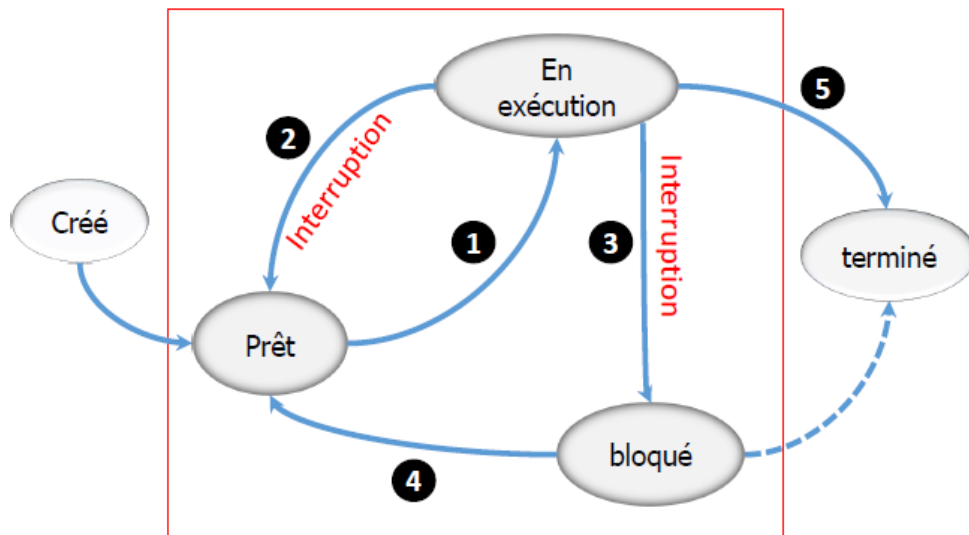
- Tout processus créé contient une entrée dans cette table.
- Chaque entrée contient un pointeur vers le PCB du processus, l'identifiant du processus.
- Elle est stockée dans l'espace mémoire du système d'exploitation.



GESTION DE PROCESSUS

ETATS D'UN PROCESSUS : STANDARDS

- Un processus peut être dans l'un des quatre états:
 - **En Exécution (ou "élu")** : le processus est en cours d'utilisation du processeur (suite à son élection).
 - **Prêt** : le processus est suspendu pour permettre l'exécution d'autres.
 - **Bloqué (Attente)** : le processus est en attente d'une ressource ou d'un évènement externe pour continuer son exécution.
 - **Terminé** : le processus est terminé normalement.



- 1 Activation ou Reprise du processus décidé par l'ordonnanceur
- 2 Réquisition provoquée par l'ordonnanceur (fin Quantum)
- 3 Manque de ressources (E/S)
- 4 Ressource disponible
- 5 Terminaison

GESTION DE PROCESSUS

ETATS D'UN PROCESSUS : PARTICULIERS

- ***Zombie*** : Si un fils se termine tout en disposant toujours d'un PID celui-ci devient un processus zombie.
 - Le cas le plus fréquent : le processus s'est terminé mais son père n'a pas (encore) lu son code de retour.
- ***Orphelin*** : si un processus père meurt avant son fils ce dernier devient orphelin.
- ***Swappé*** : Lorsqu'un processus est transféré de la mémoire centrale vers la mémoire virtuelle, il est dit "swappé". Un processus swappé peut être dans un état bloqué ou prêt.

GESTION DE PROCESSUS

CRÉATION D'UN PROCESSUS

- Il existe essentiellement quatre évènements provoquant la création d'un processus:
 1. Initialisation du système (processus *init* en UNIX)
 2. Exécution d'un appel de création de processus par un autre processus en cour d'exécution (*fork ()* en UNIX)
 3. Requête utilisateur sollicitant la création d'un nouveau processus
 4. Initiation d'un travail en traitement par lots (enchaînement automatique d'une suite de commandes)

GESTION DE PROCESSUS

FIN D'UN PROCESSUS

- L'arrêt d'un processus est causé par diverses raisons:
 1. Arrêt normal (volontaire) : fin d'exécution de la tâche affectée.
 2. Arrêt pour erreur (volontaire) : division par 0.
 3. Arrêt pour erreur fatale (involontaire) : violation accès mémoire, ...
 4. Le processus est arrêté par un autre processus (involontaire/externe) : fin de tâche d'un processus (cas de Windows), commande *kill* en UNIX.

GESTION DE PROCESSUS

INTERRUPTIONS : DÉFINITION

- **Interruption** : c'est une suspension temporaire de l'exécution d'un programme (processus) par le microprocesseur afin d'exécuter un autre programme (prioritaire).
- On parle d'interruption (IT) dans les cas de transitions "**En Exécution**" → "**Prêt**" et "**En Exécution**" → "**Bloqué**".
- Une interruption est survenue lorsque :
 - Le processus a atteint une instruction E/S
 - Le temps (quantum) attribué au processus est écoulé
 - Un processus plus urgent doit être exécuté
 - Un processus nécessite une ressource (matérielle ou logicielle)

GESTION DE PROCESSUS

INTERRUPTIONS : DÉFINITION

- Une IT est provoqué par un signal généré soit par un évènement *interne* ou *externe*.
 - **Evènements internes** : lié au processus
 - Appel système
 - Déroutement : dû généralement aux erreurs telles qu'une division par 0, débordement de la mémoire, ...
 - **Evènements externes** : panne, intervention de l'utilisateur ("Ctrl+Alt+Supp", bouton "reset"), ...
- Deux types d'IT : *matérielles* et *logicielles*
 - **IT matérielle (IRQ)** : générés par les périphériques et parviennent au processeur par l'intermédiaire du contrôleur d'IT (distribuer et organiser les IRQs).
 - **IT logicielles** : ces sont des IT internes. C'est le processeur qui appelle ces ITs.
- Si plusieurs ITs arrivent au même temps, c'est celle qui a le plus petit numéro est la plus prioritaire (IRQ horloge sys = 0, IRQ port parallèle = 7).

GESTION DE PROCESSUS

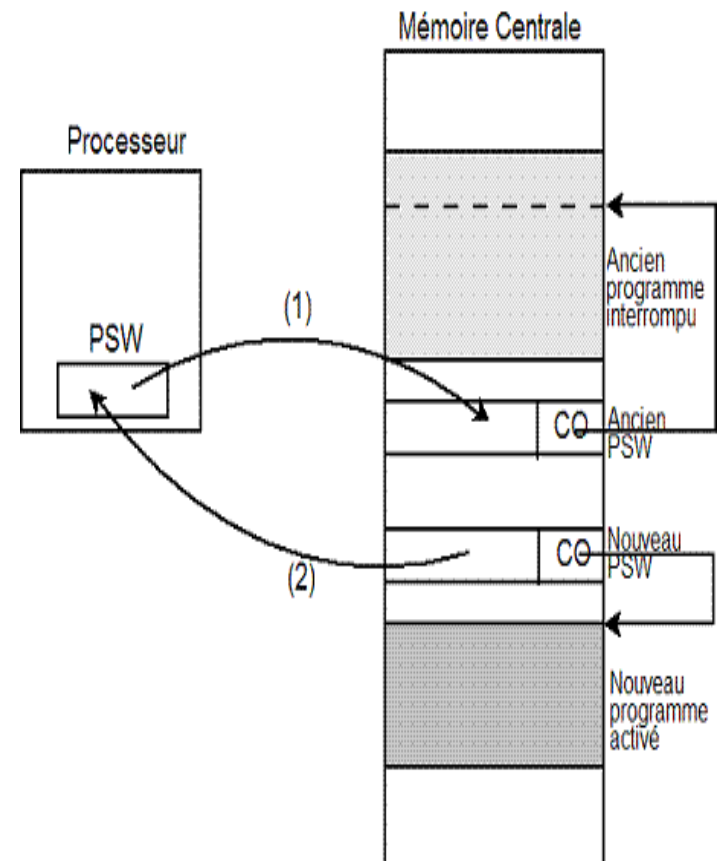
INTERRUPTIONS : TRAITEMENT (ROUTINE)

1. Arrivée de l'IT → le processus en cours est interrompu et un **gestionnaire d'IT** est chargé dans les registres du processeur et s'exécute pour traiter l'IT en question. Le gestionnaire d'IT réalise la sauvegarde du contexte du processus interrompu (compteur ordinal, registres, ...).
2. Une fois le signal de l'IT est reconnu, le gestionnaire d'IT accède la table des vecteurs d'IT et y cherche l'adresse du programme associé ("**Routine d'IT**") et l'exécute.
3. La routine d'IT accomplit les opérations liées à l'interruption concernée et donne un nouveau contenu au mot d'état : c'est l'acquiescement de l'interruption.
4. Le gestionnaire d'IT restaure le contexte et le SE reprend le processus interrompu ou charge et exécute un autre processus à partir de la file d'attente.

GESTION DE PROCESSUS

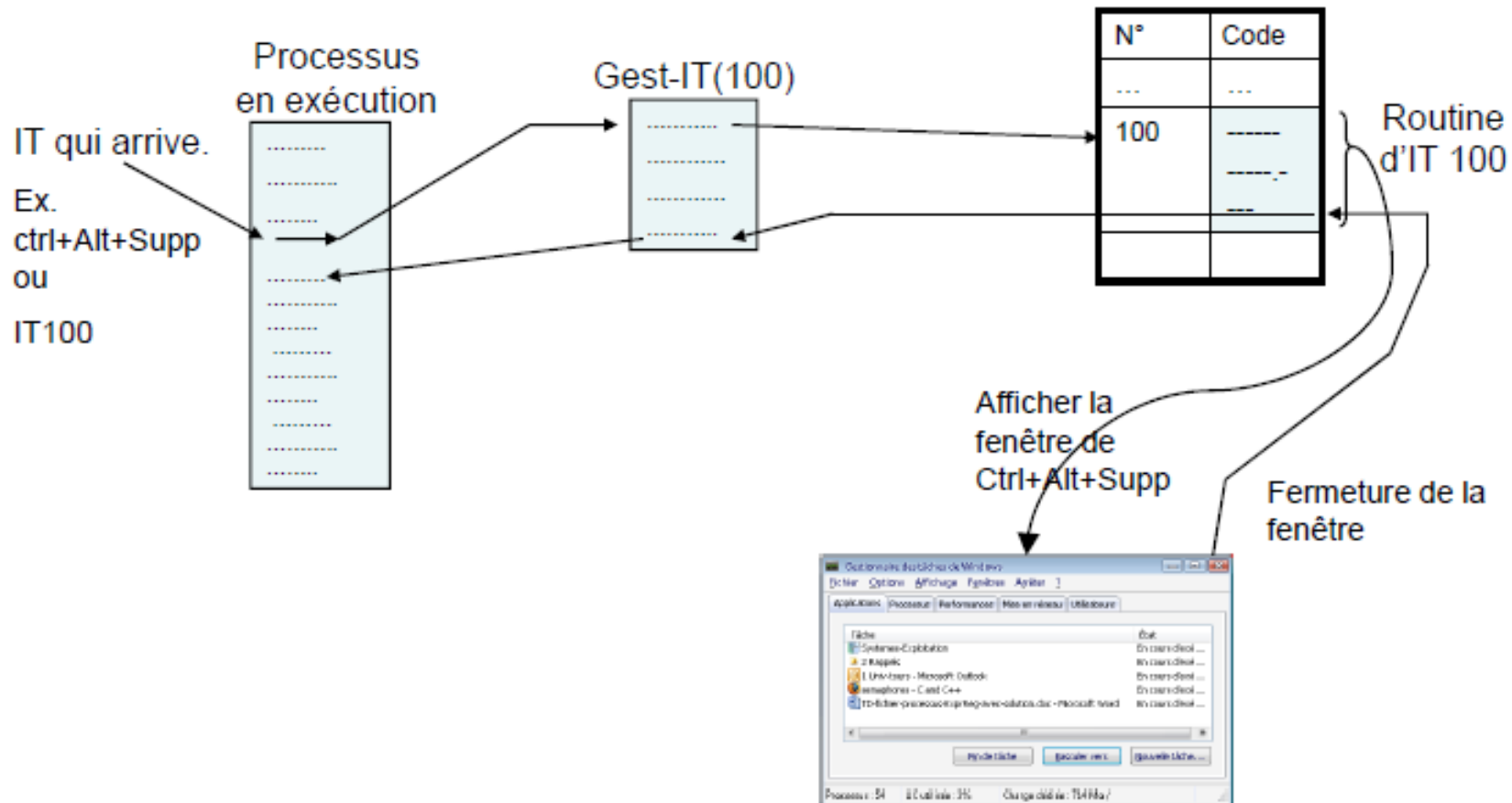
PSW : COMMUTATION DE CONTEXTE

- Le mécanisme de commutation de contexte permet de :
 - Ranger dans un emplacement spécifié de la mémoire le contenu courant du mot d'état du processeur (PSW).
 - Charger dans le mot d'état un nouveau contenu préparé à l'avance dans un emplacement spécifié de la mémoire.



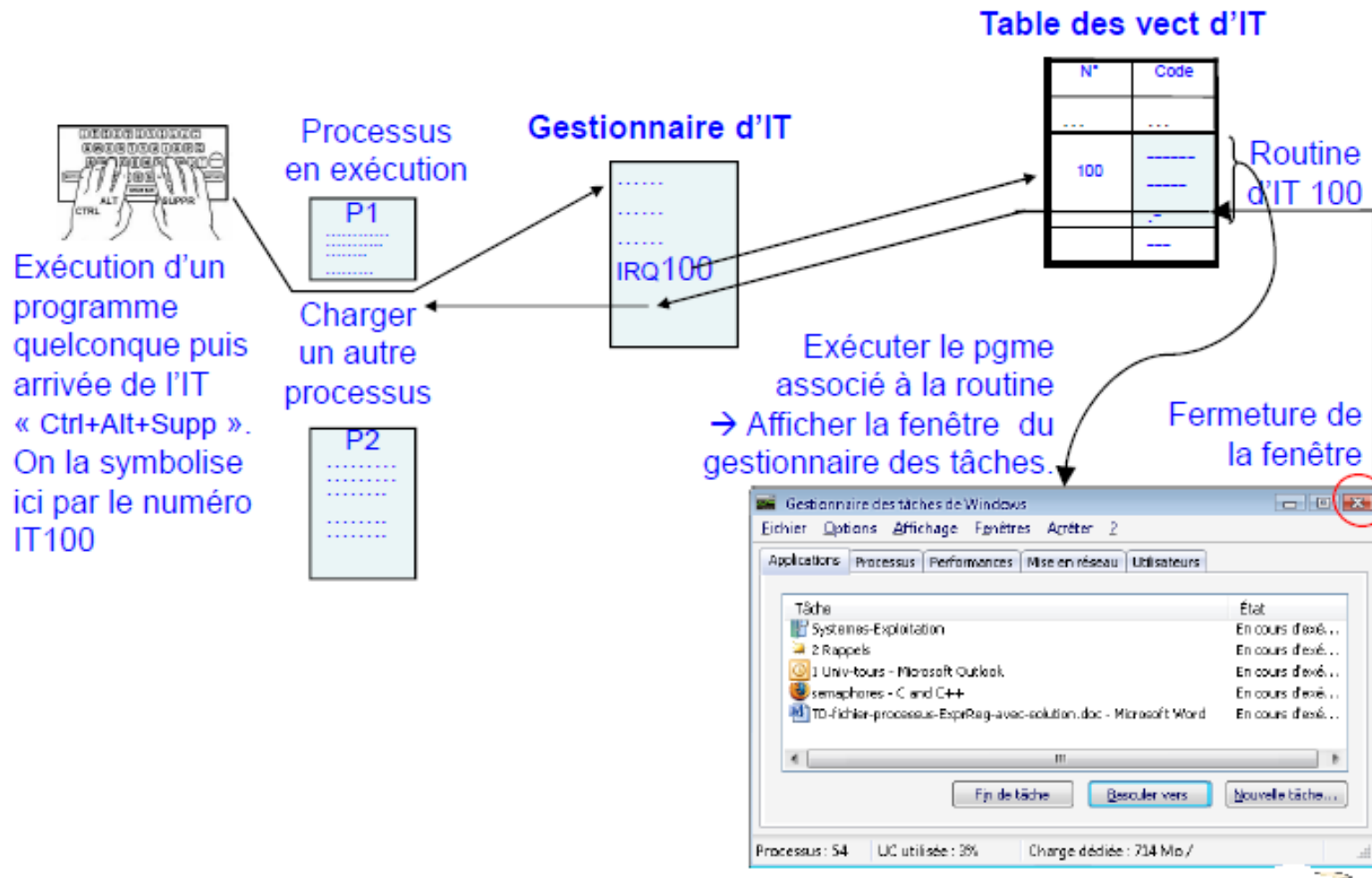
GESTION DE PROCESSUS

TRAITEMENT D'UNE IT : REPRISE DU MÊME PROCESSUS



GESTION DE PROCESSUS

TRAITEMENT D'UNE IT : COMMUTATION DES PROCESSUS



GESTION DE PROCESSUS

CRÉATION DE PROCESSUS : APPEL SYSTÈME `fork ()`

- Le parallélisme Unix bas niveau est fourni par le noyau qui duplique un processus lorsqu'on invoque l'appel-système **`fork ()`**.
- Les deux processus sont alors strictement identiques, et seule la valeur de retour de **`fork ()`** permet de les distinguer.
- Le nouveau processus comprend une copie de l'espace d'adressage du processus original.
 - Ce mécanisme permet au processus parent de communiquer facilement avec son processus enfant.
- Les deux processus (le parent et l'enfant) poursuivre l'exécution à l'instruction après le **`fork ()`**, avec une différence:
 - Le code retour du **`fork ()`** est égal à zéro pour le nouveau (enfant) processus.
 - Alors que le PID (non nulle) de l'enfant est retourné au processus père.

GESTION DE PROCESSUS

CRÉATION DE PROCESSUS : APPEL SYSTÈME FORK ()

- L'appel-système *fork ()* est déclaré dans *<unistd.h>* :
 - *pid_t fork (void);*
- Deux valeurs de retour en cas de succès:
 - Dans le processus père : valeur de retour = le PID du fils,
 - Dans le processus fils : valeur de retour = zéro.
- Sinon
 - Erreur : valeur de retour = -1.
- Afin d'obtenir le numéro du processus, il suffit de faire l'appel système `getpid()`, ou `getppid()` pour obtenir le numéro du père.
 - PID (Process Identifier)
 - PPID : numéro du processus père (Parent Process Identifier)

GESTION DE PROCESSUS

CRÉATION DE PROCESSUS : APPEL SYSTÈME FORK ()

- Primitives de gestion de processus :
 - *int/pid_t fork ();*
 - *int execlp (char *comm, char *arg0, ..., NULL);*
 - *void exit (int status);*
 - *int wait (int *ptr_status);*
 - *pid_t waitpid (pid_t pid, int *status, int opts)*
 - *int sleep (int seconds);*
 - *int getpid ();*
 - *int getppid ();*

GESTION DE PROCESSUS

CRÉATION DE PROCESSUS : APPEL SYSTÈME FORK () - EX 1

- Préciser le nombre de processus créer par les programmes ci-dessous :

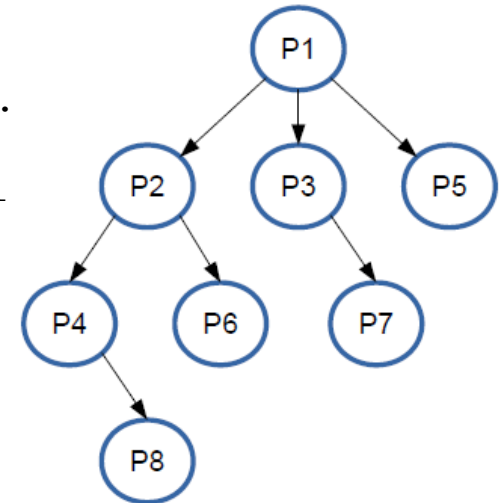
Code 1	Code 2	Code 3
<pre><i>int main()</i> <i>{</i> <i>fork();</i> <i>fork();</i> <i>fork();</i> <i>}</i></pre>	<pre><i>int main()</i> <i>{</i> <i>if (fork() > 0)</i> <i>fork();</i> <i>}</i></pre>	<pre><i>int main()</i> <i>{</i> <i>int cpt=0;</i> <i>while (cpt < 3) {</i> <i>if (fork() > 0)</i> <i>cpt++;</i> <i>else</i> <i>cpt=3;</i> <i>}</i></pre>

GESTION DE PROCESSUS

CRÉATION DE PROCESSUS : APPEL SYSTÈME `fork()` - EX 1

○ *Code 1* → 8 processus sont créés :

- L'exécution du programme crée un processus P1.
- A la lecture de la première instruction *fork()*, P1 se duplique et crée alors P2.
- Les deux processus continuent l'exécution à partir de la ligne incluse ;
- A la lecture de la seconde instruction *fork()*, P1 se duplique et crée P3 tandis que P2 crée P4.
- Les quatre processus continuent l'exécution à partir de la ligne incluse ;
- A la lecture de la troisième instruction *fork()*, P1 se duplique et crée P5, P2 crée P6, P3 crée P7 et P4 crée P8.



Code 1

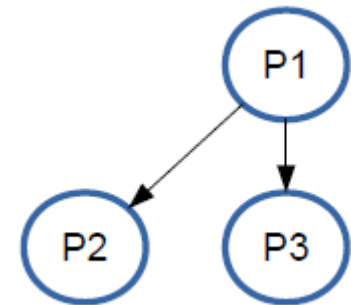
```
int main()  
{  
    fork();  
    fork();  
    fork();  
}
```

GESTION DE PROCESSUS

CRÉATION DE PROCESSUS : APPEL SYSTÈME `FORK ()` - EX 1

○ **Code 2** → 3 processus sont créés :

- L'exécution du programme crée un processus P1.
- A la lecture de la première instruction `fork ()`, P1 se duplique et crée alors P2. P1 est le processus parent, P2 le processus enfant.
- Les deux processus continuent l'exécution à partir de la ligne incluse ; Le résultat de l'appel précédent est supérieur à 0 pour P1. Ce dernier rentre donc dans la suite d'instructions conditionnée et exécute l'instruction `fork ()`.
- P1 se duplique et crée donc P3.
- En revanche, le résultat de l'appel précédent est égale à 0 pour P2, qui ne rentre donc pas dans la suite d'instructions conditionnée.



Code 2

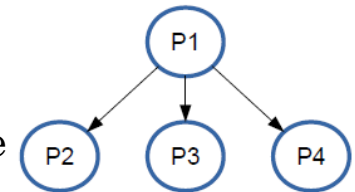
```
int main()  
{  
  if (fork() > 0)  
    fork();  
}
```

GESTION DE PROCESSUS

CRÉATION DE PROCESSUS : APPEL SYSTÈME `FORK ()` - EX 1

○ *Code 3* → 4 processus sont créés :

- L'exécution du programme crée un processus P1, qui initialise la variable *cpt* à 0.
- P1 rentre dans la boucle *while ()* et se duplique lors de l'exécution de `fork ()`. Il crée alors P2.
- Le résultat de l'appel précédent est supérieur à 0 pour P1. Ce dernier rentre donc dans la suite d'instructions conditionnée par "*if*" et incrémente son compteur *cpt* qui passe à 1.
- En revanche, le résultat de l'appel précédent est égale à 0 pour P2, qui rentre donc dans la suite d'instructions conditionnée par "*else*" et affecte *cpt* à 3. Dès lors P2 sort de la boucle et n'exécutera plus d'instruction.
- Seul P1 ré-exécute la séquence d'instruction de la boucle *while ()*, et le même schéma se reproduit : à chaque entrée dans la boucle, P1 se duplique, tandis que le processus dupliqué n'exécute aucune instruction.
- P1 aura ainsi exécuté 3 fois l'instruction `fork ()` jusqu'à ce que sa variable *cpt* atteigne 3.
- Il aura donc engendré P2, P3 et P4



Code 3

```
int main()
{
    int cpt=0;
    while (cpt < 3)
    {
        if (fork() > 0)
            cpt++;
        else
            cpt=3;
    }
}
```

GESTION DE PROCESSUS

MANIPULATION DES THREADS

○ Création de threads :

- *int pthread_create (pthread_t *thread, pthread_attr_t *attr, void *(*nomfonction), void *arg);*
- Le service **pthread_create ()** crée un thread qui exécute la fonction **nomfonction** avec l'argument **arg** et les attributs **attr**.
 - Les attributs permettent de spécifier la taille de la pile, la priorité, la politique de planification, etc. Il y a plusieurs formes de modification des attributs.

○ Suspension de threads :

- *int pthread_join (pthread_t thid, void **valeur_de_retour);*
- **pthread_join ()** suspend l'exécution d'un thread jusqu'à ce que termine le thread avec l'identificateur **thid**. Il retourne l'état de terminaison du thread.

GESTION DE PROCESSUS

MANIPULATION DES THREADS

- Terminaison de threads :
 - *void pthread_exit (void *valeur_de_retour);*
- **pthread_exit ()** permet à un thread de terminer son exécution, en retournant l'état de terminaison.
- Identité d'un thread :
 - *pthread_t pthread_self (void);*

GESTION DE PROCESSUS

ORDONNANCEMENT

- On appelle ordonnancement la stratégie d'attribution des ressources aux processus qui en font la demande.
- La partie du système d'exploitation qui effectue l'ordonnancement s'appelle ordonnanceur (Scheduler)
- L'algorithme qu'emploie l'ordonnanceur est appelé algorithme d'ordonnancement (Scheduling algorithm)
- L'ordonnancement est exigé lors de:
 - Terminaison d'un processus
 - Blocage (par exemple E/S)
 - Production d'une interruption horloge (fin quantum)

GESTION DE PROCESSUS

ORDONNANCEMENT : CRITÈRES

- La politique d'ordonnancement doit optimiser les performances du système à travers certains critères:
 - ***Rendement d'utilisation du CPU*** : pourcentage de temps pendant lequel le CPU est actif (occupation maximale).
 - ***Capacité de traitement (débit)*** : C'est le nombre de processus terminés par unité de temps.
 - ***Utilisation globale des ressources*** : assurer une occupation maximale des ressources de la machine et minimiser le temps d'attente pour l'allocation d'une ressource à un processus.
 - ***Équité*** : allouer d'une façon équitable le CPU à tous les processus de même priorité.
 - ***Temps d'attente*** : C'est le temps passé à attendre dans la file d'attente des processus prêts (*$t. \text{début d'exéc.} - t. \text{d'arrivée}$*) (Sauf pour l'algo. RR, prise en compte de quantum).
 - ***Temps de rotation (de réponse)*** : durée moyenne pour qu'un processus termine son exécution (*$t. \text{fin d'exécution} - t. \text{d'arrivée}$*).

GESTION DE PROCESSUS

ORDONNANCEMENT : CATÉGORIES DES ALGORITHMES

- On peut classer les algorithmes d'ordonnancement en deux catégories:
 - ***Non préemptif*** : l'algorithme sélectionne un processus, puis le laisse s'exécuter jusqu'à ce qu'il bloque ou libère volontairement le processeur.
 - Exemple: FIFO, SJF, ...
 - ***Préemptif*** : capable de réquisitionner le processeur dans le cas où un critère donnée est satisfait (délai, IT).
 - Exemple: Round Robin, RSJF, ...

GESTION DE PROCESSUS

ORDONNANCEMENT : NON PRÉEMPTIF

- **Premier Arrivé Premier Servi (FIFO):**
- Les tâches sont ordonnancées dans l'ordre où elles sont reçues.
- On utilise une structure de file FIFO.
- Cette stratégie désavantage les processus courts s'ils ne sont pas exécutés en premier.

PID	Temps d'arrivée	Durée de traitement	Début d'exécution	Temps de fin d'exécution	Temps de rotation
1	10	2	10	12	2
2	10,1	1	12	13	2.9
3	10,25	0.25	13	13.25	3

GESTION DE PROCESSUS

ORDONNANCEMENT : NON PRÉEMPTIF

Le Plus Court Job d'abord (SJF: Shortest Job First):

- Cet algorithme nécessite la connaissance du temps d'exécution estimé de chaque processus.
- Le processeur est attribué au processus dont le temps d'exécution estimé est minimal.

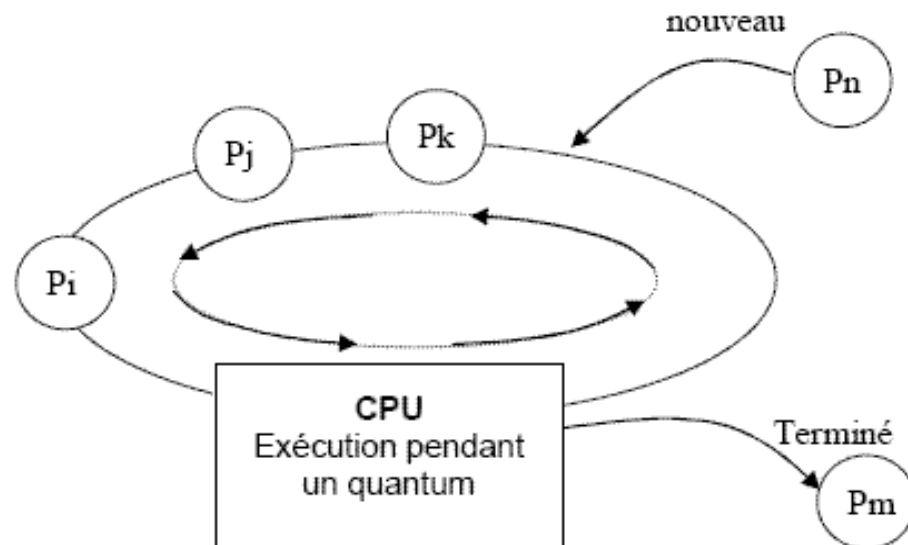
PID	Temps d'arrivée	Durée de traitement	Début d'exécution	Temps de fin d'exécution	Temps de rotation
1	10	2	10	12	2
2	10,1	1	12,25	13,25	3,15
3	10,25	0,25	12	12,25	2

GESTION DE PROCESSUS

ORDONNANCEMENT : PRÉEMPTIF

Algorithme à Tourniquet (RR : Round Robin) :

- Le contrôle du processeur est attribué à chaque processus pendant une tranche de temps maximum Q , *Quantum*, à tour de rôle.
- Lors qu'un processus s'exécute pendant un quantum, le contexte de ce dernier est sauvegardé et le processeur est attribué au processus suivant dans la liste des processus prêts.



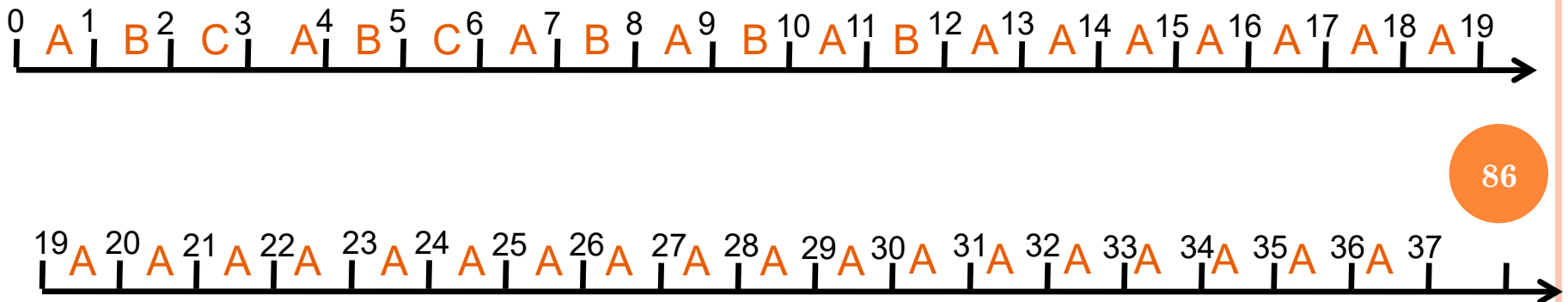
GESTION DE PROCESSUS

ORDONNANCEMENT : PRÉEMPTIF

- **Algorithme à Tourniquet (RR: Round Robin):**
- La performance de cet algorithme dépend de la valeur du quantum (quantum $\uparrow$ temps de réponse $\uparrow$).
- Exemple: quantum = 1

PID	Temps d'arrivée	Durée de traitement	Début d'exécution	Temps de fin d'exécution	Temps de rotation
A	0	30	0	37	37
B	0.2	5	1	12	11.8
C	0.5	2	2	6	5.5

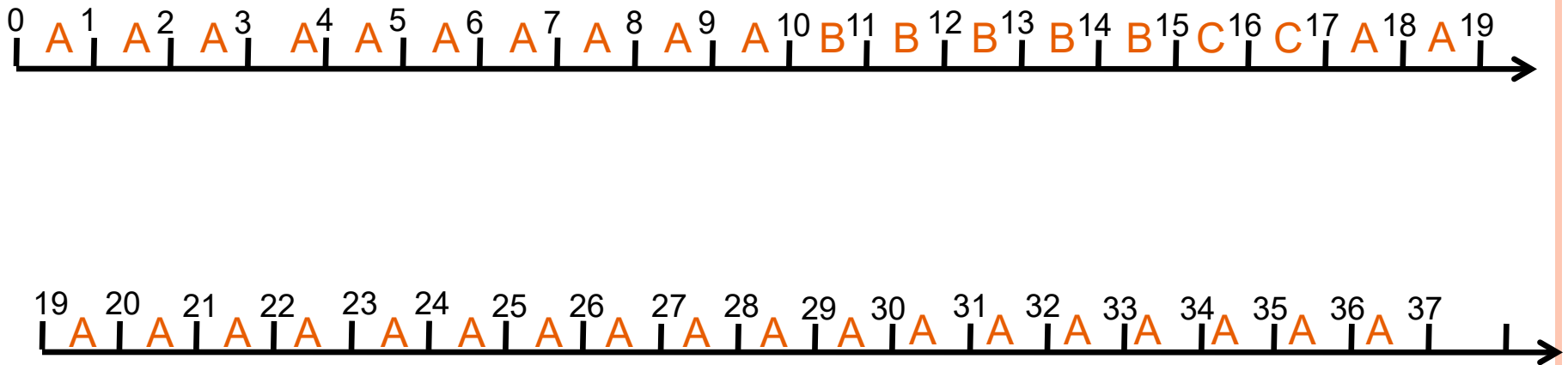
- Exécution:



GESTION DE PROCESSUS

ORDONNANCEMENT : PRÉÉMPPTIF

- **Algorithme à Tourniquet (RR: Round Robin):**
- Exemple: quantum = 10

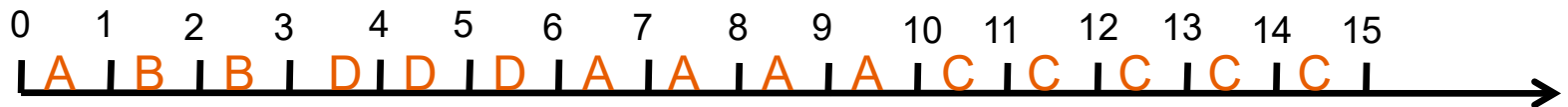


GESTION DE PROCESSUS

ORDONNANCEMENT : PRÉEMPTIF

Le temps restant le plus court (SRTF):

- C'est la version préemptive de SJF.
- Au début de chaque quantum, le processeur est attribué au processus qui a le plus petit "temps d'exécution restant".
- Exemple: on prend un quantum = 1



PID	Temps d'arrivée	Durée de traitement	Début d'exécution	Temps de fin d'exécution	Temps de rotation
A	0	5	0	10	10
B	1	2	1	3	2
C	2	5	10	15	13
D	3	3	3	6	3

GESTION DE PROCESSUS

ORDONNANCEMENT : ALGORITHMES AVEC PRIORITÉ

- Le système associe à chaque processus une priorité.
- **Priorité:** nombre mesurant l'importance du processus dans le système.
- Priorité: **statique** ou **dynamique**
- Le processeur est attribué au processus de priorité la plus haute.
- En cas d'égalité l'algorithme de "premier arrivé premier servi" est utilisé.
- L'ordonnancement des priorités peut être avec ou sans réquisition (préemptif ou non préemptif).

GESTION DE PROCESSUS

ORDONNANCEMENT : ALGORITHMES AVEC PRIORITÉ

- On suppose que la valeur la plus petite correspond à la priorité la plus élevée.

PID	Temps d'arrivée	Durée de traitement	Priorité
A	0	10	3
B	0	1	1
C	10	2	2
D	0	1	4
E	0	5	2

- Exécution:

- Sans réquisition (i.e. non préemptif)



- Avec réquisition (i.e. préemptif)



GESTION DE PROCESSUS

ORDONNANCEMENT DE PROCESSUS SOUS UNIX

L'ordonnanceur d'UNIX associe une priorité dynamique à chaque processus, recalculée périodiquement.

- La méthode Round-Robin est utilisée pour les processus de même priorité.
- La fonction de calcul de la priorité courante utilise:
 - Une priorité initiale de base correspondante au niveau d'importance du processus
(la valeur la plus petite = niveau le plus de priorité)
 - Le temps CPU utilisé ;
la priorité diminue en fonction du temps d'exécution écoulé
 - Peut être ajustée par l'utilisateur lui-même grâce à la primitive ***nice*** (en général pour diminuer le niveau de priorité)

EX 1 : ORDONNANCEMENT DE PROCESSUS

On dispose d'un ordinateur ayant une unité d'échange travaillant en parallèle avec la CPU. Un algorithme d'ordonnancement RR. Soit :

Processus	Temps CPU	E/S	Durée E/S
T1	400 ms	aucune	---
T2	40 ms	Toutes les 10 ms	250 ms
T3	300 ms	aucune	---
T4	60 ms	Toutes les 20 ms	180 ms

Décrire précisément l'évolution du système : instant, nature de l'événement (commutation, demande d'E/S, fin d'E/S), processus élu, état de la file. Donner pour chacun des processus l'instant où il termine son exécution. On prendra un quantum de 100 ms et on supposera qu'ils sont initialement Prêt et rangés dans l'ordre de leur numéro.

Hypothèses :

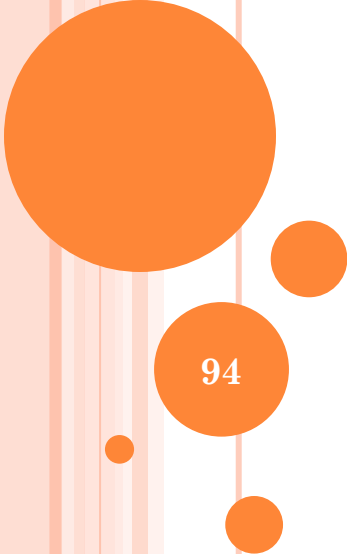
1/ les E/S sont indépendantes et peuvent donc s'effectuer en parallèle.

2/ Si la fin de processus coïncide avec le début d'E/S, la fin de processus est privilégiée

EX 2 : ORDONNANCEMENT DE PROCESSUS

En supposant qu'il existe 4 niveaux de priorité numérotés de 0 à 3 (0 étant la plus forte), reprendre la question précédente en considérant les processus suivants :

Processus	Temps CPU	E/S	Durée E/S	Priorité
T1	400 ms	aucune	---	3
T2	40 ms	Toutes les 10 ms	250 ms	0
T3	300 ms	aucune	---	2
T4	60 ms	Toutes les 20 ms	180 ms	0
T5	400 ms	aucune	---	2



GESTION DE LA MÉMOIRE PRINCIPALE

94

Allocation contiguë (MFT/MVT)

Allocation non contiguë

Va-et-vient (SWAP)

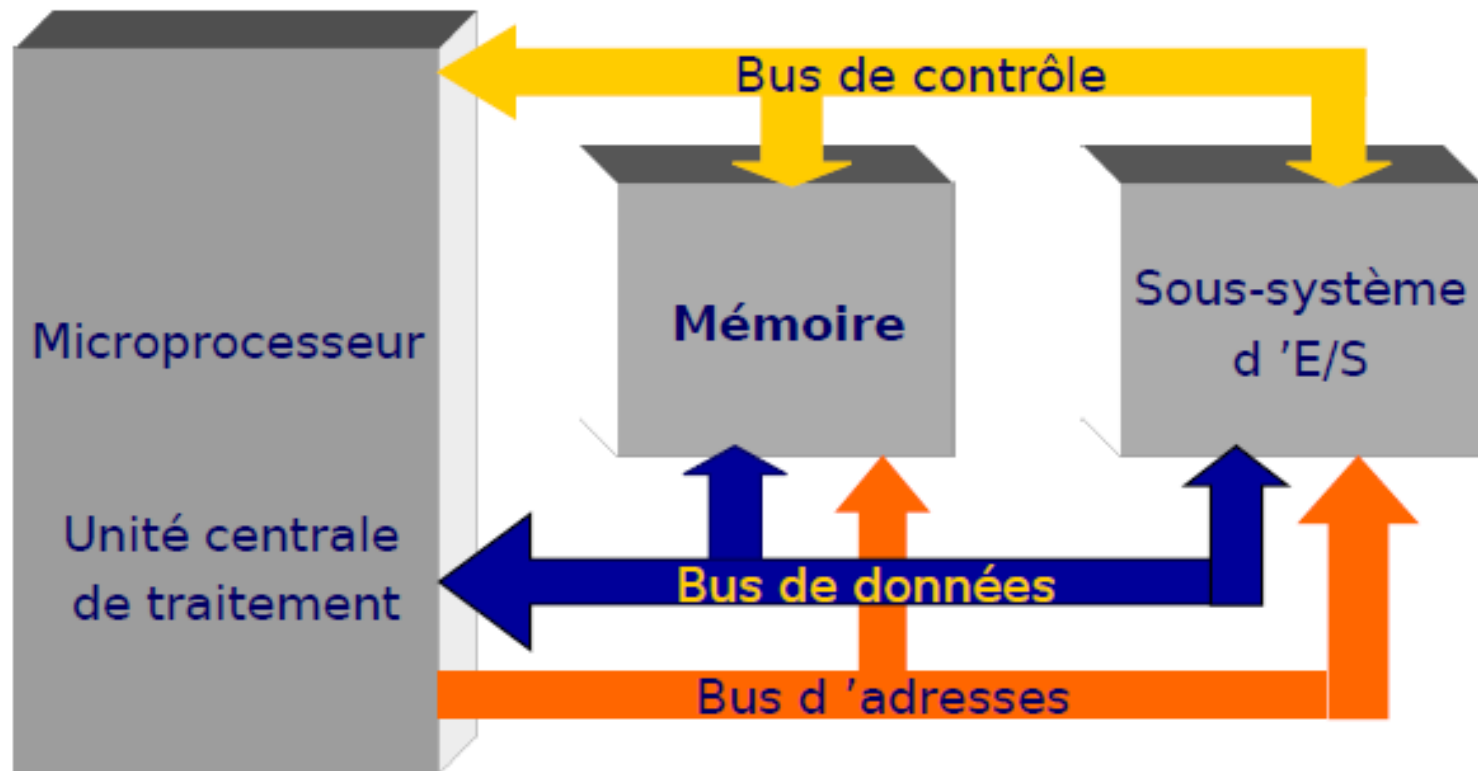
Mémoire virtuelle (Pagination/Segmentation)

GESTION DE LA MÉMOIRE PRINCIPALE (MP)

- Introduction
- Organisation de mémoire en monoprogrammation
- Organisation de mémoire en Multiprogrammation
- Organisation de la mémoire Virtuelle

GESTION DE LA MÉMOIRE PRINCIPALE (MP)

ARCHITECTURE SIMPLIFIÉE D'UN ORDINATEUR



GESTION DE LA MÉMOIRE PRINCIPALE (MP)

INTRODUCTION : LES MÉMOIRES

- Une mémoire est un circuit à semi-conducteur permettant d'enregistrer, de conserver et de restituer des informations (instructions et variables).
- Il y a *écriture* lorsqu'on enregistre des informations en mémoire, *lecture* lorsqu'on récupère des informations précédemment enregistrées.
- Une mémoire peut être représentée comme une armoire de rangement constituée de différents tiroirs.

GESTION DE LA MÉMOIRE PRINCIPALE (MP)

ORGANISATION D'UNE MÉMOIRE

- Chaque tiroir représente alors une case mémoire qui peut contenir un seul élément : des **données**.
- Le nombre de cases mémoires pouvant être très élevé, il est alors nécessaire de pouvoir les identifier par un numéro. Ce numéro est appelé **adresse**.
- Avec une adresse de n bits il est possible de référencer au plus 2^n cases mémoire.

Adresse	Case mémoire
7 = 111	
6 = 110	
5 = 101	
4 = 100	
3 = 011	
2 = 010	
1 = 001	
0 = 000	0001 1010

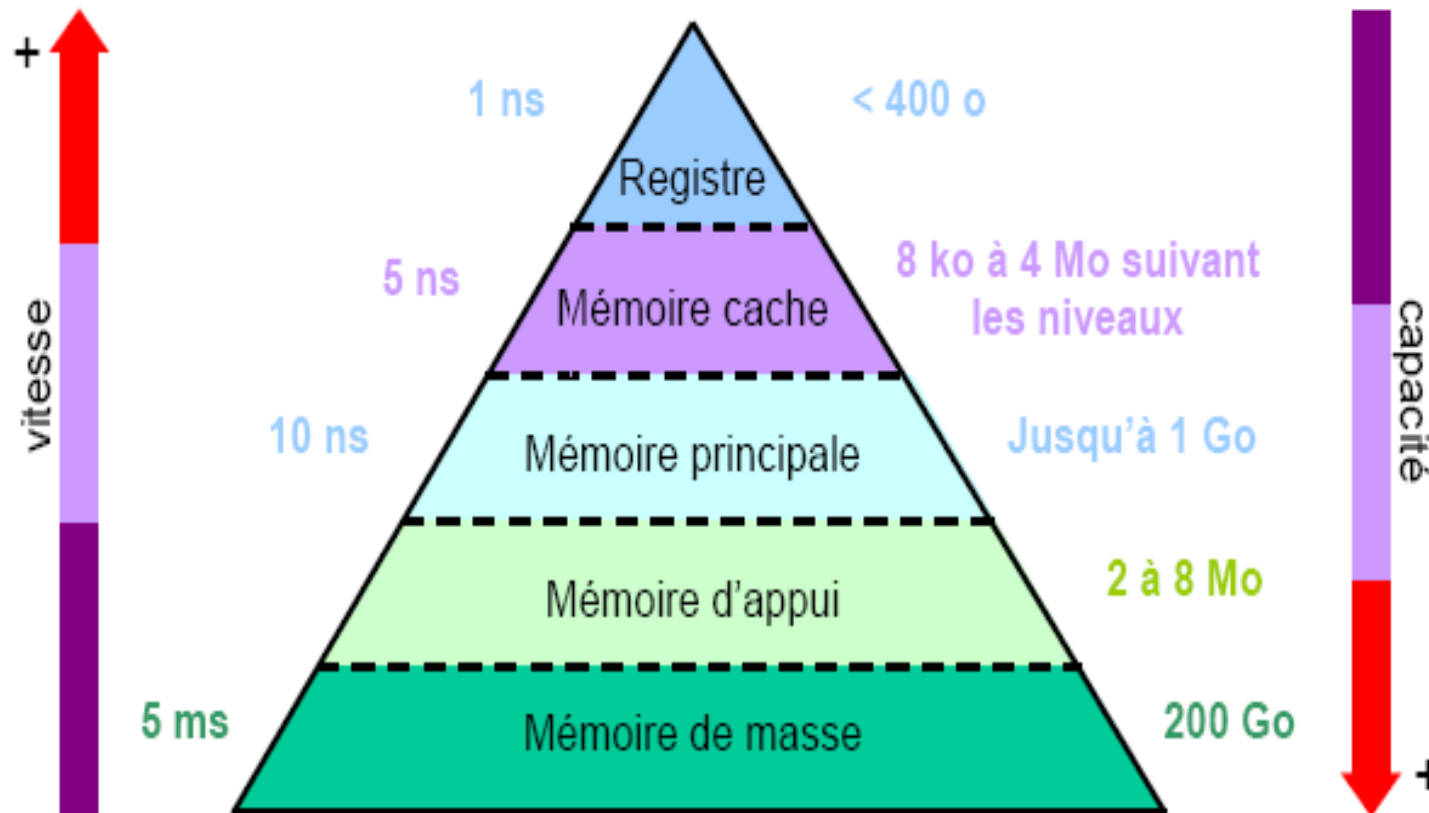
GESTION DE LA MÉMOIRE PRINCIPALE (MP)

CARACTÉRISTIQUES D'UNE MÉMOIRE

- **La capacité** : c'est le nombre total de bits que contient la mémoire. Elle s'exprime aussi souvent en octet.
- **Le format des données** : c'est le nombre de bits que l'on peut mémoriser par case mémoire. On dit aussi que c'est la largeur du mot mémorisable.
- **Le temps d'accès** : c'est le temps qui s'écoule entre l'instant où a été lancée une opération de lecture/écriture en mémoire et l'instant où la première information est disponible sur le bus de données.
- **Le temps de cycle** : il représente l'intervalle minimum qui doit séparer deux demandes successives de lecture ou d'écriture.
- **Le débit** : c'est le nombre maximum d'informations lues ou écrites par seconde.
- **La volatilité** : elle caractérise la permanence des informations dans la mémoire. L'information stockée est volatile si elle risque d'être altérée par un défaut d'alimentation électrique et non volatile dans le cas contraire.

GESTION DE LA MÉMOIRE PRINCIPALE (MP)

TYPES DE MÉMOIRES



GESTION DE LA MÉMOIRE PRINCIPALE (MP)

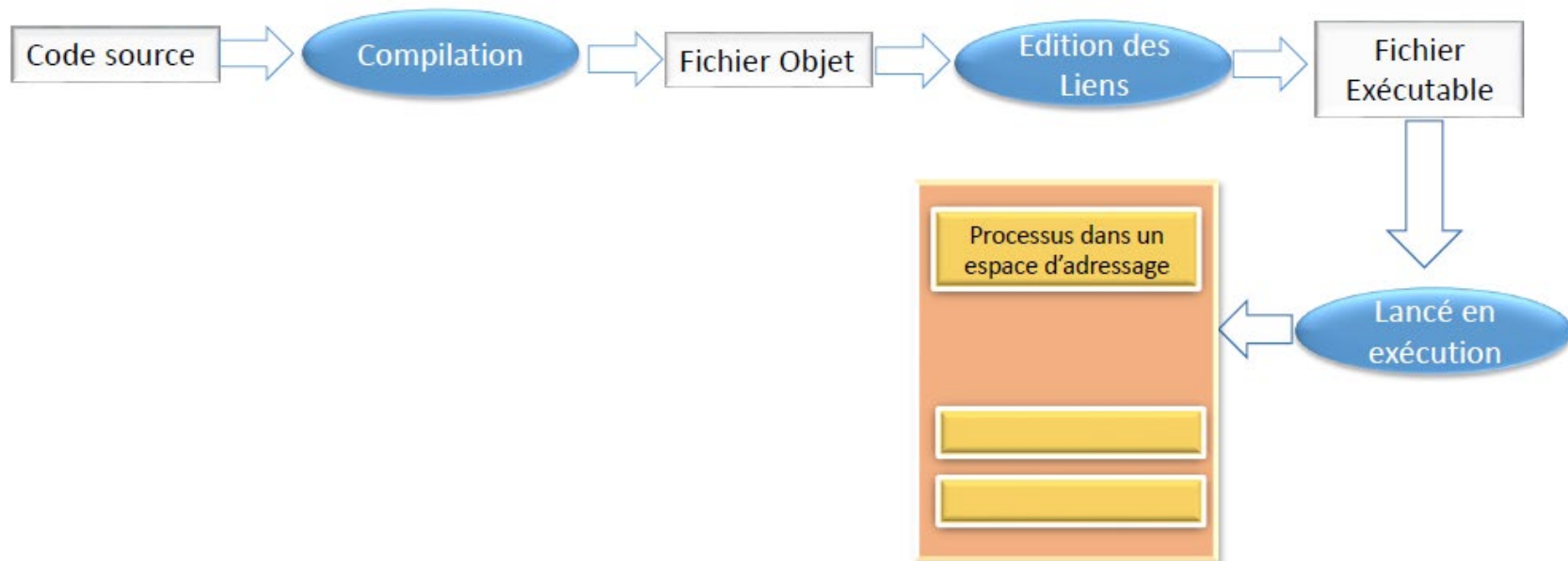
TYPES DE MÉMOIRES

- La gestion de la mémoire est l'une des tâches fondamentales d'un système d'exploitation.
- Mémoire Principale : destinée à loger les programmes exécutables pris en charge par le SE en vue de leur exécution. (mémoire vive de type RAM ou autre)
- Mémoire Cache : non obligatoire mais permet d'optimiser les performances d'accès à la MP. (mémoire matérielle située au niveau de la CPU)
- Mémoire virtuelle : technique logicielle opérée par le SE pour une gestion optimisée.

GESTION DE LA MÉMOIRE PRINCIPALE (MP)

CYCLE DE VIE D'UN PROGRAMME

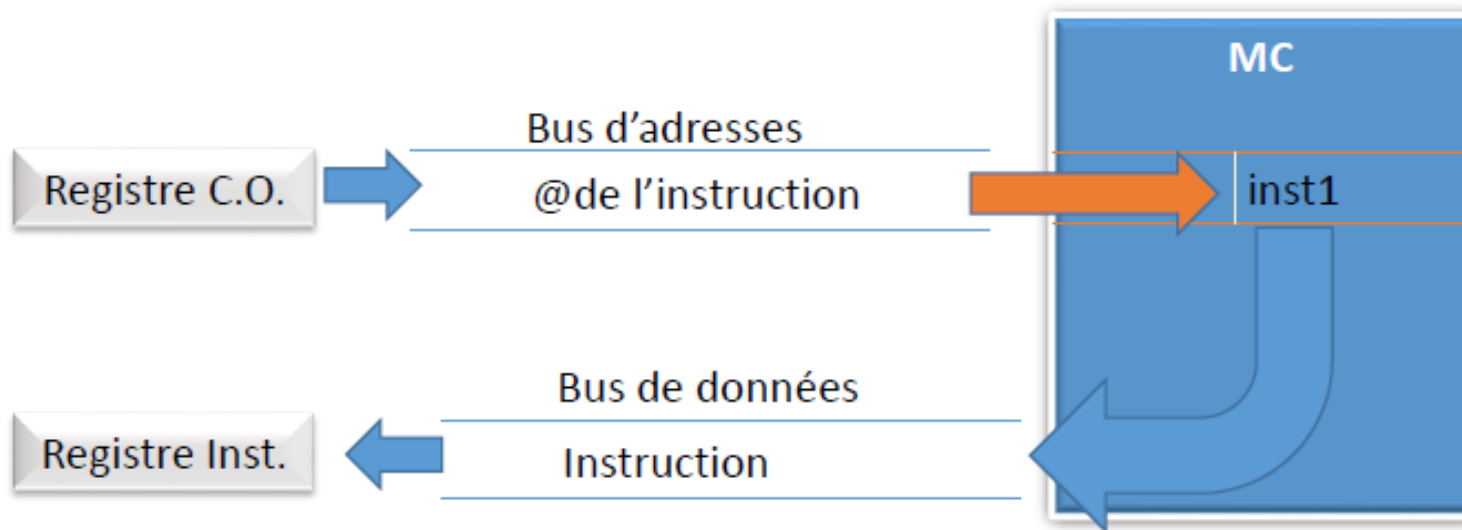
- La mémoire est une ressource importante et limitée qui doit être gérée d'une façon optimale.
- Plusieurs types de mémoires peuvent être utilisées lors de l'exécution d'un programme.



GESTION DE LA MÉMOIRE PRINCIPALE (MP)

FICHER EXÉCUTABLE

- Une suite d'instructions machine possédant chacune une **adresse logique** (compilation).
- Chaque **adresse logique** doit être traduite en une **adresse physique** (édition des liens ou chargement).



GESTION DE LA MÉMOIRE PRINCIPALE (MP)

MÉMOIRE PHYSIQUE VS MÉMOIRE LOGIQUE

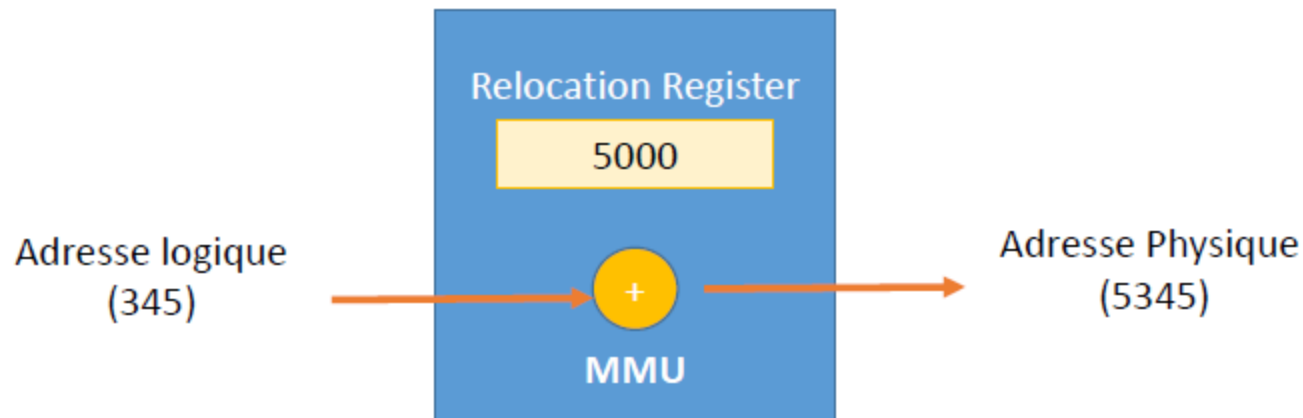
- *Mémoire physique* :
 - Mémoire principale : la RAM de la machine
 - Adresses physiques : adresses de cette mémoire
- *Mémoire logique* :
 - Espace d'adressage du processus
 - Adresses logiques : adresses dans cet espace

Adresse logique $\neq$ adresse physique

GESTION DE LA MÉMOIRE PRINCIPALE (MP)

MEMORY MANAGEMENT UNIT (MMU)

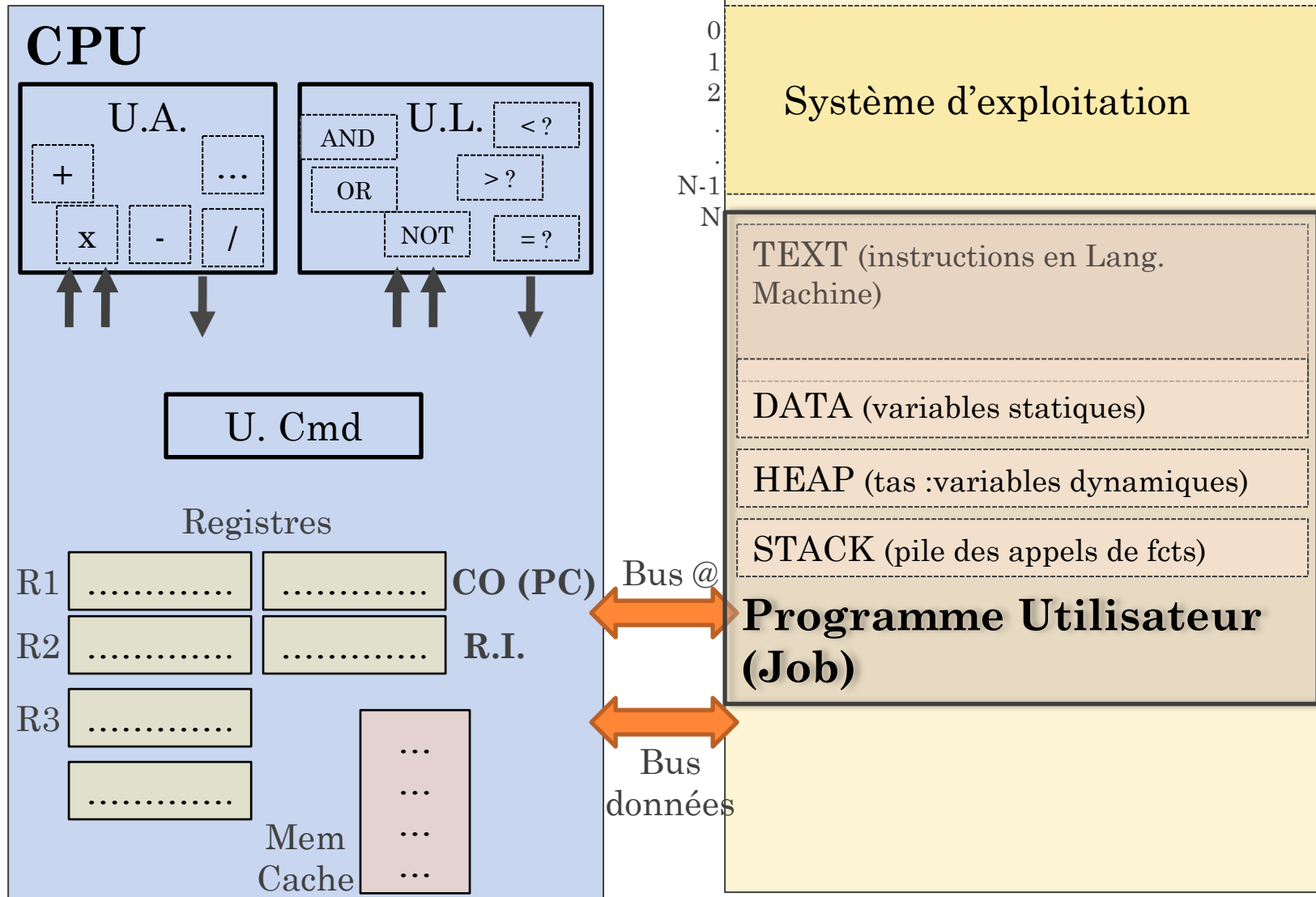
- Le MMU fait partie du CPU effectue la conversion des adresses logiques en adresses physiques



GESTION DE LA MÉMOIRE PRINCIPALE (MP)

STRUCTURE D'UN PROGRAMME EN M.P

M.P.

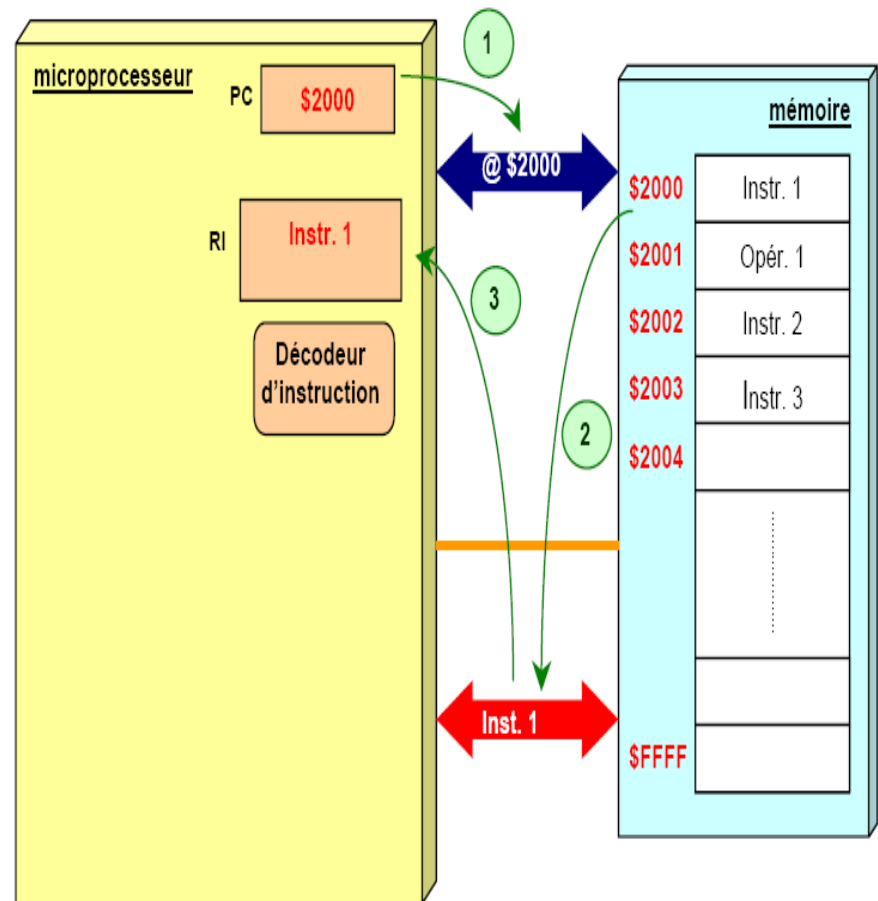


GESTION DE LA MÉMOIRE PRINCIPALE (MP)

CYCLE D'EXÉCUTION D'UNE INSTRUCTION (1)

1. Le PC contient l'adresse de l'instruction suivante du programme. Cette valeur est placée sur le bus d'adresses par l'unité de commande qui émet un ordre de lecture.
2. Au bout d'un certain temps (temps d'accès à la mémoire) le contenu de la case mémoire sélectionnée est disponible sur le bus des données.
3. L'instruction est stockée dans le registre instruction du processeur.

Phase 1: Recherche de l'instruction à traiter

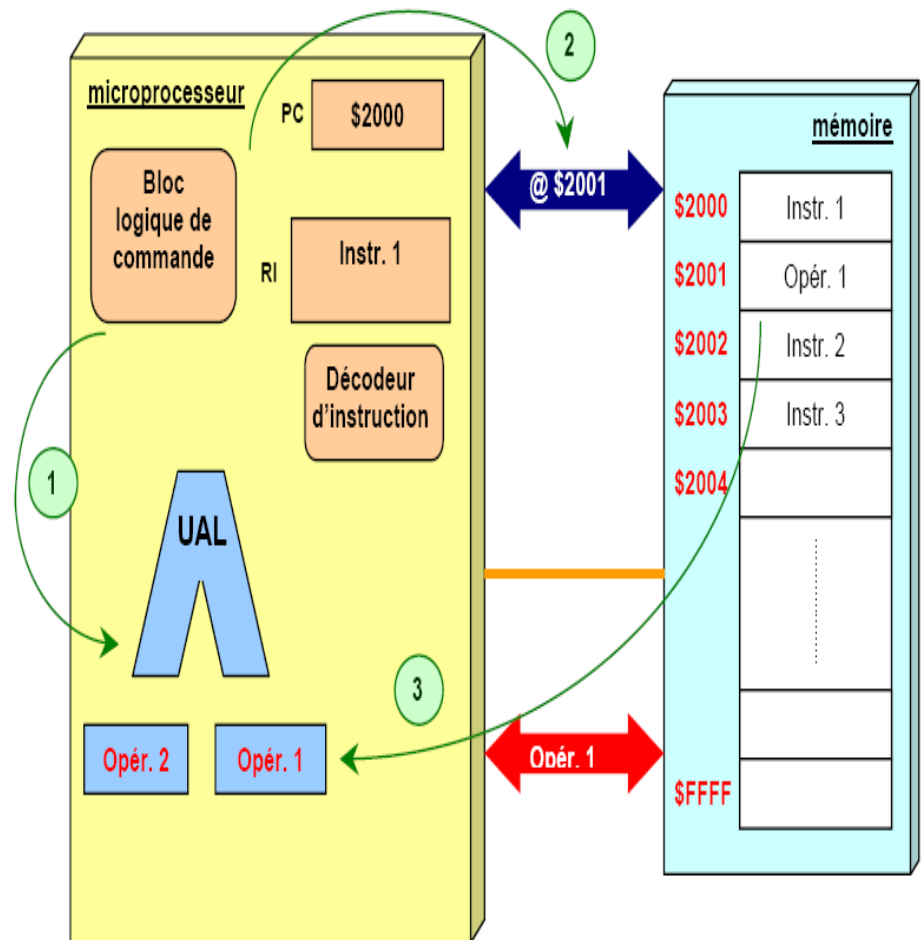


GESTION DE LA MÉMOIRE PRINCIPALE (MP)

CYCLE D'EXÉCUTION D'UNE INSTRUCTION (2)

1. L'unité de commande transforme l'instruction en une suite de commandes élémentaires nécessaires au traitement de l'instruction.
2. Si l'instruction nécessite une donnée en provenance de la mémoire, l'unité de commande récupère sa valeur sur le bus de données.
3. L'opérande est stockée dans un registre.

○ Phase 2: *Décodage de l'instruction et recherche de l'opérande*

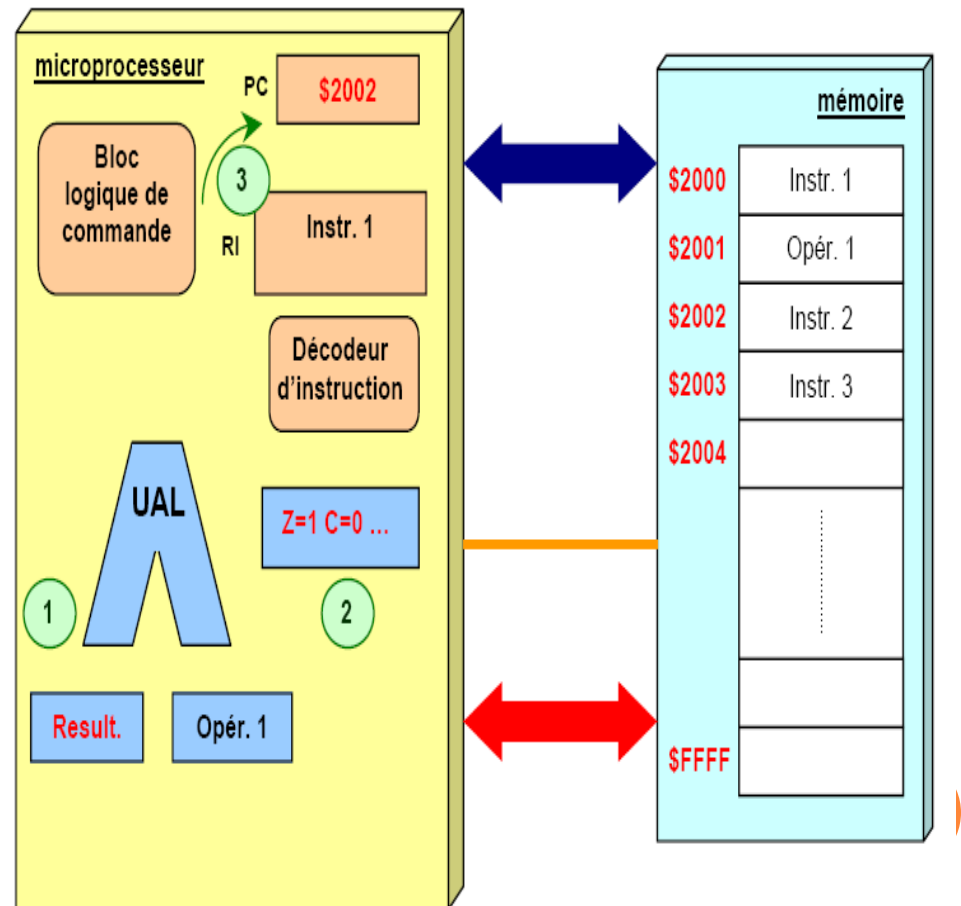


GESTION DE LA MÉMOIRE PRINCIPALE (MP)

CYCLE D'EXÉCUTION D'UNE INSTRUCTION (3)

1. Le microprogramme réalisant l'instruction est exécuté.
2. Les indicateurs d'état sont positionnés (registre d'état).
3. L'unité de commande positionne le PC pour l'instruction suivante.

○ Phase 3: Exécution de l'instruction



GESTION DE LA MÉMOIRE PRINCIPALE (MP)

CYCLE D'EXÉCUTION : RÉSUMÉ

- 
1. Charger l'instruction à exécuter dans RI (depuis MP)
 2. Décoder l'opération (interpréter le code opération)
 3. Préparer les opérandes (accès éventuels à la MP)
 4. Exécuter l'opération
 5. Incrémenter le registre CO (i.e. **CO++**)

GESTION DE LA MÉMOIRE PRINCIPALE (MP)

- Mémoire principale (centrale) : un grand tableau de mots ou cases, chacun possédant sa propre adresse.

La gestion de la MP est fondée sur 3 aspects :

- **Allocation de mémoire** : chargement d'un programme exécutable vers la MP (avant de démarrer une exécution).
 - Attribution de mémoire
 - Libération de mémoire
- **Gestion des accès** : translation (i.e. traduction) d'une *adresse logique* en une *adresse physique* (au cours d'une exécution).
- **Protection des espaces mémoire utilisées** : veiller à ce que les programmes en mémoire ne puissent pas interférer entre eux.

GESTION DE LA MÉMOIRE PRINCIPALE (MP)

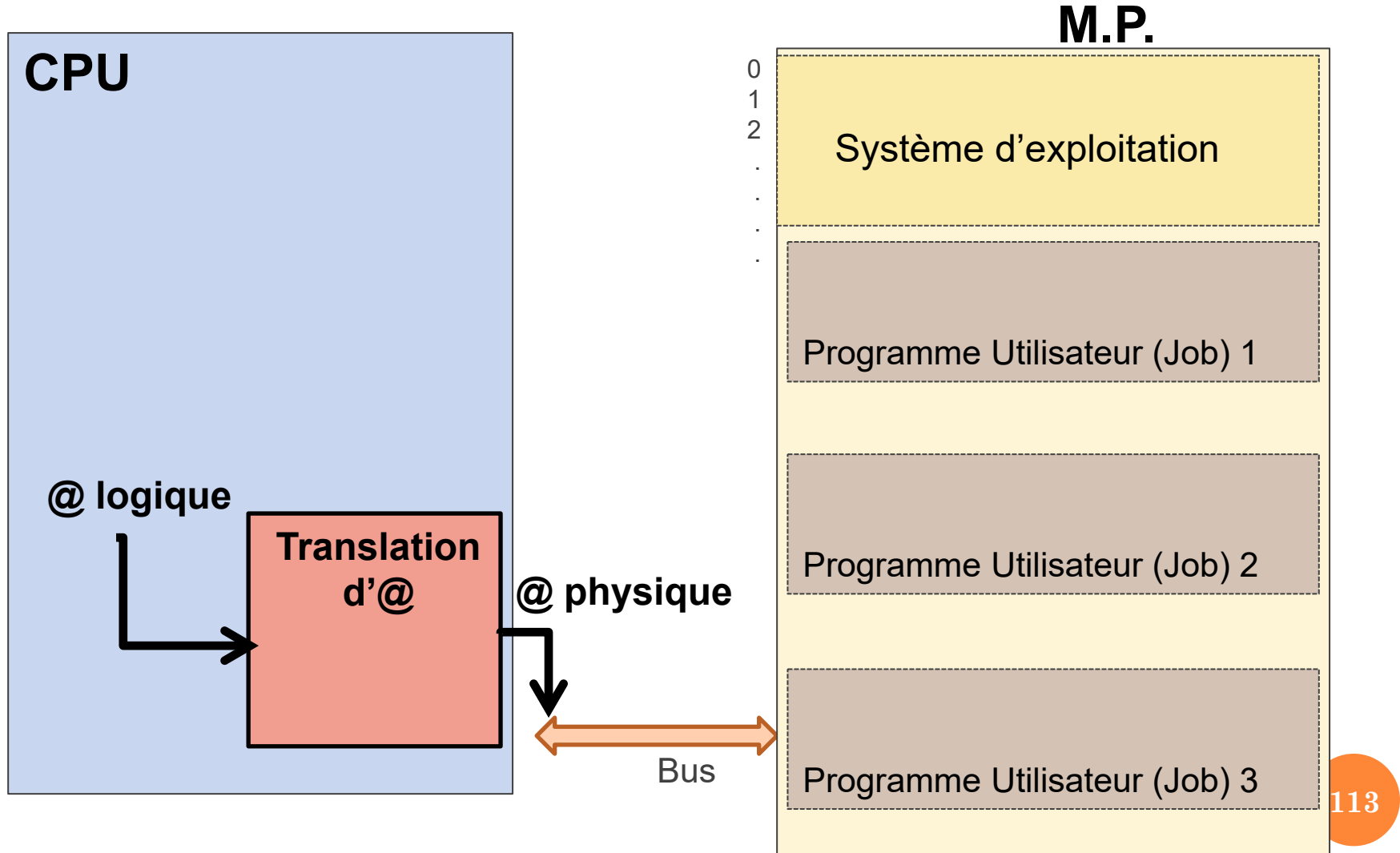
GESTION DES ACCÈS MÉMOIRE : NOTION D'ADRESSAGE

L'adressage est un mécanisme permettant d'accéder à une case en mémoire dans laquelle une information est rangée.

- Il existe trois catégories d'adresses:
 - **Adresse symbolique** : manipulée au niveau d'un programme écrit dans un langage de programmation (Exp : pointeur ou identificateur de variable dans un code source en C).
 - **Adresse logique** : générée par le CPU dans le cadre de l'exécution d'une instruction en langage machine (Exp : adresse d'une donnée ou instruction dans code exécutable).
 - **Adresse physique** : emplacement réel en mémoire manipulée par le contrôleur hardware d'accès à la mémoire vive (Exp : Read/@146/RegA ou Write/Reg5/@1306).

GESTION DE LA MÉMOIRE PRINCIPALE (MP)

GESTION DES ACCÈS MÉMOIRE : TRANSLATION D'ADRESSE



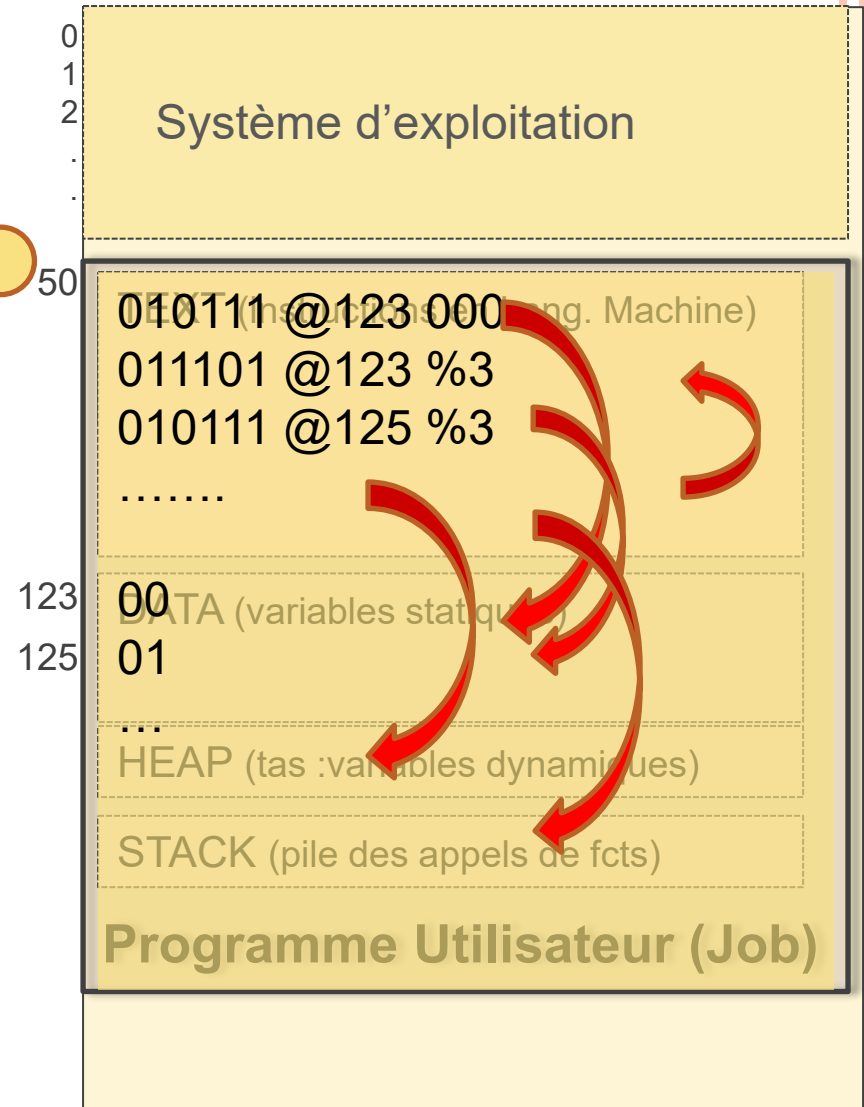
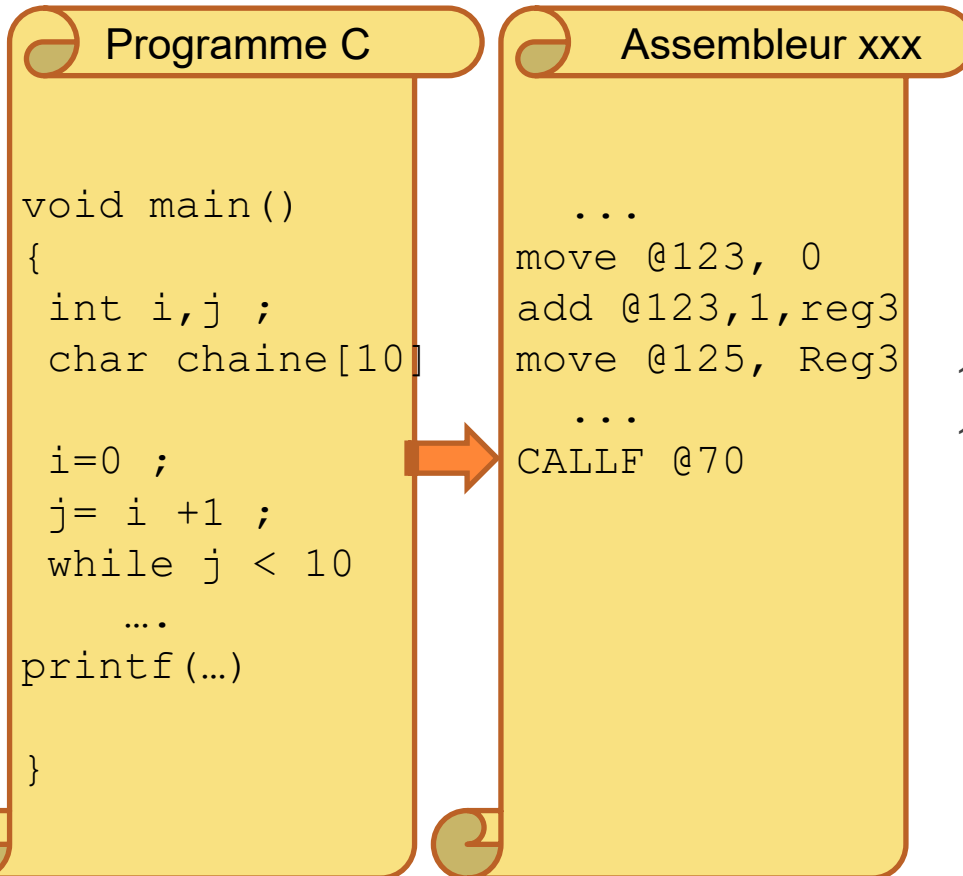
GESTION DES ACCÈS MÉMOIRE

TRANSLATION D'ADRESSE – MONOPROGRAMMATION

M.P.

Code absolu :

@logique = @physique



GESTION DES ACCÈS MÉMOIRE

TRANSLATION D'ADRESSE – MULTIPROGRAMMATION

M.P.

Code translatable : Exemple

@ physique = @ logique + @ base

Adresse de base = 50

Programme C

```
void main()  
{  
  int i,j ;  
  char chaine[10]  
  
  i=0 ;  
  j= i +1 ;  
  while j < 10  
  ...  
  printf(...)  
}
```

Assembleur xxx

```
...  
move @73, 0  
add @73,1,reg3  
move @75, Reg3  
if(LT,@75,10)      goto @04  
...  
...
```

50

010111 @73 000
011101 @73 %3
010111 @75 %3
.....

123

00 DATA (variables statiques)

125

01

...
HEAP (tas :variables dynamiques)

STACK (pile des appels de fcts)

Programme Utilisateur (Job)

2

2

ALLOCATIONS DE MÉMOIRE

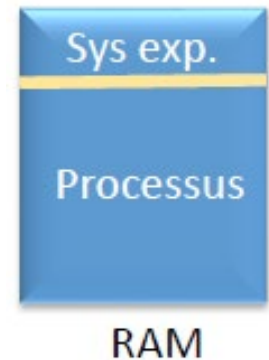
EVOLUTION DES TECHNIQUES

- Le TOUT dans la MP : l'application gère tout !
Pas de séparation SE & Programme Utilisateur.
- Monoprogrammation : SE (ou moniteur) + 1 prog. User .
- Multiprogrammation : plusieurs programmes User logés au sein de la MP :
 - Multiprogramming with a Fixed Number of Tasks (MFT)
 - Multiprogramming with a Variable Number of Tasks (MVT)
 - Mémoire virtuelle : Va et vient (swap)
 - Pagination et segmentation

GESTION DE LA MÉMOIRE PRINCIPALE (MP)

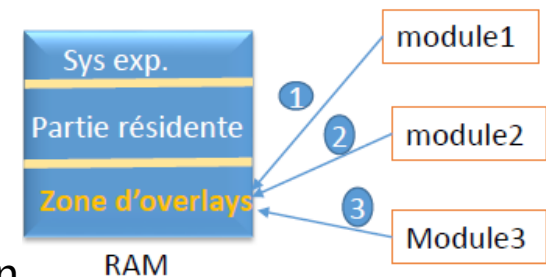
MONOPROGRAMMATION

- Mémoire réservée au système d'exploitation.
- Mémoire réservée au seul processus en exécution.
- Allocation contiguë en mémoire : ensemble de mots contigus insécables -- espace d'adressage linéaire.
- La taille du programme ne doit pas dépasser la taille réservée au prog user



→ Solution : Stratégie *d'overlays (recouvrement)*.

- **Stratégie de recouvrement** : découper le prog en modules de telle sorte que le module ne se charge en mémoire que lors de son exécution.



GESTION DE LA MÉMOIRE PRINCIPALE (MP)

MULTIPROGRAMMATION : PLUSIEURS PROCESSUS EN MÉMOIRE

- L'espace mémoire est divisé en N partitions de taille fixe et de préférence inégales.
- Allocation des processus aux partitions :
 - Une file d'attente par partition
 - Une file d'attente pour toutes les partitions
- Techniques utilisées :
 - **Allocation contigüe** : ensemble de mots contigus insécables -
- espace d'adressage linéaire.
 - Statique : nombre et taille fixes → MFT
 - Dynamique : nombre et taille variables → MVT
 - **Allocation non contigüe** : ensemble de mots sécables, c.-à-d. le programme peut être divisé en plus petits morceaux, chaque morceau étant lui même un ensemble de mots contigus.
 - Morceaux de tailles fixes → Pagination
 - Morceaux de tailles variables → Segmentation

GESTION DE LA MÉMOIRE PRINCIPALE (MP)

PARTITIONS CONTIGÜES FIXES – MFT (1/3)

- La mémoire est divisée en un nombre fixe de partitions de tailles non nécessairement identiques.
- Dans chaque partition on ne peut loger qu'un seul processus unique (un seul programme).
- Ce partitionnement se fait au démarrage du système.
- Le système d'exploitation maintient une table de description des partitions.

Adresse	Taille	Etat
312k	8K	allouée
320k	32K	allouée
352k	32K	libre
384k	120K	libre
504k	520K	allouée

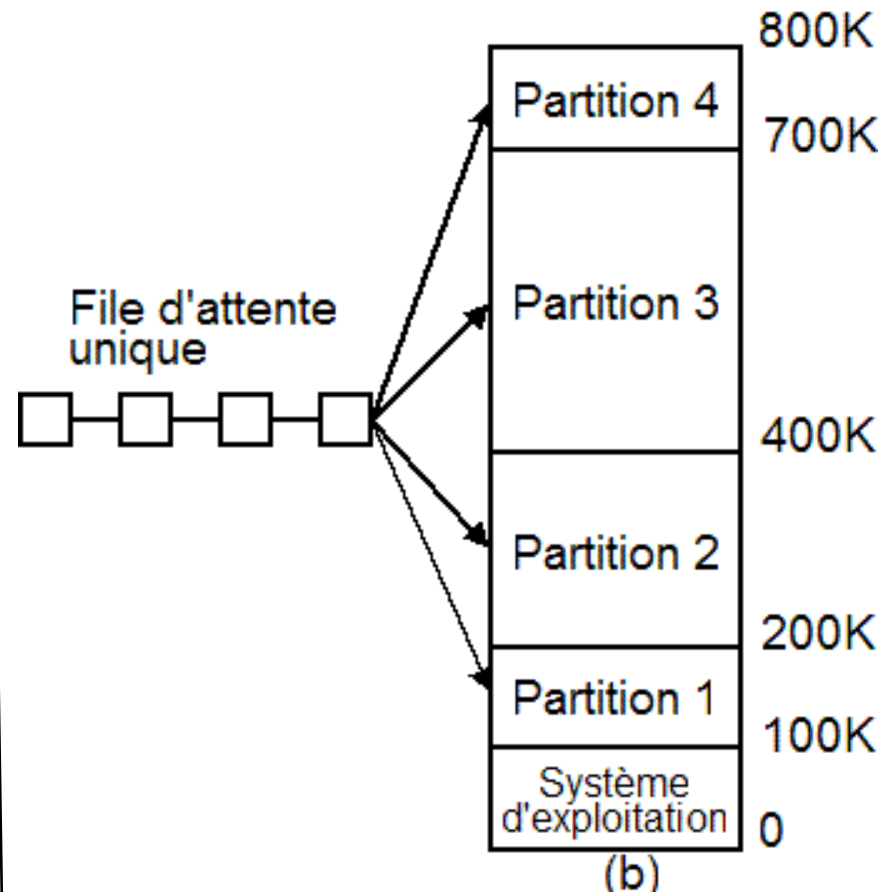
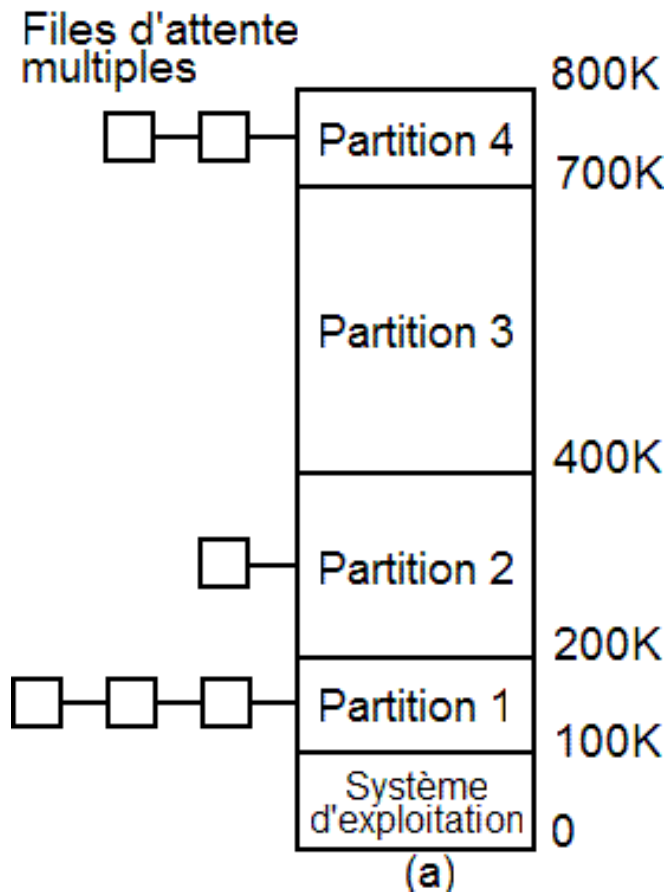
GESTION DE LA MÉMOIRE PRINCIPALE (MP)

PARTITIONS CONTIGÜES FIXES – MFT (2/3)

- Quand un processus arrive, il peut être placé dans la file d'attente des entrées de la plus petite des partitions assez larges pour le contenir.
- Le tri des processus qui arrivent en différentes files d'attente présente des inconvénients lorsque la file d'attente pour une grande partition est vide tandis que celle d'une petite partition est pleine (figure a).
- Une autre organisation possible consiste à gérer une seule file d'attente (figure b).
- Dès qu'une partition devient libre, toute tâche placée en tête de file d'attente et dont la taille convient peut être chargée dans cette partition vide et exécutée.
- **Inconvénient** : fragmentation interne.

GESTION DE LA MÉMOIRE PRINCIPALE (MP)

PARTITIONS CONTIGÜES FIXES – MFT (3/3)



GESTION DE LA MÉMOIRE PRINCIPALE (MP)

PARTITIONS CONTIGÜES DYNAMIQUES – MVT

- La mémoire est partitionnée dynamiquement selon la demande → le nombre et la taille des zones varient dynamiquement.
- Lorsqu'un processus est terminé, sa partition est récupérée pour être réutilisée (complètement ou partiellement) par d'autres processus.
- Pour cela le gestionnaire de la mémoire doit garder trace des partitions occupées et /ou des partitions libres.
- Le lancement des processus dans les partitions se fait selon des algorithmes de placement (allocation).
- **Inconvénient** : fragmentation externe.

GESTION DE LA MÉMOIRE PRINCIPALE (MP)

ALGORITHMES DE PLACEMENT (D'ALLOCATION)

- La mémoire est formée d'un ensemble de zones libres et de zones occupées (allouées).
- Allouer un programme P de taille Taille (P) : trouver une zone libre telle que Taille (zone libre) $\geq$ Taille (P).
- On distingue les algorithmes de placement suivants:
 - **Stratégie du premier qui convient (First Fit):** la liste des partitions libres est triée par ordre des adresses croissantes. La recherche commence par la partition libre de plus basse adresse et continue jusqu'à la rencontre de la première partition dont la taille est au moins égale à celle du processus en attente.
 - **Stratégie du meilleur qui convient (Best Fit):** on alloue la plus petite partition dont la taille est au moins égale à celle du processus en attente. La table des partitions libres est de préférence triée par tailles croissantes.
 - **Stratégie du pire qui convient (Worst Fit):** on alloue au processus la partition de plus grande taille.

GESTION DE LA MÉMOIRE PRINCIPALE (MP)

EXEMPLE DE TECHNIQUE MVT

Soit une MC dont la table des partitions libres est la suivante :

Adresse	Taille
352K	32K
504K	520K

Soit les demandes suivantes

P4 (24 K),
P5 (128 K),
P6 (256 K) .

Evolution de la table des partitions libres:

First Fit:

init	P4	P5	P6
32K	8K	8K	8K
520K	520K	392K	136K

Best Fit:

init	P4	P5	P6
32K	8K	8K	8K
520K	520K	392K	136K

Worst Fit:

init	P4	P5	P6
32K	32K	32K	32K
520K	496K	368K	112K

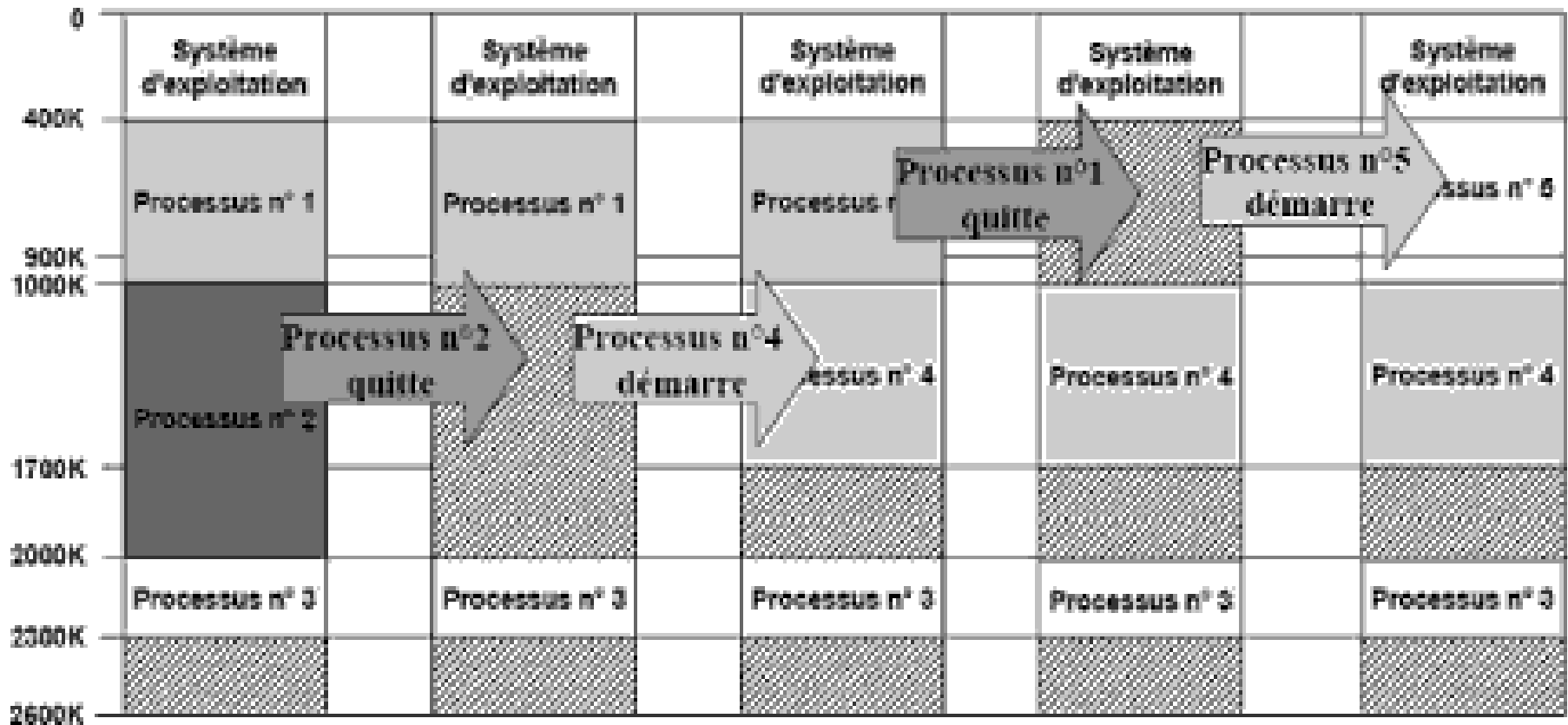
GESTION DE LA MÉMOIRE PRINCIPALE (MP)

INDICATEURS DE PERFORMANCE

- La performance de l'allocateur mémoire se mesure en espace non utilisé ou « gaspillé » sur une période de temps (plus ou moins longue).
- ***Fragmentation*** = espace non utilisé.
- Deux catégories de fragmentations :
 - *Fragmentation interne* : espace alloué au processus mais non utilisé (au sein de la même partition mémoire).
 - *Fragmentation externe* : espace non alloué mais non utilisé ou non utilisable (entre les partitions mémoire).

GESTION DE LA MÉMOIRE PRINCIPALE (MP)

FRAGMENTATION ET COMPACTAGE



GESTION DE LA MÉMOIRE PRINCIPALE (MP)

INCONVÉNIENTS DE L'ALLOCATION CONTIGÜE

- Exigence d'allouer le programme en une zone d'un seul tenant.
- Fragmentation
 - Fragmentation interne dans le cas des partitions fixes (MFT).
 - Fragmentation externe dans le cas des partitions dynamiques (MVT).
- Si un processus nécessite un espace d'avantage, le SE augmente la taille de sa partition en lui ajoutant des partitions voisines s'ils existent ou le processus est déplacé ou bien les processus voisins sont déplacés → Nécessité d'une opération de compactage de la mémoire.

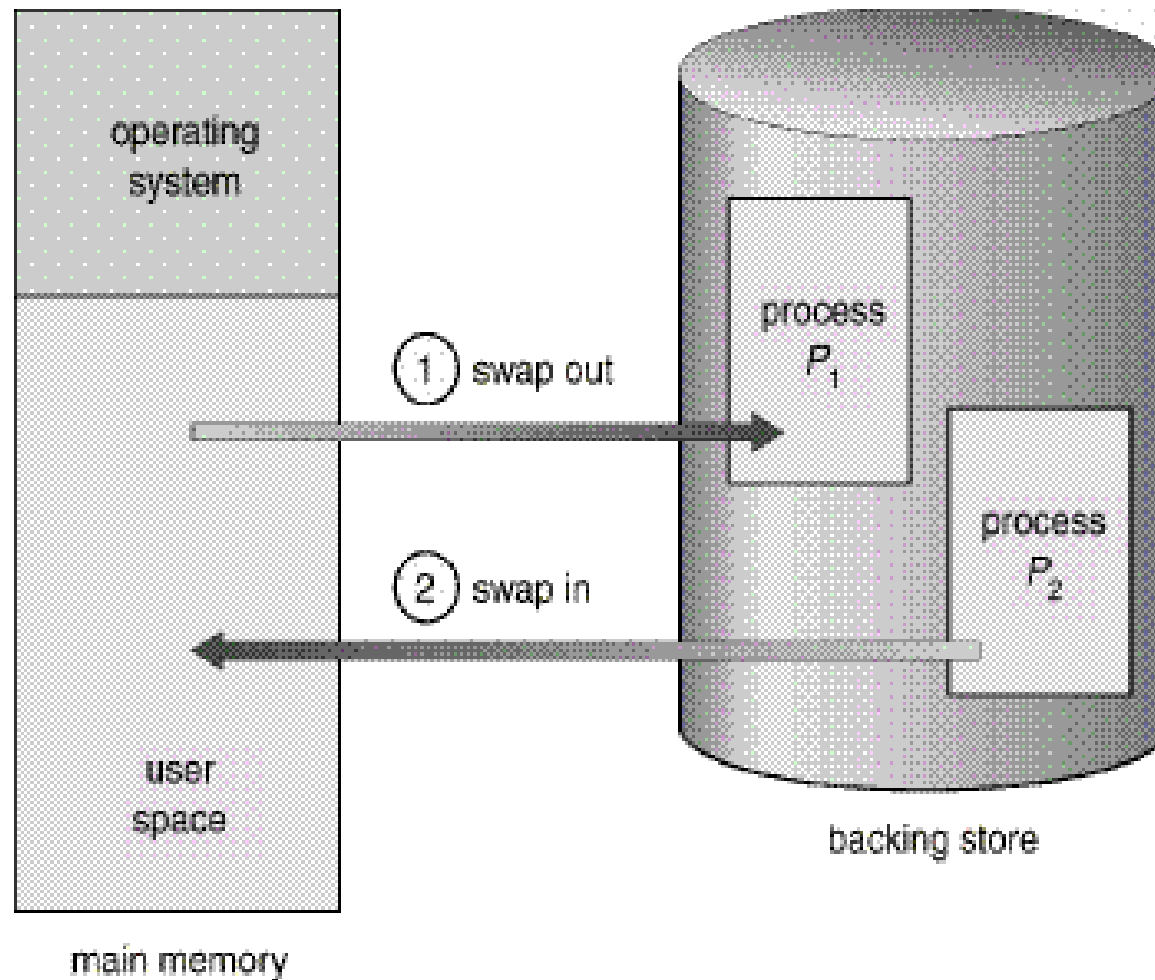
GESTION DE LA MÉMOIRE PRINCIPALE (MP)

ALLOCATION NON CONTIGÛE : VA ET VIENT (1/3)

- Parfois la mémoire principale est insuffisante pour maintenir tous les processus courants actifs (Taille des processus en cours d'exécution > la taille de la mémoire physique).
- Déplacer temporairement les processus supplémentaires sur une mémoire provisoire (partie réservée du disque = zone d'échange (swap)) et les charger pour qu'ils s'exécutent dynamiquement.
- La stratégie appelée **va-et-vient** (*swap*) consiste à considérer chaque processus dans son intégralité.
- En cas de réquisition du processeur, le programme en cours doit être sauvegardé sur disque avant le chargement en mémoire principale de son successeur, dans sa totalité, pour exécution.

GESTION DE LA MÉMOIRE PRINCIPALE (MP)

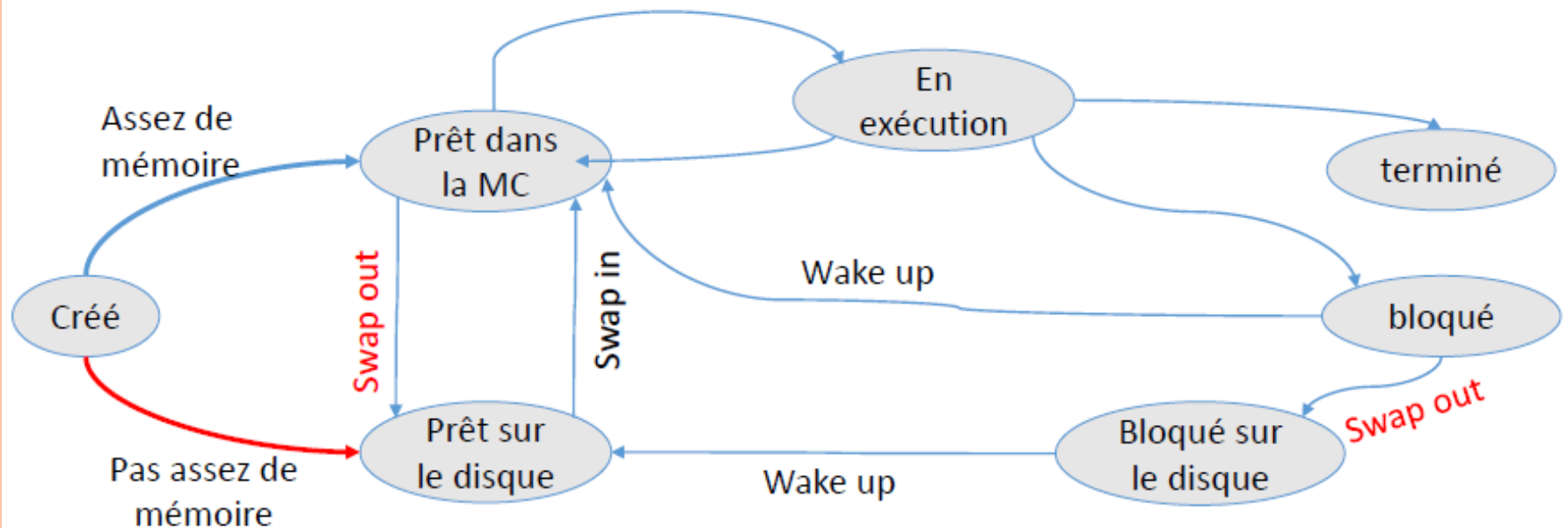
ALLOCATION NON CONTIGÜE : VA ET VIENT (2/3)



GESTION DE LA MÉMOIRE PRINCIPALE (MP)

ALLOCATION NON CONTIGÜE : VA ET VIENT (3/3)

○ Diagramme d'états



GESTION DE LA MÉMOIRE PRINCIPALE (MP)

ALLOCATION NON CONTIGÜE : MÉMOIRE VIRTUELLE

- Taille du programme > taille mémoire disponible → Le SE conserve les parties (du processus) en cours d'utilisation en mémoire et le reste sur le disque → Nécessite une zone d'échange (swap) comme pour le Va-et-Vient.
- Mémoire qui a une taille irréaliste plus grande que la mémoire physique.
- Permet aux programmes d'avoir une taille plus grande que la mémoire physique.
- La taille de la mémoire virtuelle est fixée par la taille des registres.
 - Registre de 32 bits → Adresse logiques des 0 à $2^{32}-1$ → Un espace d'adressage de 4 Go (mots de 1 octet).
- Quand un programme attend le chargement d'une partie de lui-même → il est en attente d'E/S.
- ***Mémoire virtuelle : Pagination & Segmentation***

GESTION DE LA MÉMOIRE PRINCIPALE (MP)

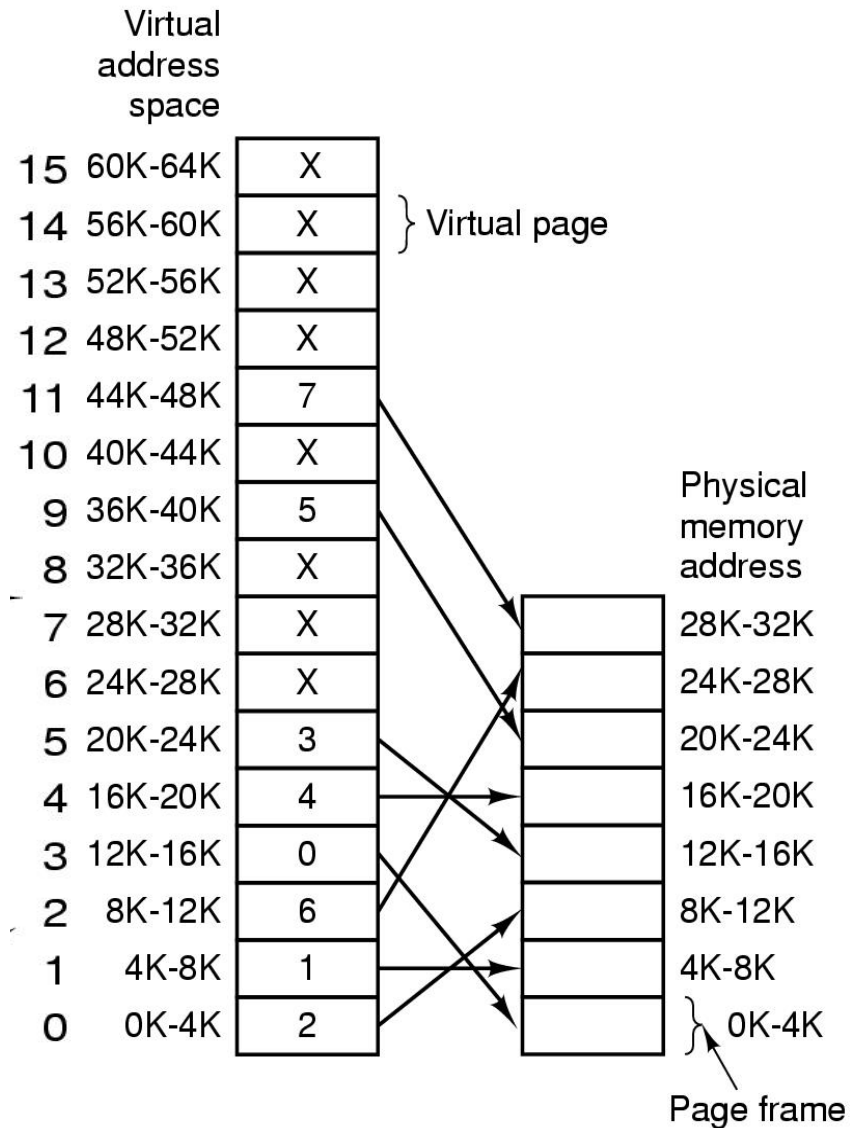
ALLOCATION NON CONTIGUË – PAGINATION (1/)

- La pagination permet d'avoir en mémoire un processus dont les adresses sont non contiguës.
- La solution de pagination consiste à découper l'espace d'adressage d'un programme, ou **espace virtuel**, en zones de taille fixe appelée **pages**.
- La mémoire physique est également découpée en **cadres** ou **cases** (frame) ayant la taille d'une page de sorte que chaque page peut être implantée dans n'importe quelle case de la mémoire réelle.
- Les pages sont successives et les cases également mais deux pages successives ne sont pas forcément placées dans deux cases successives.

GESTION DE LA MÉMOIRE PRINCIPALE (MP)

ALLOCATION NON CONTIGUË – PAGINATION (/)

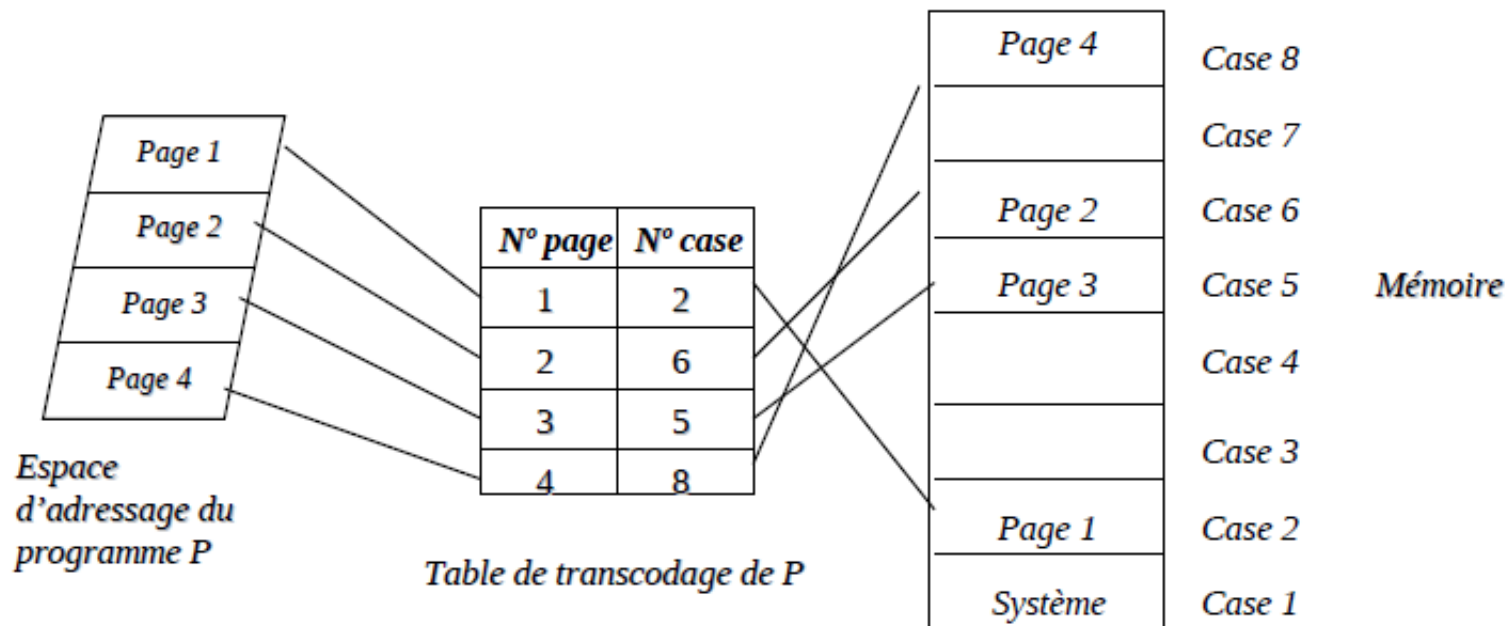
- Processeur 16 bits $\rightarrow 2^{16}$ adresses virtuelles
- Mémoire logique de taille 64 Ko
- Mémoire physique de 32 Ko
- Page de 4 Ko
- Nbr pages virtuelles = $64/4 = 16$
- Nbr cadres de pages = $32/4 = 8$



GESTION DE LA MÉMOIRE PRINCIPALE (MP)

PAGINATION – TABLE DE PAGES

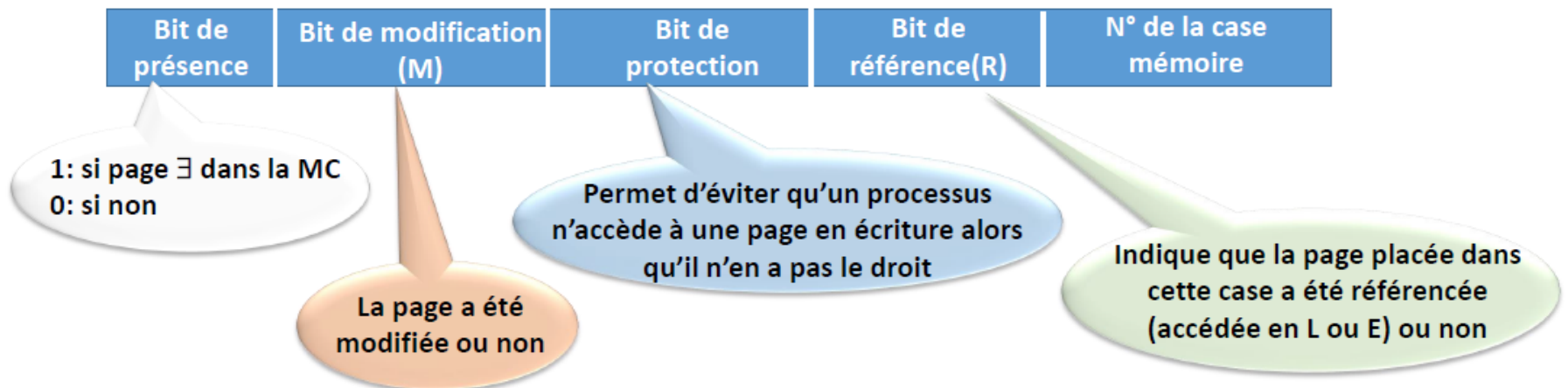
- La traduction des adresses logiques en adresses physiques utilise une table des pages qui est située en mémoire centrale ou dans des registres, et dans laquelle les entrées successives correspondent aux pages virtuelle consécutives.
- Chaque processus dispose de sa propre table de pages. Elle décrit toutes les pages du processus.



GESTION DE LA MÉMOIRE PRINCIPALE (MP)

PAGINATION – TABLE DE PAGES

- Une entrée de la table d'indice i décrit la page N° i et contient généralement le numéro de la case mémoire qui contient la page mais également plusieurs informations nécessaires pour le SE.



GESTION DE LA MÉMOIRE PRINCIPALE (MP)

PAGINATION – TRADUCTION D'ADRESSES (1^{ÈRE} MÉTHODE) (1/)

- 1) Détermination de l'adresse logique : $AL = (p, d)$
- p : numéro de la page
 - d : déplacement dans la page (offset)
- $p = \text{Adresse logique décimale} \mathbf{div}$ taille de page.
 - $d = \text{Adresse logique décimale} \mathbf{mod}$ taille de page.
 - Exemple : cherchons l'AL en fonction de x et y associée à l'adresse 22480 dans l'espace d'adressage. Avec 4 ko est la taille de la page. Chaque page peut contenir 4096 adresses car un mot mémoire = 1 octet.
 - $p = 22480 \div 4096 = 5.$
 - $d = 22480 \bmod 4096 = 2000 .$
- $AL = (5, 2000).$

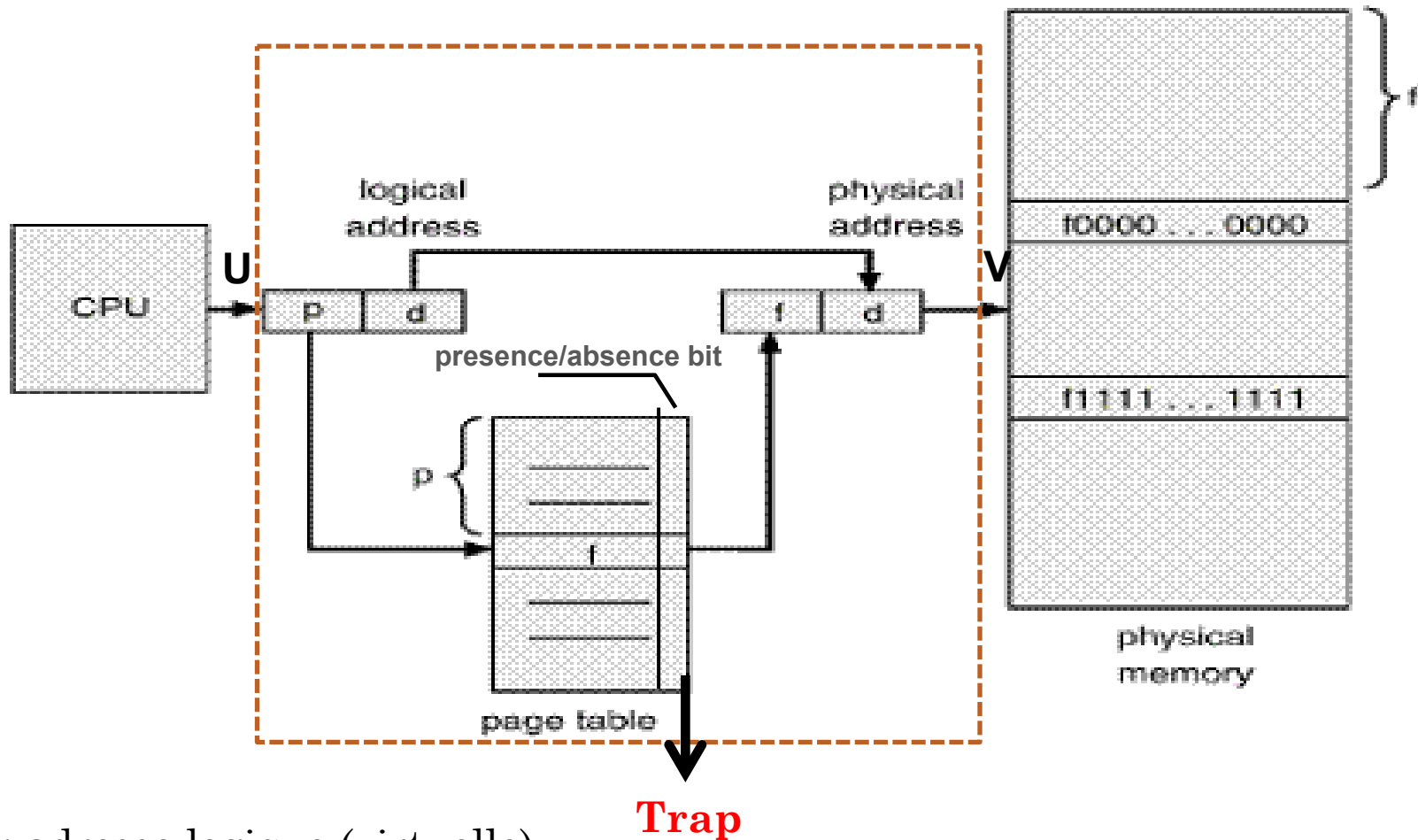
GESTION DE LA MÉMOIRE PRINCIPALE (MP)

PAGINATION – TRADUCTION D'ADRESSES (1^{ÈRE} MÉTHODE) (1/)

- 2) Recherche de la case correspondante à la page p .
 - a. Si la page n'est pas chargée en mémoire → Défaut de page.
 - b. Si la page est chargée en mémoire → Obtention du numéro de case (c) où p est chargée.
- 3) Détermination de l'adresse physique : $AP = (c, d)$
 - $AP = c * \text{taille case} + d$

GESTION DE LA MÉMOIRE PRINCIPALE (MP)

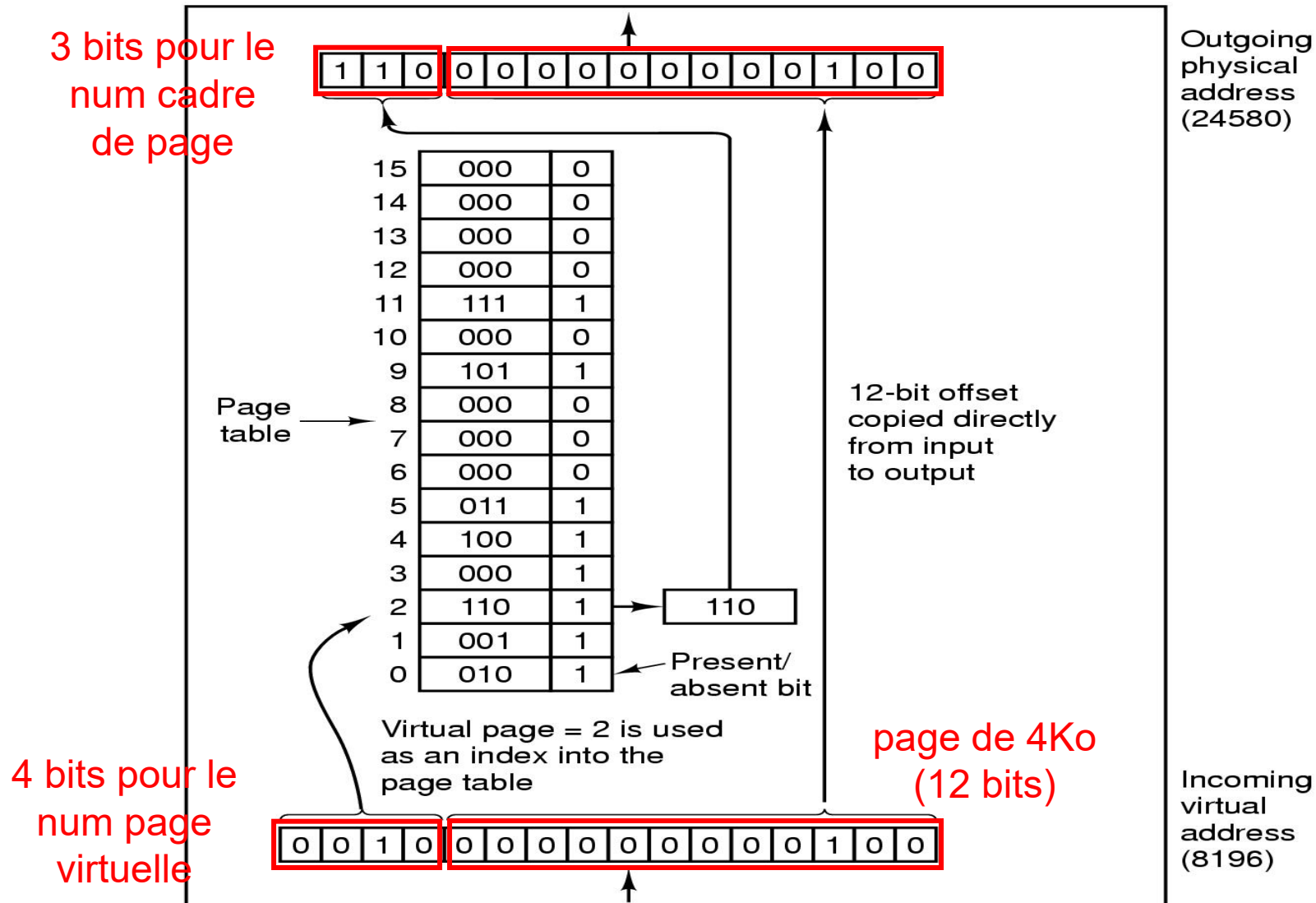
PAGINATION – TRADUCTION D'ADRESSES (2^{ÈME} MÉTHODE) (1/)



U : adresse logique (virtuelle)
V : adresse physique en MP

GESTION DE LA MÉMOIRE PRINCIPALE (MP)

PAGINATION – TRADUCTION D'ADRESSES (2^{ÈME} MÉTHODE) (1/)



GESTION DE LA MÉMOIRE PRINCIPALE (MP)

PAGINATION – DÉFAUT DE PAGE

- Que se passe-t-il si le programme essaye par exemple de faire appel à une page non présente ?
- **Défaut de page** : événement correspondant à la tentative d'accès à une adresse mémoire dans une page non chargée physiquement en MP.
- Le défaut de page génère un déroutement (interruption interne) qui donne la main au SE pour traiter le défaut de page.
- Le traitement du défaut de page fait appel à un algorithme de remplacement de page qui s'appuie sur une stratégie de remplacement (FIFO, LFU, LRU).

GESTION DE LA MÉMOIRE PRINCIPALE (MP)

PAGINATION – TRAITEMENT DE DÉFAUT DE PAGE

- « Trap » généré par la MMU → appel routine.
- La routine vérifie l'origine du trap (*défaut de page* / violation d'accès mémoire / ...).
- Sélectionner un cadre en mémoire (une page victime) pour logger la page manquante → Algorithme de Remplacement de page.
- « éventuel » swap out de la page victime.
- Swap in de la page demandée (...).
- Mise à jour de la table des pages.
- Reprise de l'exécution de l'instruction interrompue.

GESTION DE LA MÉMOIRE PRINCIPALE (MP)

PAGINATION – ALGORITHME DE REMPLACEMENT DE PAGE : FIFO

- **Algorithme FIFO**
- La page à remplacer est celle qui a été chargée en mémoire depuis longtemps (la plus anciennement chargée).
- La mise en œuvre est simple: Le SE utilise la structure de liste. Il enlève la page en tête de la liste et ajoute une page à la fin de la liste.
- Algorithme n'est pas optimal: la page la plus ancienne n'est pas forcément la moins utilisée.

GESTION DE LA MÉMOIRE PRINCIPALE (MP)

PAGINATION – ALGORITHME DE REMPLACEMENT DE PAGE : FIFO

○ Algorithme FIFO

- Ayant une MC de 3 cases (page frames) et les demandes d'allocations des pages selon l'ordre suivant: 4, 7, 3, 0, 1, 7, 3, 8, 5, 4, 5, 3, 4, 7.

	4	7	3	0	1	7	3	8	5	4	5	3	4	7
Cases de la MP	4	4	4	0	0	0	3	3	3	4	4	4	4	4
		7	7	7	1	1	1	8	8	8	8	3	3	3
			3	3	3	7	7	7	5	5	5	5	5	7
Défaut de pages	X	X	X	X	X	X	X	X	X	X		X		X

Nombre de défaut de pages = 12
Taux de défaut de pages = 12/14

GESTION DE LA MÉMOIRE PRINCIPALE (MP)

PAGINATION – ALGORITHME DE REMPLACEMENT DE PAGE : LRU

- *Algorithme LRU (Least Recently Used)*
- Cet algorithme suppose que si une page a été utilisée plusieurs fois récemment , elle va encore l'être à l'avenir et celle ignorée depuis longtemps le restera encore.
- La mise en œuvre de cet algorithme se base sur l'utilisation d'une table. À chaque page on associe un bit de marquage et un compteur.
- La mise à jour de cette table se fait à chaque nouvel intervalle de temps.
- *Inconvénients :*
 - Assez dure à mettre en œuvre
 - Coût élevé
- *Avantages:*
 - Des bons résultats
 - Assez utilisé

GESTION DE LA MÉMOIRE PRINCIPALE (MP)

PAGINATION – ALGORITHME DE REMPLACEMENT DE PAGE : LRU

- **Algorithme LRU (Least Recently Used)**
- Dans chaque intervalle :
 - Chaque fois qu'une page est référencée → Bit de marquage prend la valeur 1
 - À la fin de chaque intervalle de temps, le SE consulte la table de pages :
 - Si la page n'a pas été référencée, le compteur est incrémenté de 1.
 - Les compteurs des pages qui ont été référencées sont remis à 0.
 - Ensuite le bit de marquage est remis à 0 pour l'intervalle suivant.
- *La page à remplacer est celle qui a le compteur le plus élevé.*

Page virtuelle présente en MC

Bit de marquage

Compteur

GESTION DE LA MÉMOIRE PRINCIPALE (MP)

PAGINATION – ALGORITHME DE REMPLACEMENT DE PAGE : LRU

- Algorithme LRU (Least Recently Used)
- Ayant une MC de 3 cases (page frames) et les demandes d'allocations des pages selon l'ordre suivant: suivant: 1, 2, 3, 4, 2, 1, 5, 6, 2, 1, 2, 3, 7, 6, 3, 2, 1, 2, 3, 6.
- Première méthode : Méthode de pile (stack)

	1	2	3	4	2	1	5	6	2	1	2	3	7	6	3	2	1	2	3	6
Cases de la MP	1	2	3	4	2	1	5	6	2	1	2	3	7	6	3	2	1	2	3	6
		1	2	3	4	2	1	5	6	2	1	2	3	7	6	3	2	1	2	3
			1	2	3	4	2	1	5	6	6	1	2	3	7	6	3	3	1	2
Défaut de pages	X	X	X	X		X	X	X	X	X		X	X	X		X	X			X

Nombre de défaut de pages = 15

Taux de défaut de pages = 15/20

GESTION DE LA MÉMOIRE PRINCIPALE (MP)

PAGINATION – ALGORITHME DE REMPLACEMENT DE PAGE : LRU

○ Deuxième méthode : utilisation du compteur

	1	2	3	4	2	1	5	6	2	1	2	3	7	6	3	2	1	2	3	6	
	1	1	1	4	4	4	5	5	5	1	1	1	7	7	7	2	2	2	2	2	
		2	2	2	2	2	2	6	6	6	6	3	3	3	3	3	3	3	3	3	
			3	3	3	1	1	1	2	2	2	2	2	6	6	6	1	1	1	6	
1	0	1	2			0	1	2		0	1	2					0	1	2		
2		0	1	2	0	1	2		0	1	0	1	2				0	1	0	1	2
3			0	1	2							0	1	2	0	1	2	3	0	1	
4				0	1	2															
5							0	1	2												
6								0	1	2	3			0	1	2					0
7													0	1	2						
	X	X	X	X		X	X	X	X	X		X	X	X		X	X			X	

Cases de la MP

Compteurs

147

Défaut de pages

GESTION DE LA MÉMOIRE PRINCIPALE (MP)

PAGINATION – ALGORITHME DE REMPLACEMENT DE PAGE : LFU

- **Algorithme LFU (Least Frequently Used)**
- On remplace la page qui a été le moins souvent utilisée.
- Cet algorithme suppose que la page la moins utilisée dans le présent n'est pas celle qui le sera dans le futur.
- La mise en œuvre se fait par une table : à chaque page on associe un bit d'utilisation et un compteur.
- À chaque référencement de page :
 - Si pas de défaut de page alors
 - On ajoute 1 au compteur de la page référencée de nouveau (qui existe déjà dans la M)
 - Si défaut de page :
 - On attribut 1 au compteur de la page référencée (insérée)
 - On remet à 0 le compteur de la page expulsée
- *La page à remplacer est celle qui a le compteur le moins élevé.*

GESTION DE LA MÉMOIRE PRINCIPALE (MP)

PAGINATION – ALGORITHME DE REMPLACEMENT DE PAGE : LFU

	8	0	1	2	0	3	0	4	2	3	0	3	2	1	2
	8	8	8	2	2	2	2	4	4	3	3	3	3	3	3
		0	0	0	0	0	0	0	0	0	0	0	0	0	0
			1	1	1	3	3	3	2	2	2	2	2	1	2
0		1	1	1	2	2	3	3	3	3	4	4	4	4	4
1			1	1	1	0								1	0
2				1	1	1	1	0	1	1	1	1	2	0	1
3						1	1	1	0	1	1	2	2	2	2
4								1	1	0					
8	1	1	1	0											

Cases de la MP

Compteurs des fréquences

X X X X X X X X X X X X

Défaut de pages

149

Nombre de défaut de pages = 10

Taux de défaut de pages = 10/15

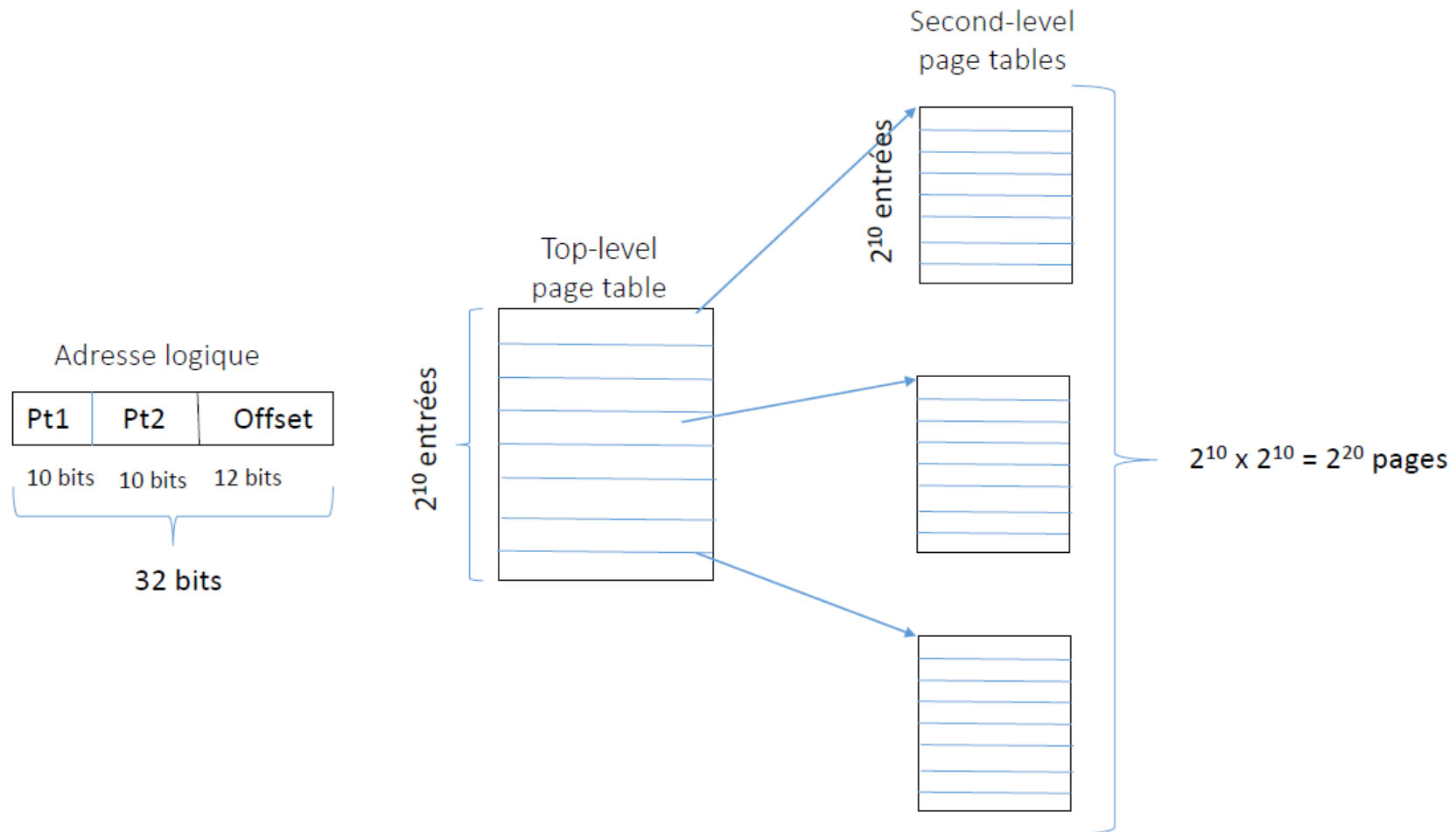
GESTION DE LA MÉMOIRE PRINCIPALE (MP)

PAGINATION MULTINIVEAUX

- La plupart des SE modernes supportent un espace logique très grand (table de pages excessivement grande).
- Solution : ***subdiviser les pages***
 - Pagination à 2 niveaux
 - Généralisation : plusieurs niveaux
- Performance: Chaque niveau est stocké comme une table séparée en mémoire.
- Ainsi, la conversion d'une adresse logique à une adresse physique peut demander des accès mémoire supplémentaires.
- Mémoire cache permet une performance raisonnable: un registre du CPU contient l'adresse de la table du niveau le plus haut.

GESTION DE LA MÉMOIRE PRINCIPALE (MP)

PAGINATION MULTINIVEAUX



GESTION DE LA MÉMOIRE PRINCIPALE (MP)

PAGINATION – INCONVÉNIENTS

- ***Fragmentation interne*** : désigne l'espace non utilisé au sein d'une page. Une page placée en mémoire occupe une case entière même si elle ne comporte que 2 adresses affectées.
- ***La taille de la table des pages est énorme*** : si la page est de taille = 4 ko et la Mémoire virtuelle = 4 Go (bus d'adresses de 32 bits) → Le nombre de pages = $4 \text{ Go} / 4 \text{ ko} = \text{environ un million de pages} = 1 \text{ M pages}$.
 - Une table de pages contenant un million d'entrée est une structure de données gigantesque → La taille sera pénalisante lors du parcours. Si une entrée nécessite 4 octets, 4 Mo octets sont nécessaires pour charger cette table.
- ***Le partage de code est difficile à mettre en œuvre dans un système paginé***. Le contenu d'une page est généralement inconnu du programmeur.

GESTION DE LA MÉMOIRE PRINCIPALE (MP)

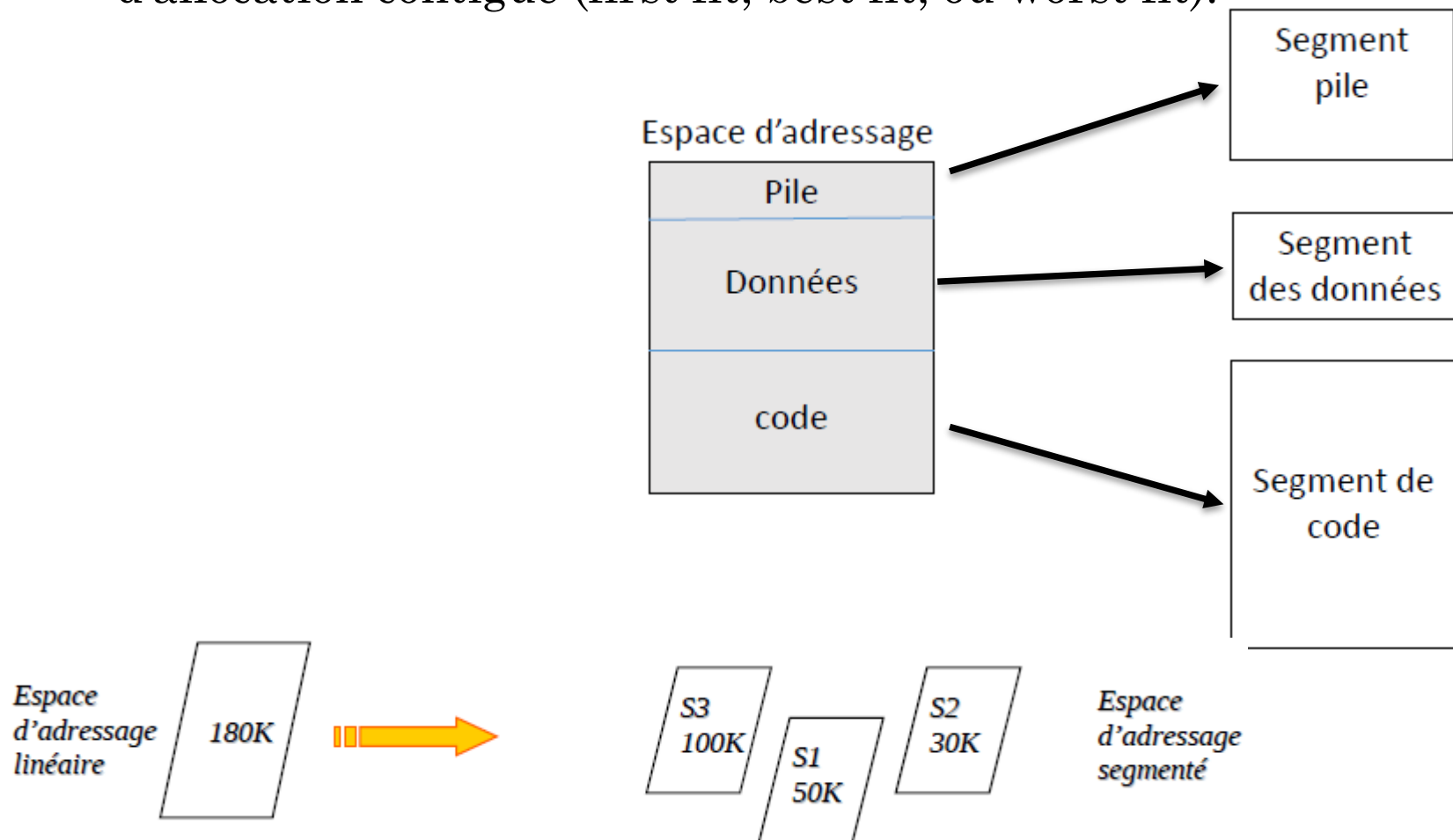
SEGMENTATION

- La segmentation est analogue à la pagination sauf que la taille d'un segment est variable.
- L'espace d'adressage logique est divisé en un ensemble de segments.
- Un segment étant un espace d'adressage linéaire formé d'adresses contiguës.
- ***Segment*** = une partie logique du programme : segment de code, segment de données, et segment de pile.
- *Intérêt* : faciliter la gestion par les processus de leur espace propre (exemple le partage).
- Chaque segment possède un numéro et une longueur.
- Table de segments : pour chaque segment correspond une pair de valeur (base, limite).

GESTION DE LA MÉMOIRE PRINCIPALE (MP)

SEGMENTATION

- L'Allocation de l'espace mémoire applique un algorithme d'allocation contiguë (first fit, best fit, ou worst fit).



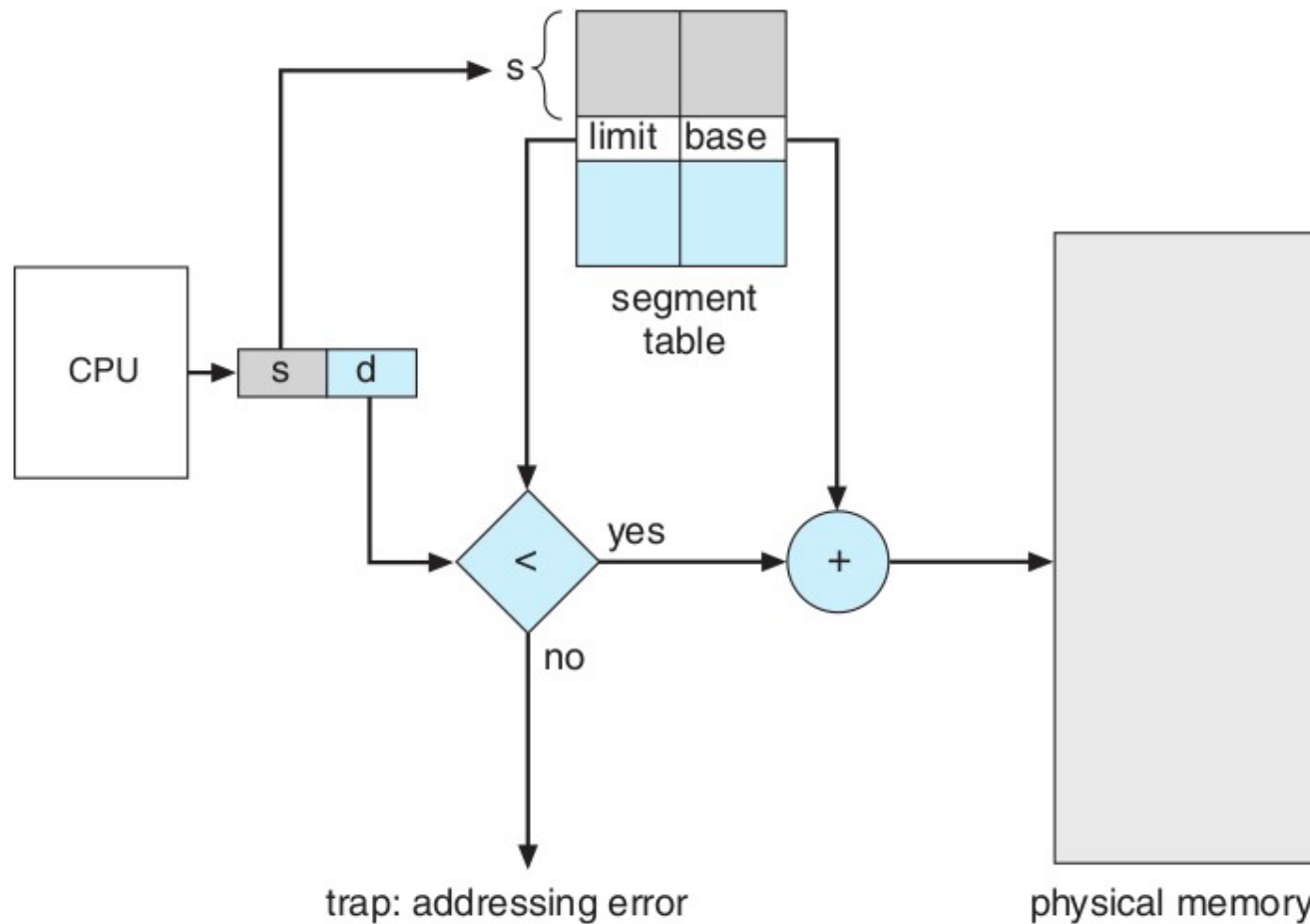
GESTION DE LA MÉMOIRE PRINCIPALE (MP)

SEGMENTATION – TRADUCTION D'ADRESSES

- 1) Détermination de l'adresse logique : $AL = (s, d)$
 - s : numéro du segment
 - d : déplacement dans le segment
- 2) Recherche du couple (base, limite) correspondant au segment s .
 - a. Si le déplacement d est supérieur à la limite $\rightarrow$ Trap : erreur d'adressage.
 - b. Si le déplacement d est inférieur à la limite $\rightarrow$ Obtention de la base (b).
- 3) Détermination de l'adresse physique : $AP = (b, d)$
 - $AP = b + d$

GESTION DE LA MÉMOIRE PRINCIPALE (MP)

SEGMENTATION – TRADUCTION D'ADRESSES



GESTION DE LA MÉMOIRE PRINCIPALE (MP)

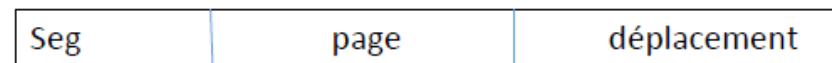
PAGINATION VS SEGMENTATION

Considérations	Pagination	Segmentation
Le programmeur doit-il connaître la technique utilisée?	NON	OUI
Combien y a-t-il d'espaces d'adressage linéaires?	1	Plusieurs
L'espace total d'adressage peut-il dépasser la taille de la mémoire physique?		OUI
Les procédures et les données peuvent-elles être séparées et protégées séparément?	NON	OUI
Peut-on traiter facilement des tables dont les tailles varient?	NON	OUI
Le partage de procédures entre utilisateurs est-il simplifié?	NON	OUI
Pourquoi cette technique a-t-elle été inventée ?	Pour obtenir un grand espace d'adressage linéaire sans avoir à acheter de la mémoire physique	Pour permettre la séparation des programmes et des données dans des espace d'adressage logiquement indépendants et pour faciliter le partage et la protection

GESTION DE LA MÉMOIRE PRINCIPALE (MP)

SEGMENTATION PAGINÉE

- La segmentation paginée :
 - Chaque segment sera paginé.
 - Autrement dit, le champ déplacement d du couple (s, d) de l'adresse virtuelle sera interprété comme un numéro de page et un déplacement (p, d').
 - (s, d) = (s, p, d').



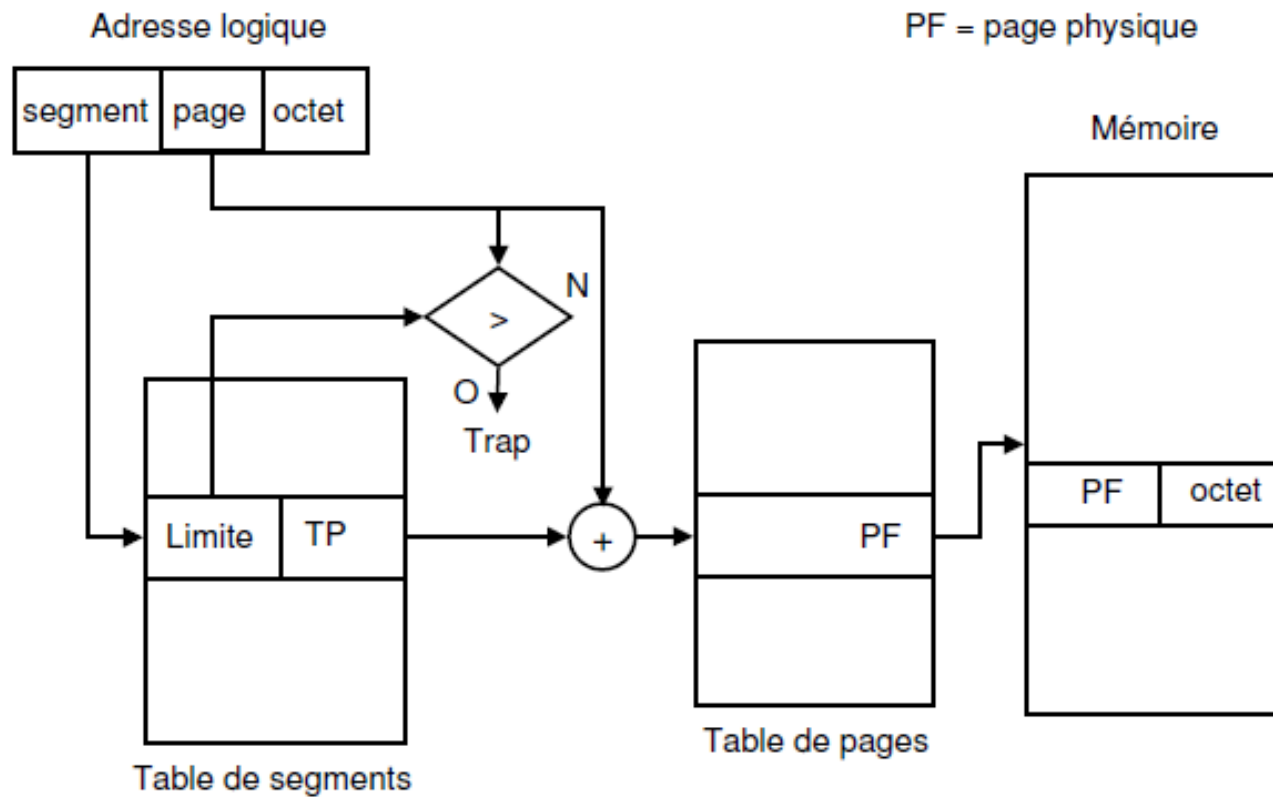
Adresse logique

32bit = 10 bits + 10 bits + 12 bits.



GESTION DE LA MÉMOIRE PRINCIPALE (MP)

SEGMENTATION PAGINÉE



GESTION DE LA MÉMOIRE PRINCIPALE (MP)

RÉSUMÉ

- L'allocation en **partitions** considère le programme comme un ensemble d'adresses insécables.
 - Problème de fragmentation nécessite d'une opération de compactage de la mémoire centrale.
- La **pagination** découpe l'espace d'adressage du programme en pages et la mémoire physique en cases de même taille.
 - Une adresse générée par le processeur est de la forme (N° page, déplacement).
 - La table des pages du processus permet de traduire l'adresse paginée en adresse physique.
- La **segmentation** découpe l'espace d'adressage du programme en segments correspondant à des morceaux logiques du programme.
 - Une adresse générée par le processeur est de la forme (N° segment, déplacement).
 - La table des segments du processus permet de transformer l'adresse segmentée en adresse physique.
- Segmentation et pagination sont très souvent associées.